

Appunti di metodi numerici

l'arte della programmazione

Nicolò Mone

Qui di solito ci va la common creative license ma chi ce l'ha e chi se ne fotte

Qualcosa di commovente

Introduzione

Pisa, 27 aprile 2026

Nicolò Mone

Indice

Introduzione	5
1 Numeri casuali	11
1.0.1 Introduzione	11
1.0.2 Generatori congruenziali lineari	12
1.0.3 Combinazione di generatori	13
1.0.4 Generatori basati su sistemi caotici	14
1.0.5 Generatori come sistemi dinamici a molti stati	15
1.1 Generazione di distribuzioni non uniformi	15
1.1.1 Dal caso uniforme a una distribuzione arbitraria	15
1.1.2 Metodo della funzione cumulativa inversa	16
1.1.3 Trasformazioni in più dimensioni	17
1.1.4 Generazione di distribuzioni gaussiane: metodo di Box-Muller	18
1.2 Rejection method	19
1.2.1 Metodo del rifiuto per la distribuzione Gamma	20
1.3 Integrazione Monte Carlo	22
2 Ricerca degli zeri di una funzione	29
2.1 Introduzione	29
2.2 Metodo di bisezione	30
2.2.1 Osservazioni sul metodo di bisezione	31
2.3 Metodo di Newton-Raphson	32
2.3.1 Analisi dell'errore	32
2.3.2 Osservazioni sul metodo di Newton-Raphson	33
2.4 Caso dei polinomi, metodo di Laguerre	33
2.5 Generalizzazione a più dimensioni di Newton-Raphson	34
2.6 Conclusioni	35
3 Analisi numerica nell'integrazione	37
3.1 Introduzione	37
3.2 Formule di Newton-Cotes su intervallo singolo	38
3.2.1 Alcune formule locali	39
3.3 Intervalli estesi	39
3.3.1 Valutazione dell'errore e formula esatta dei trapezi	42
3.3.2 Integrazione di Romberg	42
3.3.3 Quadrature Gaussianhe	44
3.3.4 Integrali impropri	46

4	Integrazione numerica delle Equazioni Differenziali Ordinarie (ODE)	49
4.0.1	Metodi multi-step	50
4.0.2	Metodi one-step	55
4.0.3	Equazioni stiff	60
4.0.4	Boundary Value Problems (BVP)	61
5	Integrazione numerica di equazioni differenziali alle derivate parziali (PDEs)	63
5.1	Introduzione	63
5.2	Differenze finite	64
5.3	Flux-conservative PDEs	65
5.3.1	Forward Time, Centered Space	66
5.3.2	Metodo di Lax	67
5.3.3	Metodo staggered leap-frog	67
5.3.4	Metodo two-step Lax-Wendroff	67
5.3.5	Schroedinger e operatori split	68
5.4	Diffusive initial value problems	68
5.5	Metodi di Fourier	68
5.6	Cyclic Reduction	68
5.7	Metodi di rilassamento	68
5.7.1	Una soluzione pratica	69
6	Metodo delle differenze finite	73
7	Algoritmo di Thomas	77
8	Metodo di Jacobi	81
8.1	Introduzione	81
8.1.1	Struttura della matrice di iterazione	81
8.1.2	Procedimento iterativo	82
8.1.3	Convergenza del metodo	82
8.2	Condizione per la convergenza	83
8.3	Flusso illimitato	86
8.4	Come funziona il metodo di Jacobi	87
8.5	Implementazione per equazione di diffusione	88
8.6	Velocità di convergenza	88
9	Gauss-Siedel, SOR & LOR	91
9.1	Metodo di Gauss-Seidel	91
9.2	Metodo di Successive Overrelaxation (SOR)	93
9.2.1	Matrice di iterazione del metodo SOR	93
9.2.2	Applicazione pratica	94
9.3	Implementazione pratica del metodo SOR	95
9.4	Strategia iterativa Red & Black	95
9.5	Metodo Line Overrelaxation (LOR)	96
10	Metodo alternate direction implicit (ADI)	99
10.1	La matrice di iterazione in ADI	99
10.1.1	Procedura iterativa a due passi	99

10.1.2	Schema iterativo	100
10.1.3	Struttura delle matrici	100
10.1.4	Risoluzione delle semi-iterazioni	101
10.1.5	Matrice di iterazione del metodo ADI	102
10.1.6	Convergenza del metodo ADI	102
10.2	ADI per problemi separabili	103
10.2.1	Raggio spettrale della matrice di iterazione $\mathbf{J}_\omega$	104
10.2.2	Scelta ottimale dei parametri ω	104
10.3	Considerazioni finali	106
11	Metodi del gradiente	107
11.1	Formulazione variazionale del problema	107
11.2	Interpretazione tramite errore	108
11.2.1	Relazione tra errore e funzionale	108
11.3	Metodi di discesa	109
11.3.1	Algoritmo del Steepest Descent Method	112
11.3.2	Interpretazione geometrica	112
11.4	Conjugate Gradient Method	114
12	Metodi del sottospazio di Krylov	117
12.0.1	Contesto generale	117
12.1	Sistema di equazioni lineari con matrice generale $\mathbf{A}$	117
12.2	Approssimazione di $\mathbf{A}^{-1}$	118
12.3	Il sottospazio di Krylov	118
12.3.1	Interpretazione	118
12.3.2	Forma del residuo	118
12.3.3	Metodo di Krylov	119
12.4	Da Conjugate Gradients a GMRES	119
12.4.1	Idea comune	119
12.5	Metodo Generalized Minimal RESidual (GMRES)	120
12.6	Metodo di Arnoldi	120
12.6.1	Procedura	120
12.6.2	Forma matriciale	121
12.7	Metodo GMRES	121
12.7.1	Minimizzazione del residuo	122
12.8	GMRES con Riavvio (Restarted GMRES)	123
12.9	Precondizionamento	123
12.9.1	Scelte comuni	124
12.9.2	Differenze tra preconditionamento a sinistra e a destra	124
12.9.3	Esempi di preconditionamento	124
13	Accelerazione del Metodo delle Potenze: Metodo di Wielandt	127
13.1	Convergenza del Metodo delle Potenze	127
13.1.1	Spostamento Spettrale	127
13.2	Metodo di Wielandt	128
13.2.1	Osservazioni Finali	129
14	Accelerazione del Metodo delle Potenze: Metodo di Chebyshev	131
14.1	Polinomi Matriciali e Accelerazione	131

14.1.1	Applicazione dei Polinomi di Chebyshev	132
14.1.2	Minimizzazione del Residuo	132
14.1.3	Implementazione Iterativa	133
14.1.4	Confronti e Stima del Dominance Ratio	134
15	Metodi dei Residui Pesati e Applicazioni ai Calcoli di Core	135
15.1	Introduzione al Metodo dei Residui Pesati	135
15.2	Formulazione del Metodo dei Residui Pesati	135
15.3	Scelta delle Funzioni Peso	136
15.3.1	Metodo di Galerkin	136
15.3.2	Metodo dei Sottodomini	136
15.3.3	Metodo di Collocazione	137
15.4	Applicazione al Calcolo del Core del Reattore	138
15.5	Applicazione del Metodo dei Sottodomini	138
15.5.1	Formulazione del Bilancio Neutronico	138
15.5.2	Integrazione del Termine di Assorbimento e Sorgente	139
15.5.3	Equazione Discretizzata	139
15.5.4	Condizioni di Continuità della Corrente	139
15.5.5	Equazione a 7 Punti	139
15.6	Metodo CUBBOX	140
15.6.1	Approssimazione del Flusso con Polinomi di Terzo Ordine	140
15.6.2	Determinazione dei Coefficienti c_x, c_y, c_z	140
15.7	Conclusione	140
16	Approssimanti di Padé	141
16.1	Introduzione	141
16.2	Applicazione alla funzione esponenziale	141
16.2.1	Approssimante $R_{(0,1)}(\theta)$	141
16.2.2	Approssimante $R_{(1,0)}(\theta)$	142
16.2.3	Approssimante $R_{(1,1)}(\theta)$	142

Capitolo 1

Numeri casuali

1.0.1 Introduzione

In molte applicazioni di analisi numerica è necessario disporre di sequenze di numeri casuali. In realtà i computer non producono numeri casuali, generano sequenze *pseudo-casuali*, cioè successioni determinate da algoritmi deterministici che imitano alcune proprietà statistiche della casualità.

In linea di principio esistono algoritmi che sfruttano sorgenti fisiche per produrre numeri "realmente" (?) casuali, ma nella maggior parte delle applicazioni numeriche si utilizzano generatori pseudocasuali.

Un generatore di numeri casuali deve produrre sequenze che siano, almeno dal punto di vista statistico, il più possibile indipendenti. Poiché l'algoritmo che produce tali numeri è deterministico, l'obiettivo è costruire procedure che simulino bene il comportamento di una successione indipendente.

Un punto importante è che il grado di "casualità" richiesto dipende fortemente dall'applicazione. In alcune simulazioni fisiche o nei metodi Monte Carlo sono richieste proprietà statistiche molto rigorose.

Un riferimento classico sull'argomento è

D. E. Knuth, *Seminumerical Algorithms*, volume 2 di *The Art of Computer Programming*, Addison-Wesley (1981).

Considerazioni sui generatori Quando si utilizza un generatore di numeri pseudocasuali è sempre opportuno mantenere un atteggiamento critico nei confronti dei risultati prodotti. In altre parole, bisogna sempre controllare che il generatore non introduca correlazioni indesiderate o artefatti statistici.

Un criterio empirico utile è il seguente: se due generatori diversi producono risultati significativamente differenti per lo stesso esperimento numerico, è probabile che almeno uno dei due presenti problemi.

"Be suspicious. Once you've been suspicious, stop and be suspicious again."

- Riccardo Mannella

1.0.2 Generatori congruenziali lineari

Uno dei generatori più elementari e diffusi è il *linear congruential generator*. La sequenza viene costruita tramite la relazione ricorsiva:

$$x_{n+1} = ax_n + c \pmod{m}.$$

Qui a è il moltiplicatore, c è l'incremento, m è il modulo. Scelti a , c , m ed un 'seed' x_n , la sequenza è continua da sé in modo del tutto deterministico.

Questo tipo di generatore ha il vantaggio di essere molto veloce e di poter essere implementato con grande efficienza anche a basso livello.

Tuttavia presenta alcuni limiti importanti. Le sequenze generate possono mostrare correlazioni interne. Innanzitutto perché la successione può assumere solamente valori fra 0 ed $m - 1$

In particolare, se si rappresentano graficamente i punti $(x_n, x_{n+1}, \dots, x_{n+k-1})$ in uno spazio k -dimensionale, si osserva che tali punti non riempiono uniformemente lo spazio, ma si dispongono su un insieme di iperpiani.

Più precisamente, i punti generati occupano uno spazio di dimensione $k - 1$ e il numero massimo di tali piani è al più $m^{1/k}$, per questo si scelgono sempre alti valori di m per non vedere la periodicità.

Per esempio, scegliendo $m = 2^{32}$, e $k = 3$, si ottiene $m^{1/k} \approx 1600$.

Altri esempi con $k = 3$:

- per $a = 421$, $c = 1663$, e $m = 7875$, 100 punti e $m^{1/k} \approx 19.9$;
- per $a = 4096$, $c = 150889$, e $m = 714025$, 10'000 punti e $m^{1/k} \approx 63.8$.

Questo significa che i punti tridimensionali generati dal metodo si dispongono su un numero relativamente limitato di piani, evidenziando quindi una struttura geometrica non desiderata nella sequenza pseudocasuale.

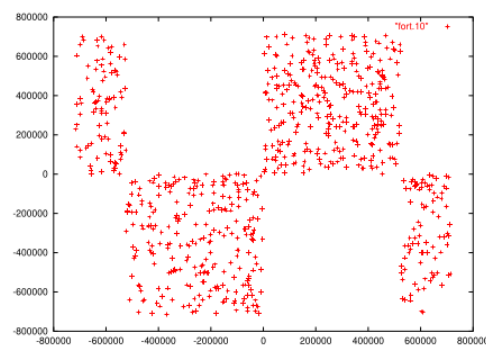
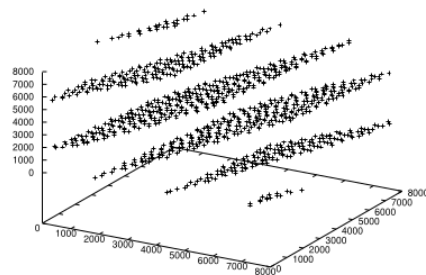


FIG. 1.1: Come si dispongono i punti della successione pseudo-casuale? si nota l'emergere di iperpiani affini che indicano correlazione statistica. La rappresentazione è tridimensionale perché $k = 3$. Gli assi rappresentano i punti $(x_n, x_{n+1}, x_{n+3-1})$.

La rappresentazione grafica mostra chiaramente che i punti non riempiono uniformemente lo spazio tridimensionale, ma si dispongono su un insieme relativamente piccolo di piani.

1.0.3 Combinazione di generatori

Una possibile strategia per migliorare le proprietà statistiche dei generatori congruenziali consiste nel combinare più generatori tra loro.

Un esempio è la routine RAN1.F, che combina tre metodi congruenziali differenti. In particolare, vengono utilizzati generatori distinti per i bit più significativi, per quelli meno significativi e per una procedura di *reshuffling* della sequenza.

Se la velocità di esecuzione è il fattore principale, è possibile utilizzare una combinazione più semplice di due generatori congruenziali. Questa strategia è adottata nella routine RAN2. Tuttavia, in questo caso il generatore tende a campionare una porzione più limitata dello spazio delle configurazioni.

```

1  !!! RAN1.F90
2  ! questa è solo una routine di esempio
3  REAL FUNCTION ran1(IDUM)
4      ! Generatore di numeri pseudo-casuali uniformi in (0,1)
5      ! Se IDUM < 0 oppure se è la prima chiamata, la routine
6      ! inizializza lo stato interno del generatore.
7      implicit none
8
9      integer, intent(inout) :: idum
10     integer :: j
11     integer, save :: iff = 0
12     integer, save :: ix1, ix2, ix3
13     real, save :: r(97)
14     ! Queste variabili devono conservare il loro valore tra
15     chiamate successive alla funzione.
16     ! Parametri (m, a, c) dei tre generatori congruenziali:
17     integer, parameter :: m1 = 259200, ia1 = 7141, ic1 = 54773
18     integer, parameter :: m2 = 134456, ia2 = 8121, ic2 = 28411
19     integer, parameter :: m3 = 243000, ia3 = 4561, ic3 = 51349
20     real, parameter :: rm1 = 3.8580247e-6
21     real, parameter :: rm2 = 7.4373773e-6
22
23     IF (idum < 0 .OR. iff = 0) THEN
24         iff=1
25         ix1=mod(ic1-idum,m1)
26         ix1=mod(ia1*ix1+ic1,m1)
27         ix2=mod(ix1,m2)
28         ix1=mod(ia1*ix1+ic1,m1)
29         ix3=mod(ix1,m3)
30         DO j=1, 97
31             ix1=mod(ia1*ix1+ic1,m1)
32             ix2=mod(ia2*ix2+ic2,m2)
33             r(j)=(REAL(ix1)+REAL(ix2)*rm2)*rm1
34         CONTINUE
35         idum=1
36     ENDIF

```

```

37     ix1=mod(ia1*ix1+ic1,m1)  ! Aggiornamento dei tre
      generatori congruenziali
38     ix2=mod(ia2*ix2+ic2,m2)
39     ix3=mod(ia3*ix3+ic3,m3)
40
41     j=1+(97*ix3)/m3 ! Selezione dell'elemento della tabella di
      shuffle
42
43     IF(j > 97 .OR. j < 1) then
44         stop "Errore in ran1: indice fuori intervallo"
45     ENDIF
46     ! Controllo di sicurezza: j dovrebbe sempre stare tra 1 e
      97.
47
48     ran1=r(j)
49     ! Il numero pseudo-casuale restituito è l'elemento j-esimo
      della tabella di shuffle.
50
51     ! L'elemento appena estratto viene sostituito con un nuovo
      valore costruito dai due generatori ix1 e ix2.
52     ! Questo è il cuore del meccanismo di shuffle.
53     r(j)=(REAL(ix1)+REAL(ix2)*rm2)*rm1
54     RETURN
55
56 END

```

Listing 1.1: RAN1.f90, combinazione di generatori congruenziali, esempio ispirato al "*Numerical Recipes*" di Press, Teukolsky, Vetterling, Flannery. Nota bene: questa è solamente una routine, per essere utilizzata va chiamata

1.0.4 Generatori basati su sistemi caotici

Un approccio alternativo consiste nello sfruttare il comportamento caotico di alcuni sistemi dinamici.

L'idea è quella di utilizzare un sistema dinamico deterministico che presenti una dinamica caotica controllata, e impiegare la successione dei suoi stati come sequenza pseudocasuale.

Un esempio classico è dato dalla mappa logistica:

$$x_{n+1} = \lambda x_n (1 - x_n) \quad (1.1)$$

Al variare del parametro λ il sistema in eq.(1.1) mostra un comportamento sempre più complesso, fino a entrare in regime caotico (come si vede in figura).

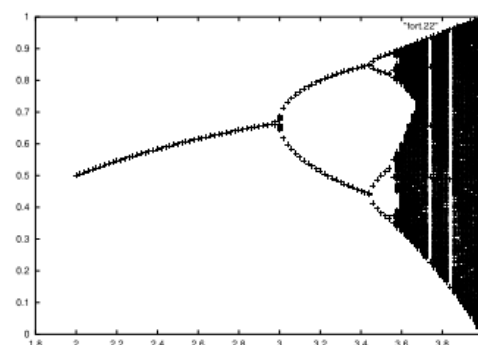


FIG. 1.2: La [curva logistica](#), sembra di essere tornati a [lezione da Paolini](#), wow

Tuttavia l'utilizzo diretto di sistemi caotici come generatori pseudocasuali presenta diversi problemi pratici. In particolare, è necessario stabilire quanto il comportamento caotico del sistema sia effettivamente adeguato per simulare una sequenza statisticamente indipendente.

1.0.5 Generatori come sistemi dinamici a molti stati

Un modo più efficace per costruire generatori pseudocasuali consiste nel progettare sistemi dinamici con un numero elevato di stati interni.

L'idea è quella di costruire una macchina a molti stati in cui il valore successivo della sequenza dipende da una combinazione di stati precedenti. Questo schema può essere rappresentato come una rete di variabili interne che interagiscono tra loro attraverso operazioni di somma o sottrazione.

Questa idea è alla base dei cosiddetti algoritmi *add and carry* oppure *subtract and carry*. Tra i generatori di questo tipo si trovano, ad esempio,

- la routine **RAN3**: si mantiene una tabella di numeri precedenti e si genera il nuovo valore come differenza di due elementi della sequenza (In forma semplificata: $x_n = x_{nj} - x_{nk}$);
- il generatore **rcarry**: questo è un generatore della famiglia subtract-with-carry. È simile a quello precedente ma introduce una variabile aggiuntiva chiamata carry (l'idea è: $x_n = x_{nr} - x_{ns} - c$).

Questi algoritmi presentano in genere periodi di ripetizione molto lunghi e buone proprietà statistiche (risolvendo parzialmente il problema della struttura geometrica in iperpiani).

1.1 Generazione di distribuzioni non uniformi

1.1.1 Dal caso uniforme a una distribuzione arbitraria

Dopo aver costruito un generatore di numeri pseudo-casuali uniformi, il problema successivo consiste nel produrre variabili aleatorie con una distribuzione assegnata. In altre parole, dato un generatore che fornisce variabili uniformi, vogliamo ottenere campioni distribuiti secondo una densità prefissata.

Sia dunque x una variabile casuale distribuita uniformemente in un certo intervallo, con densità $p(x)$, e supponiamo di voler costruire una nuova variabile y con densità $p(y)$. La relazione fondamentale tra le due densità si ottiene imponendo l'invarianza della probabilità sotto trasformazione di variabile. Si ha infatti $|p(y) dy| = p(x) dx$, da questa relazione segue immediatamente che:

$$p(y) = p(x) \left| \frac{dx}{dy} \right|.$$

Questa formula esprime il fatto che la densità della nuova variabile dipende dalla densità di partenza e dal modo in cui la trasformazione $x \mapsto y$ dilata o contrae gli intervalli.

Il problema centrale diventa allora quello di determinare una trasformazione opportuna tra x e y . Ciò si riconduce a risolvere $\frac{dx}{dy} = f(y)$.

In sostanza, assegnata la distribuzione voluta, si cerca una funzione di trasformazione tale che la variabile uniforme iniziale venga mappata nella nuova variabile con la densità desiderata.

1.1.2 Metodo della funzione cumulativa inversa

Il procedimento precedente si chiarisce riscrivendo la trasformazione in termini di funzione integrale della densità.

Ricordando la relazione $\frac{dx}{dy} = f(y)$ ed integrando rispetto a y , si ottiene

$$x = F(y) ,$$

dove la funzione $F(y)$ è data, a meno di costanti additive o di opportune normalizzazioni, dall'integrale della densità:

$$F(y) = \int p(y) dy .$$

Se la funzione F è invertibile, allora si può esprimere y in funzione di x come

$$y(x) = F^{-1}(x).$$

Questo è il contenuto essenziale del metodo della trasformazione inversa: si genera una variabile uniforme x , si costruisce la funzione cumulativa associata alla densità desiderata, e infine si ottiene la variabile distribuita secondo $p(y)$ applicando l'inversa della cumulativa.

Il metodo funziona quindi molto bene quando l'inversa dell'integrale di $p(y)$ è disponibile in forma esplicita, oppure può essere calcolata numericamente in modo efficiente.

Esempio: distribuzione esponenziale

Un esempio particolarmente importante è quello della distribuzione esponenziale. Si consideri la trasformazione

$$y(x) = -\ln x.$$

Se x è uniforme nell'intervallo $(0, 1)$, allora

$$\frac{dx}{dy} = -e^{-y}, \quad \left| \frac{dx}{dy} \right| = e^{-y}.$$

Poiché la densità di x è uniforme, cioè costante su $(0, 1)$, dalla relazione generale

$$p(y) dy = p(x) \left| \frac{dx}{dy} \right| dy$$

segue che, a meno della costante uniforme $p(x)$, la densità di y è

$$p(y) dy = e^{-y} dy.$$

Si ottiene dunque la distribuzione esponenziale.

Interpretazione geometrica del metodo

Ripartiamo da $F(y) = \int p(y) dy$; si osserva che la variabile uniforme x viene ottenuta come valore della cumulativa associata a y . Invertendo questa relazione, si ha ancora

$$y(x) = F^{-1}(x).$$

Geometricamente, la funzione $F(y)$ cresce dal valore minimo al valore massimo della probabilità cumulata. Se si estrae un numero uniforme x tra 0 e 1, lo si può interpretare come un livello di probabilità cumulata; il valore corrispondente di y si ottiene leggendo sull'asse orizzontale il punto in cui la cumulativa raggiunge proprio quel livello.

In altre parole, il metodo della trasformazione inversa trasforma una distribuzione uniforme della probabilità cumulata in una distribuzione non uniforme della variabile fisica.

Esempio: tempi di decadimento

Sappiamo che quando i nuclei decadono hanno una densità di probabilità $p(t) = \lambda e^{-\lambda t}$ di decadere, nel senso che $p(t) dt$ è la probabilità di decadere nell'intervallo di tempo t , $t + dt$; inoltre sappiamo che la cumulativa di questa probabilità,

$$F(t^*) = P(T < t^*) = \int_0^{t^*} p(T) dT = 1 - e^{-\lambda t^*}$$

ci dà invece la probabilità che il nucleo sia decaduto prima dell'istante t^* .

Partiamo quindi dalla probabilità accumulata ed scegliamo un valore $u \in [0, 1]$, per poi porre l'uguaglianza $1 - e^{-\lambda t^*} = u$ che invertita restituisce $t^* = -\frac{1}{\lambda} \log(1 - u)$. Questo passaggio equivale ad estrarre una percentuale di nuclei decaduti e a chiedersi 'quanto tempo t^* si dovrebbe aspettare per vedere una frazione u di nuclei decadere?'.

Nonostante il processo sia poissoniano ed ogni atomo decada dopo un tempo diverso, si può continuare ad estrarre valori u quanti sono gli atomi e questo ci permette trovando tutti i tempi corrispondenti di ricostruire la distribuzione esponenziale iniziale.

1.1.3 Trasformazioni in più dimensioni

Il metodo di trasformazione non è limitato al caso unidimensionale, ma si estende anche a distribuzioni in più variabili.

Se $(x_1, x_2, \dots)$ è un vettore di variabili di partenza e $(y_1, y_2, \dots)$ è il nuovo vettore di variabili, allora la relazione tra le densità è regolata dal determinante jacobiano della trasformazione:

$$p(y_1, y_2, \dots) dy_1 dy_2 \dots = \left| \frac{\partial(x_1, x_2, \dots)}{\partial(y_1, y_2, \dots)} \right| p(x_1, x_2, \dots) dy_1 dy_2 \dots$$

Questa espressione generalizza direttamente la formula unidimensionale.

Anche qui il significato è lo stesso: la densità si modifica in funzione della deformazione locale dei volumi elementari operata dalla trasformazione.

1.1.4 Generazione di distribuzioni gaussiane: metodo di Box-Muller

Un'applicazione molto importante del caso multidimensionale è la generazione di variabili gaussiane: il metodo di [Box-Muller](#), che permette di ottenere due variabili gaussiane indipendenti a partire da due variabili uniformi indipendenti.

Si considerino due variabili uniformi indipendenti x_1 e x_2 , entrambe distribuite uniformemente in $(0, 1)$. Si definiscono allora le nuove variabili

$$y_1 = \sqrt{-2 \ln x_1} \cos(2\pi x_2) = r \cos \theta,$$

$$y_2 = \sqrt{-2 \ln x_1} \sin(2\pi x_2) = r \sin \theta.$$

Queste formule costituiscono la trasformazione di Box-Muller. Le due variabili uniformi x_1 e x_2 vengono reinterpretate come variabili ausiliarie che costruiscono, rispettivamente, il raggio e l'angolo in coordinate polari nel piano delle variabili gaussiane. In particolare, $r = \sqrt{-2 \ln x_1}$ e $\theta = 2\pi x_2$.

Questa trasformazione è progettata in modo tale che la densità congiunta di (y_1, y_2) risulti gaussiana e radialmente simmetrica, così da avere una routine che trasformi due variabili indipendenti distribuite uniformemente in due variabili distribuite gaussianamente.

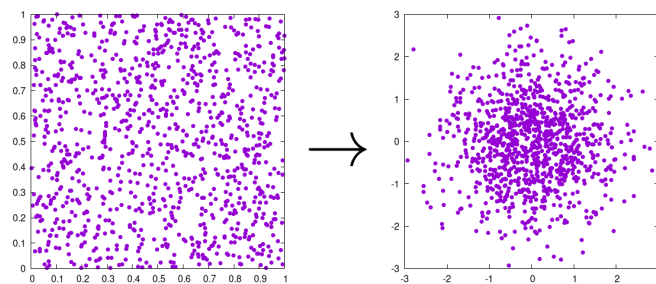


FIG. 1.3: Immagine presa da [QUI](#) <-

Calcolo del Jacobiano nella trasformazione di Box-Muller Per giustificare il metodo bisogna analizzare lo jacobiano della trasformazione. Partiamo dalla trasformazione sopra presentata, con definizioni di y_1, y_2, r, θ . Si ricava:

$$x_1 = e^{-r^2/2}, \quad x_2 = \frac{\theta}{2\pi}.$$

Poiché x_1 e x_2 sono uniformi e indipendenti, la loro densità congiunta è costante nel quadrato unitario: $p(x_1, x_2) = 1$, $0 < x_1 < 1$, $0 < x_2 < 1$.

Ora, il passaggio da (x_1, x_2) a (r, θ) produce il fattore jacobiano

$$\begin{vmatrix} \frac{\partial x_1}{\partial r} & \frac{\partial x_1}{\partial \theta} \\ \frac{\partial x_2}{\partial r} & \frac{\partial x_2}{\partial \theta} \end{vmatrix} = \begin{vmatrix} -re^{-r^2/2} & 0 \\ 0 & \frac{1}{2\pi} \end{vmatrix}.$$

si ottiene $\left| \frac{\partial(x_1, x_2)}{\partial(r, \theta)} \right| = \frac{r}{2\pi} e^{-r^2/2}$.

Per passare poi da coordinate polari a coordinate cartesiane si usa il noto elemento di area $dy_1 dy_2 = r dr d\theta$, cioè $dr d\theta = \frac{1}{r} dy_1 dy_2$.

Combinando i due passaggi, si trova che la densità di (y_1, y_2) è proporzionale a $\frac{r}{2\pi} e^{-r^2/2}$.
 $\frac{1}{r} = \frac{1}{2\pi} e^{-r^2/2}$. Poiché $r^2 = y_1^2 + y_2^2$, segue che:

$$P(y_1, y_2) = \frac{1}{2\pi} \exp\left(-\frac{y_1^2}{2} - \frac{y_2^2}{2}\right).$$

Questa è precisamente la distribuzione normale bidimensionale standard con componenti indipendenti.

Osservazione conclusiva su questa parte

Riassumendo, il principio generale è il seguente: una volta ottenute variabili uniformi affidabili, si possono costruire distribuzioni non uniformi applicando trasformazioni opportune. Nel caso unidimensionale il punto chiave è l'inversione della cumulativa; nel caso multidimensionale entra in gioco il determinante jacobiano della trasformazione.

Questo quadro teorico prepara direttamente al metodo di *rejection*, che diventa essenziale quando l'inversa della cumulativa non è disponibile in forma trattabile.

1.2 Rejection method

Il metodo del rifiuto permette di generare distribuzioni non uniformi senza richiedere il calcolo esplicito dell'integrale della densità.

Si consideri una distribuzione di probabilità desiderata $p(x)$, e una funzione $f(x)$ tale che $f(x) > p(x) \quad \forall x$, l'idea è utilizzare una coppia di variabili casuali uniformi per campionare la distribuzione $p(x)$.

Si procede nel seguente modo:

- si estrae un valore x_s uniformemente distribuito nel dominio;
- si estrae una seconda variabile casuale x_1 uniforme tra 0 e $f(x_s)$;
- si accetta x_s se vale la condizione $x_1 < p(x_s)$

Se la condizione non è soddisfatta, il punto viene rifiutato e si ripete la procedura. Questo metodo non richiede la conoscenza della funzione cumulativa, ma soltanto una funzione $f(x)$ che sovrastimi $p(x)$.

Interpretazione geometrica: Il metodo può essere interpretato geometricamente. Si consideri il grafico delle funzioni $p(x)$ e $f(x)$, con $f(x)$ tale che $f(x) \geq p(x)$

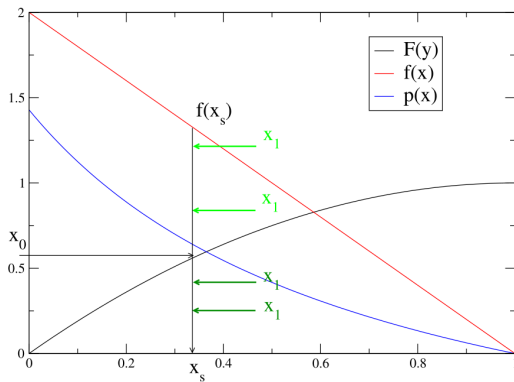


FIG. 1.4: rappresentazione grafica del *rejection method*

Si generano punti uniformemente nella regione delimitata da $f(x)$.

Un punto viene accettato se cade sotto la curva $p(x)$.

In questo modo, i punti accettati risultano distribuiti secondo la densità desiderata.

In fig.(??), i punti accettati sono quelli che cadono sotto $p(x)$, mentre gli altri vengono scartati.

1.2.1 Metodo del rifiuto per la distribuzione Gamma

La distribuzione Gamma di ordine intero $a > 0$ rappresenta la distribuzione del tempo di attesa per il verificarsi del a -esimo evento in un processo di Poisson con valore medio unitario ed è data da:

$$p_a(x) dx = \frac{x^{a-1} e^{-x}}{\Gamma(a)} dx, \quad x > 0, \quad (1.2)$$

Per valori piccoli di a , è possibile generare direttamente la distribuzione come somma di variabili esponenziali indipendenti.

Questo equivale a calcolare il prodotto di a variabili casuali uniformi e successivamente prendere il logaritmo.

Per valori più grandi di a , la distribuzione assume una forma a campana: con massimo circa in $x \approx a$ e larghezza dell'ordine $\sqrt{a}$

Funzione di confronto

Per applicare il metodo del rifiuto, si introduce una funzione di confronto.

Una scelta conveniente che si rivela utile è la distribuzione di Lorentz:

$$p(y) dy = \frac{1}{\pi(1+y^2)} dy \quad (1.3)$$

Integrandola si ottiene

$$F(y) = \int p(y) dy = \frac{1}{\pi} \arctan(y) \quad (1.4)$$

Funzione di maggiorazione

Si introduce una funzione $f(x)$ della forma

$$f(x) = \frac{c_0}{1 + (x - x_0)^2/a_0^2} \quad (1.5)$$

Questa funzione deve essere scelta in modo tale che $f(x) \geq p(x)$ e che il prodotto $a_0 c_0$ sia il più piccolo possibile, al fine di minimizzare il numero di rifiuti.

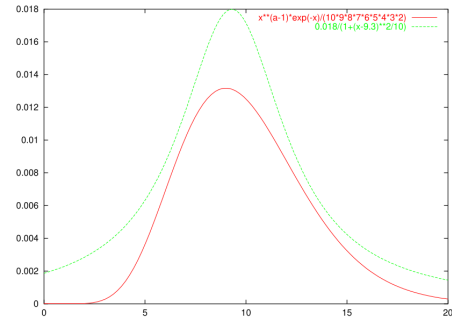


FIG. 1.5: *Gamma function* in rosso, mentre in verde si vede la lorentziana, funzione di maggiorazione $f(x)$.

Algoritmo

Si genera una variabile uniforme $u \in [0, 1]$ e si pone $x = a_0 \tan(\pi u) + x_0$ cioè $x = f^{-1}(y)$.

Successivamente si verifica la condizione di accettazione (rejection test).

Questo procedimento è implementato, ad esempio, nella routine **GAMDEV**.

```

1  #include <math.h>
2  float gamdev(int ia, long *idum){
3  \ \ Returns a deviate distributed as a gamma distribution of
   integer order ia,
4  \ \ ie, a waiting time to the i-th event in a Poisson process
   of unit mean,
5  \ \ using ran1(idum) as the source of uniform deviates.
6
7      float ran1(long *idum);
8      void nrerror(char error_text[]);
9      int j;
10     float am,e,s,v1,v2,x,y;
11     if (ia < 1) nrerror("Error in routine gamdev");
12     if (ia < 6) {
13         \ \ Use direct method, adding waiting
14         x = 1.0;
```

```

15     times.
16     for (j=1;j<=ia;j++) x *= ran1(idum);
17     x = -log(x);
18     }
19     else {          \\ Use rejection method.
20         do {
21             do {
22                 \\ These four lines generate the tangent of a
23                 \\ random angle,
24                 \\ ie, they are equivalent to y = tan{ * ran1(
25                 idum)}.
26                 do {
27                     v1 = ran1(idum);
28                     v2 = 2.0*ran1(idum) - 1.0;
29                 } while (v1*v1+v2*v2 > 1.0);
30                 y= v2 / v1;
31                 am = ia - 1;
32                 s = sqrt( 2.0*am+1.0 );
33                 x = s*y + am;
34                 } while (x <= 0.0);
35                 e=(1.0+y*y)*exp(am*log(x/am)-s*y);
36             } while (ran1(idum) > e);
37         }
38     return x;
39
40 \\ We decide whether to reject x: Reject in region of zero
41 probability.
42 \\ Ratio of prob. fn. to comparison fn. Reject on basis of a
43 second uniform deviate.
44 }

```

Listing 1.2: Implementazione del metodo GAMDEV

1.3 Integrazione Monte Carlo

Il metodo di integrazione Monte Carlo è strettamente collegato al metodo del rifiuto.

Esso permette di stimare integrali multidimensionali mediante campionamento casuale.

In uno spazio n -dimensionale, dato un insieme di N punti x_i , vale la stima

$$\int_V f dV \approx V \langle f \rangle \pm V \sqrt{\frac{\langle f^2 \rangle - \langle f \rangle^2}{N}} \quad (1.6)$$

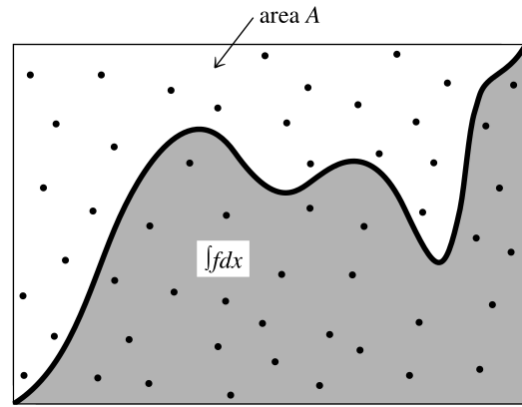
dove

$$\langle f^n \rangle = \frac{1}{N} \sum_{i=1}^N f^n(x_i) \quad (1.7)$$

Interpretazione

L'idea è quella di incorporare una regione difficile da campionare all'interno di una regione più semplice, dalla quale è possibile estrarre punti uniformemente.

Successivamente si utilizza il campionamento casuale per stimare l'integrale.



Il metodo può essere scritto in molti modi diversi in base a in quante dimensioni stiamo campionando, in quale spazio estraiamo i nostri punti e che forma ha la funzione, ma il concetto rimane sempre questo.

FIG. 1.6: Esempio visuale preso da "Numerical Recipes in C". L'integrale della funzione f è stimato come l'area del rettangolo A moltiplicata per la frazione di punti che si trova sotto la curva f

Esempio: area di un cerchio

Si consideri il calcolo dell'area di un cerchio unitario.

Si definisce una funzione

$$f = \begin{cases} 1 & \text{all'interno del cerchio} \\ 0 & \text{all'esterno} \end{cases} \quad (1.8)$$

Il volume della regione di campionamento è $V = 4$, la stima dell'area è $\int_V f dV \approx V \langle f \rangle$

Per $N = 1000$ si ottiene

$$\begin{aligned} \langle f \rangle &= 0.797 \\ \text{area} &= 3.19 \pm 0.05 \end{aligned}$$

Per $N = 10^6$:

$$\begin{aligned} \langle f \rangle &= 0.78616 \\ \text{area} &= 3.145 \pm 0.005 \end{aligned}$$

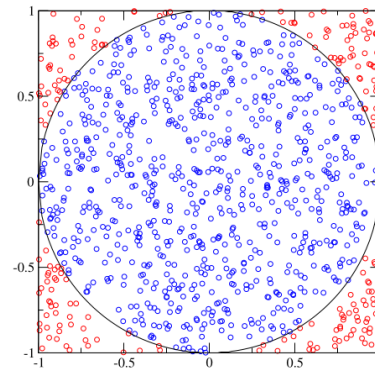


FIG. 1.7: Integrale Monte Carlo dell'area di un cerchio

Esempio: disco con foro

Si consideri un disco di raggio 1 con un foro di raggio 0.5 centrato nel punto $(0.5, 0)$.

Si vogliono calcolare:

$$x_{cm} = \frac{\int x dV}{\int dV} \quad (1.9)$$

$$I_{0,0} = \int (x^2 + y^2) dV \quad (1.10)$$

Le funzioni da integrare sono:

- $f_x = x$ all'interno della regione, zero fuori;
- $f_y^2 = y^2$, ecc.

Per $N = 10^6$ si ottiene:

$$x_{cm} = -0.393 \pm 0.004$$

$$y_{cm} = -0.000 \pm 0.005$$

$$I_{0,0} \approx 1.275$$

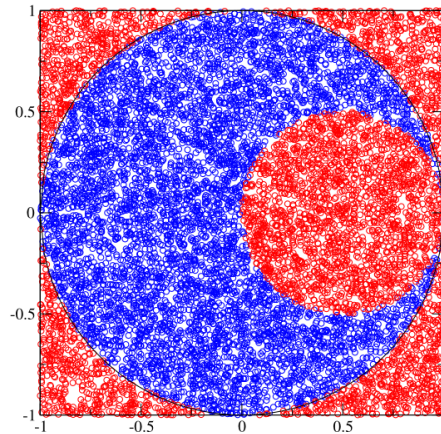


FIG. 1.8: Integrale Monte Carlo dell'area di un cerchio forato

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4
5  def inside_holed_disk(x, y):
6      """
7      Restituisce True se (x,y) appartiene al disco grande
8      ma non al foro interno.
9      """
10     in_big_disk = x**2 + y**2 <= 1.0**2
11     in_hole = (x - 0.5)**2 + y**2 <= 0.5**2
12     return in_big_disk & (~in_hole)
13
14
15  def monte_carlo_holed_disk(n_samples=100_000, seed=1234):
16      """
17      Campiona punti uniformi nel quadrato [-1,1]x[-1,1]
18      e stima:
19      - area della regione
20      - integrale di x
21      - integrale di y
22      - integrale di x^2+y^2
23      - centro di massa
24      - momento d'inerzia rispetto all'origine
25      """
26     rng = np.random.default_rng(seed)
27
28     # Campionamento uniforme nel quadrato
29     x = rng.uniform(-1.0, 1.0, n_samples)
30     y = rng.uniform(-1.0, 1.0, n_samples)
31

```

```

32     # Area del quadrato contenitore
33     V = 4.0
34
35     # Maschera della regione
36     mask = inside_holed_disk(x, y)
37
38     # Funzioni da integrare, prolungate con zero fuori dalla
           regione
39     f_area = mask.astype(float)
40     f_x = np.where(mask, x, 0.0)
41     f_y = np.where(mask, y, 0.0)
42     f_inertia = np.where(mask, x**2 + y**2, 0.0)
43
44     # Stime Monte Carlo degli integrali
45     area_est = V * np.mean(f_area)
46     int_x_est = V * np.mean(f_x)
47     int_y_est = V * np.mean(f_y)
48     inertia_est = V * np.mean(f_inertia)
49
50     # Errori statistici (deviazione standard della media)
51     area_err = V * np.std(f_area, ddof=1) / np.sqrt(n_samples)
52     int_x_err = V * np.std(f_x, ddof=1) / np.sqrt(n_samples)
53     int_y_err = V * np.std(f_y, ddof=1) / np.sqrt(n_samples)
54     inertia_err = V * np.std(f_inertia, ddof=1) / np.sqrt(
       n_samples)
55
56     # Centro di massa:  $x_{cm} = (x \, dA) / (Area)$ ,  $y_{cm} = (y \, dA) / ($ 
           Area)
57     x_cm_est = int_x_est / area_est
58     y_cm_est = int_y_est / area_est
59
60     results = {
61         "x": x,
62         "y": y,
63         "mask": mask,
64         "area": area_est,
65         "area_err": area_err,
66         "int_x": int_x_est,
67         "int_x_err": int_x_err,
68         "int_y": int_y_est,
69         "int_y_err": int_y_err,
70         "x_cm": x_cm_est,
71         "y_cm": y_cm_est,
72         "inertia": inertia_est,
73         "inertia_err": inertia_err,
74     }
75
76     return results
77
78
79 def plot_sample_points(x, y, mask, max_points=15000):

```

```

80     """
81     Mostra i punti Monte Carlo:
82     - blu = punti accettati nella regione
83     - grigio chiaro = punti fuori
84     """
85     n = min(len(x), max_points)
86
87     x_plot = x[:n]
88     y_plot = y[:n]
89     mask_plot = mask[:n]
90
91     fig, ax = plt.subplots(figsize=(7, 7))
92
93     ax.scatter(x_plot[~mask_plot], y_plot[~mask_plot], s=4,
94               alpha=0.3, label="Fuori regione")
95     ax.scatter(x_plot[mask_plot], y_plot[mask_plot], s=4,
96               alpha=0.6, label="Dentro regione")
97
98     # Disegno del bordo del disco esterno
99     t = np.linspace(0, 2*np.pi, 500)
100    ax.plot(np.cos(t), np.sin(t), linewidth=2)
101
102    # Disegno del bordo del foro
103    ax.plot(0.5 + 0.5*np.cos(t), 0.5*np.sin(t), linewidth=2)
104
105    ax.set_aspect("equal")
106    ax.set_xlim(-1.05, 1.05)
107    ax.set_ylim(-1.05, 1.05)
108    ax.set_title("Campionamento Monte Carlo del disco forato")
109    ax.set_xlabel("x")
110    ax.set_ylabel("y")
111    ax.legend()
112    ax.grid(True, alpha=0.3)
113    plt.show()
114
115    def running_area_estimate(n_samples=50_000, seed=1234):
116        """
117        Calcola la stima progressiva dell'area per visualizzare la
118        convergenza.
119        """
120        rng = np.random.default_rng(seed)
121        x = rng.uniform(-1.0, 1.0, n_samples)
122        y = rng.uniform(-1.0, 1.0, n_samples)
123
124        mask = inside_holed_disk(x, y).astype(float)
125
126        cumulative_mean = np.cumsum(mask) / np.arange(1, n_samples

```

```

127     return area_running
128
129
130 def plot_running_area(area_running):
131     """
132     Mostra la convergenza della stima dell'area.
133     """
134     fig, ax = plt.subplots(figsize=(8, 4.5))
135     ax.plot(area_running, linewidth=1.2, label="Stima Monte
        Carlo progressiva")
136
137     # Valore esatto dell'area:
138     # area disco grande - area foro =  $\pi - \pi/4 = 3\pi/4$ 
139     exact_area = 3.0 * np.pi / 4.0
140     ax.axhline(exact_area, linestyle="--", linewidth=2, label=
        "Valore esatto 3/4")
141
142     ax.set_title("Convergenza della stima dell'area")
143     ax.set_xlabel("Numero di campioni")
144     ax.set_ylabel("Area stimata")
145     ax.legend()
146     ax.grid(True, alpha=0.3)
147     plt.show()
148
149
150 if __name__ == "__main__":
151     results = monte_carlo_holed_disk(n_samples=200_000, seed
        =42)
152
153     print("=== Stime Monte Carlo ===")
154     print(f"Area          = {results['area']:.6f} ± {results['
        area_err']:.6f}")
155     print(f" x dA        = {results['int_x']:.6f} ± {results['
        int_x_err']:.6f}")
156     print(f" y dA        = {results['int_y']:.6f} ± {results['
        int_y_err']:.6f}")
157     print(f"x_cm         = {results['x_cm']:.6f}")
158     print(f"y_cm         = {results['y_cm']:.6f}")
159     print(f"I_0          = {results['inertia']:.6f} ± {results
        ['inertia_err']:.6f}")
160
161     plot_sample_points(results["x"], results["y"], results["
        mask"], max_points=12000)
162
163     area_running = running_area_estimate(n_samples=50_000,
        seed=42)
164     plot_running_area(area_running)

```

Listing 1.3: Esercizio di integrazione Monte Carlo del disco bucato in Python, scritta da ChatGpt 5.3

Capitolo 2

Ricerca degli zeri di una funzione

2.1 Introduzione

La determinazione degli zeri di una funzione, ovvero la risoluzione dell'equazione $f(x) = 0$, costituisce uno dei problemi fondamentali dell'analisi numerica. Nonostante la formulazione apparentemente semplice, si tratta in generale di un problema non banale quando non si ha una soluzione analitica.

L'idea di base è la seguente: si cerca inizialmente di individuare in maniera approssimata una regione dello spazio in cui è presente uno zero della funzione; successivamente si utilizza un algoritmo numerico per raffinare progressivamente la stima.

Esempio introduttivo: Consideriamo la funzione

$$f(x) = x + \frac{x^2}{10},$$

e supponiamo di studiarla nell'intervallo

$$[-15, 10].$$

Un primo approccio consiste nel valutare la funzione in un numero finito di punti (ad esempio 30 punti equispaziati) e osservare il comportamento del grafico.

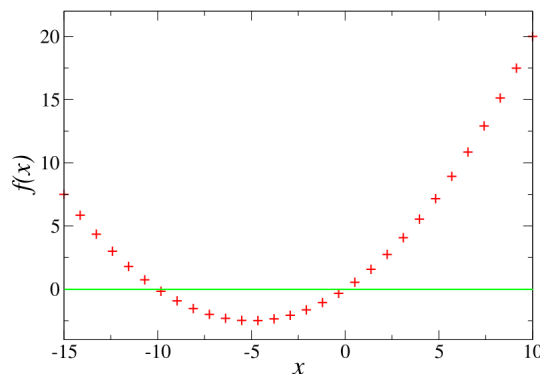


FIG. 2.1: esempio di funzione $f(x)$ plottata con soli 30 punti per stimare i segmenti in cui cercare lo zero.

Questo permette di individuare visivamente la regione in cui la funzione cambia segno, e quindi dove è presente uno zero.

Questa fase preliminare è essenziale: senza una buona stima iniziale della posizione dello zero, molti metodi numerici risultano inefficaci o addirittura divergenti.

Casi difficili e comportamenti patologici: Esistono tuttavia funzioni per le quali la localizzazione dello zero può risultare estremamente complicata dal punto di vista numerico.

Un esempio significativo è dato dalla funzione

$$f(x) = 3x^2 + 1 + \frac{\ln[(\pi - x)^2]}{\pi^4}.$$

In questo caso esistono due zeri situati approssimativamente in

$$x \approx \pi \pm 10^{-667}.$$

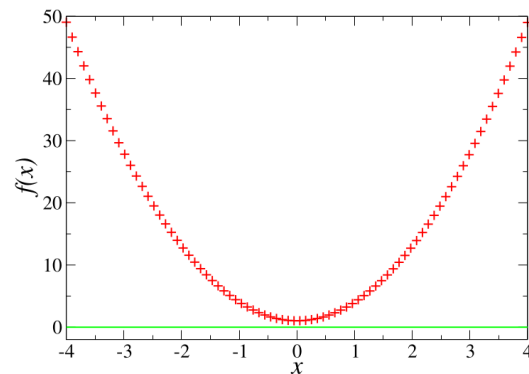


FIG. 2.2: Funzione $f(x)$ plottata con 80 punti nel range $[-4; 4]$

Si tratta di zeri estremamente vicini al valore π , tanto che una loro individuazione diretta tramite semplice campionamento risulta praticamente impossibile.

Tuttavia, osservando la struttura della funzione, si nota che il termine logaritmico diventa molto grande e negativo quando x si avvicina a π . Questo suggerisce che lo zero si trovi nelle vicinanze di tale punto.

Questo esempio mostra chiaramente come, in molti casi, sia necessario combinare osservazioni qualitative sulla funzione con tecniche numeriche.

Nel seguito assumeremo di essere riusciti a individuare un intervallo che contiene uno zero della funzione e ci concentreremo su come raffinarne la posizione.

2.2 Metodo di bisezione

Una volta individuato un intervallo in cui è presente uno zero, il metodo più semplice per determinarlo con precisione è il **metodo di bisezione**.

Questo metodo si basa su una proprietà fondamentale delle funzioni continue: se una funzione cambia segno in un intervallo, allora deve necessariamente annullarsi in almeno un punto interno.

Idea del metodo Supponiamo di conoscere due punti x_1 e x_2 tali che la funzione cambia segno nell'intervallo $[x_1, x_2]$, ovvero:

$$f(x_1) > 0, \quad f(x_2) < 0,$$

Si definisce quindi il punto medio $x_m = \frac{x_1 + x_2}{2}$ e si valuta il segno della funzione nello stesso:

- se $f(x_m) > 0$, lo zero si trova nell'intervallo $[x_m, x_2]$, quindi si pone $x_1 = x_m$;
- se $f(x_m) < 0$, lo zero si trova nell'intervallo $[x_1, x_m]$, quindi si pone $x_2 = x_m$.

Il procedimento viene quindi iterato, restringendo progressivamente l'intervallo che contiene lo zero.

Stima dell'errore: Se inizialmente la lunghezza dell'intervallo è $\epsilon = |x_1 - x_2|$, dopo n iterazioni la lunghezza dell'intervallo si riduce a $\delta \approx \epsilon / 2^n$.

Questo fornisce una stima diretta dell'errore sulla posizione dello zero.

Esempio numerico del metodo di bisezione: Consideriamo la funzione

$$f(x) = \frac{x^3}{5} + \frac{x}{5} + 0.1.$$

Applicando il metodo di bisezione si ottiene la seguente sequenza di approssimazioni:

i	x_r	δ
1	-0.9	0.9
2	-0.45	0.45
3	-0.23	0.23
4	-0.338	0.11
5	-0.394	0.06
6	-0.422	0.03

Si osserva come l'intervallo contenente lo zero si restringa progressivamente e la stima della radice converga.

Questo esempio illustra chiaramente il comportamento del metodo: la convergenza è garantita, ma relativamente lenta, poiché l'errore si riduce esponenzialmente con fattore $1/2$ ad ogni iterazione.

2.2.1 Osservazioni sul metodo di bisezione

Il metodo di bisezione è estremamente robusto e presenta alcune caratteristiche fondamentali, esso converge purché la funzione sia continua e si riesca a individuare un intervallo in cui essa cambia segno. Tuttavia, la convergenza è relativamente lenta, in quanto l'errore si riduce secondo la legge $\delta = \epsilon / 2^n$.

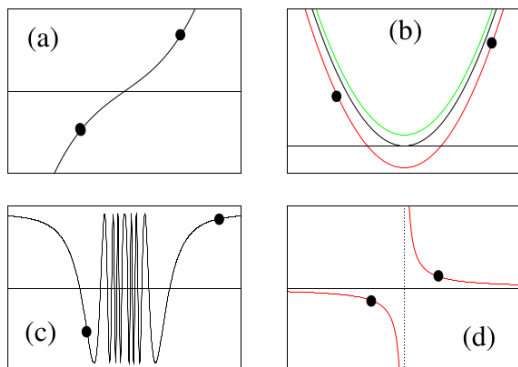


FIG. 2.3: Casi particolari che si possono verificare.

Esistono diversi casi particolari che è opportuno considerare:

- **Caso normale:** la funzione cambia segno una sola volta nell'intervallo e il metodo converge allo zero corretto.
- **Radici multiple:** la funzione può toccare l'asse senza cambiare segno; in questo caso il metodo può non individuare correttamente lo zero.
- **Radici molto vicine:** possono esistere più zeri estremamente vicini tra

loro, rendendo difficile l'identificazione numerica.

- **Discontinuità:** se la funzione pre-

senta una discontinuità, il metodo converge comunque, ma al punto di discontinuità anziché a uno zero reale.

2.3 Metodo di Newton-Raphson

Un approccio più sofisticato per il raffinamento della radice è il metodo di **Newton-Raphson**: supponiamo di avere una stima iniziale x della radice e di voler trovare una stima migliorata x_r . Si approssima la funzione tramite lo sviluppo di Taylor al primo ordine:

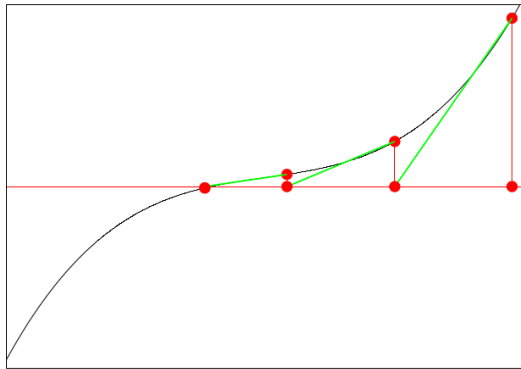
$$f(x_r) \approx f(x) + (x_r - x) \frac{df(x)}{dx} \stackrel{!}{=} 0 .$$

Imponendo la condizione $f(x_r) \stackrel{!}{=} 0$, da cui segue immediatamente

$$x_r = x - \frac{f(x)}{f'(x)} . \quad \Rightarrow \quad x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} . \quad (2.1)$$

Questa relazione definisce una procedura iterativa.

Esempio numerico: Consideriamo nuovamente la funzione $f(x) = \frac{x^3}{5} + \frac{x}{5} + 0.1$.



Applicando il metodo di Newton-Raphson si ottiene la seguente sequenza:

i	x_r	$f(x_r)$
0	1.9	
1	1.1173	0.60
2	0.4825	0.22
4	-0.4714	0.02
5	-0.4257	0.0006
7	-0.4239	$< 10^{-8}$

Si osserva una convergenza molto più rapida rispetto al metodo di bisezione.

2.3.1 Analisi dell'errore

Per comprendere meglio la velocità di convergenza, consideriamo una radice x_r e poniamo $x = x_r + \epsilon$, così che sviluppando la funzione:

$$f(x + \epsilon) = f(x) + \epsilon f'(x) + \frac{\epsilon^2}{2} f''(x) .$$

Stavolta non buttiamo via il termine del secondo ordine così applicando l'iterazione di Newton-Raphson di eq.(2.1) si ottiene una stima per l'errore:

$$\epsilon_{i+1} = -\frac{\epsilon_i^2 f''(x_i)}{2f'(x_i)}.$$

Questo mostra che l'errore si riduce **quadraticamente**, ovvero

$$\epsilon_{i+1} \sim \epsilon_i^2,$$

il che spiega la rapidissima convergenza del metodo.

2.3.2 Osservazioni sul metodo di Newton-Raphson

Il metodo di Newton-Raphson presenta numerosi vantaggi:

- convergenza molto rapida;
- esatto per funzioni lineari;
- estendibile a più dimensioni;
- non richiede il bracketing della radice.

Tuttavia presenta anche alcune criticità:

- richiede una buona stima iniziale, altrimenti...
- ... può divergere o oscillare;
- può bloccarsi in punti stazionari;
- fallisce in presenza di discontinuità;
- richiede che $f'(x) \neq 0$.

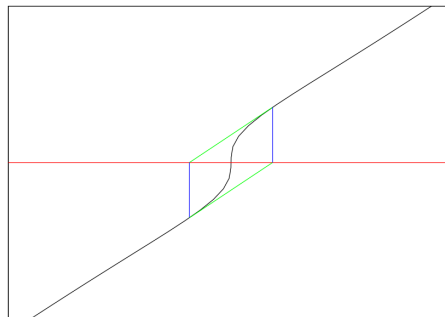


FIG. 2.4: esempio visivo di radice che fa andare in palla il metodo di Newton-Raphson.

2.4 Caso dei polinomi, metodo di Laguerre

Nel caso particolare dei polinomi esistono metodi specifici molto efficienti.

Consideriamo un polinomio scritto nella sua forma fattorizzata:

$$P_n(x) = (x - x_1)(x - x_2) \cdots (x - x_n).$$

Si può portare il prodotto ad essere una sommatoria usando il logaritmo:

$$\ln |P_n(x)| = \sum_i \ln |x - x_i|.$$

Derivando:

$$\frac{d}{dx} \ln |P_n(x)| = \sum_i \frac{1}{x - x_i} = \frac{P'_n(x)}{P_n(x)} \equiv G,$$

Questa nuova quantità G possiamo trattarla come una grandezza che ci dice in base al punto x in cui siamo quanto siamo vicini a ciascuna radice (vedi l'inverso, se siamo molto vicini, G diventa molto grande). In particolare la radice più vicina x_k contribuirà molto di più nella sommatoria, quindi $x_k \approx x - \frac{1}{G}$ in prima approssimazione.

$$\frac{d^2}{dx^2} \ln |P_n(x)| = \sum_i \frac{1}{(x - x_i)^2} = \left[\frac{P'_n}{P_n} \right]^2 - \frac{P''_n}{P_n} = H.$$

Possiamo esprimere H in funzione del polinomio e di G , che porta informazioni sulla distanza delle radici: $H = G^2 - P''_n/P_n$.

Si ottiene quindi $G = \frac{1}{a} + \frac{n-1}{b}$ e $H = \frac{1}{a^2} + \frac{n-1}{b^2}$, dove in particolare a è la distanza della radice più vicina ($a = x - x_k$) e b è la distanza di 'tutte' le altre radici (facendo finta siano tutte ugualmente distanti, $\sum_{i \neq k} (x - x_i)^{-2} \approx \frac{n-1}{b^2}$), così abbiamo una prima stima infine della distanza della radice:

$$a = \frac{n}{G \pm \sqrt{(n-1)(nH - G^2)}}.$$

Il metodo consiste nell'iterare $x \rightarrow x - a$, scegliendo il segno che massimizza il denominatore.

2.5 Generalizzazione a più dimensioni di Newton-Raphson

Il problema della ricerca degli zeri si estende naturalmente a più dimensioni.

Si considerano sistemi del tipo $f_i(\mathbf{X}) = 0$, con $i = 1, \dots, n$, dove $\mathbf{X} = (x_1, \dots, x_n)$ è un vettore in $\mathbb{R}^n$.

Si generalizza il metodo di Newton-Raphson scrivendo

$$f_i(\mathbf{X} + \delta) \approx f_i(\mathbf{X}) + \sum_j \frac{\partial f_i}{\partial x_j} \delta_j.$$

Definendo $a_{ij} = \frac{\partial f_i}{\partial x_j}$, coefficienti dello jacobiano, si ottiene il sistema lineare:

$$\sum_j a_{ij} \delta_j = -f_i(\mathbf{X}).$$

Risolvendo per δ si ottiene $\delta_j = -\sum_i (a_{ij})^{-1} f_i(\mathbf{X})$ e si itera seguendo $X_i^{(\text{new})} = X_i^{(\text{old})} + \delta_i$.

Esempio bidimensionale: Un esempio di sistema non lineare è

$$f_1(x, y) = x^2 + y^2 - 1,$$

$$f_2(x, y) = y - x^2 + \frac{x}{2}.$$

La soluzione corrisponde all'intersezione delle due curve.

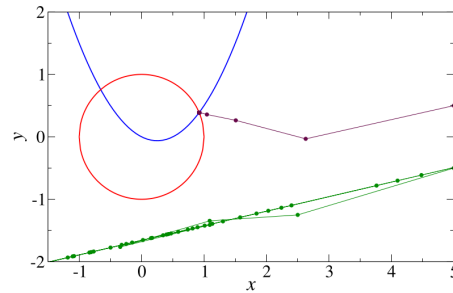


FIG. 2.5: esempio bidimensionale

2.6 Conclusioni

Riassumendo:

- è sempre necessario avere un'idea preliminare della posizione della radice;
- il metodo di bisezione è robusto ma lento;
- il metodo di Newton è rapido ma delicato;
- per i polinomi esistono metodi dedicati;
- in più dimensioni il metodo di Newton è lo strumento principale, ma è sensibile alle condizioni iniziali.

La scelta del metodo più adatto dipende quindi dal problema specifico, dal grado di informazione disponibile sulla funzione e dal compromesso desiderato tra stabilità e velocità di convergenza.

Capitolo 3

Analisi numerica nell'integrazione

3.1 Introduzione

Il problema dell'integrazione numerica può essere risolto, in linea di principio, con l'aiuto di equazioni differenziali ordinarie (ODEs):

$$\int_a^x f(y) dy = g(x) \quad \Rightarrow \quad \frac{dg}{dx} = f(x),$$

con boundary $x_0 = a$.

Sebbene questa osservazione sia formalmente corretta, è preferibile sviluppare metodi diretti di integrazione numerica, analogamente a quanto si è fatto per le equazioni differenziali, dove la scelta del metodo appropriato non è affatto immediata.

Un algoritmo generale di integrazione numerica ha la struttura

$$\int_a^b f(x) dx = \sum_i w_i f(x_i),$$

dove w_i sono pesi e x_i i nodi di valutazione.

In genere si pone $x_i = a + ih$ con passo h uniforme e si scrive $f_i = f(x_i)$.

Si distinguono due grandi classi di formule:

- **Formule chiuse:** utilizzano anche i valori agli estremi a e b .
- **Formule aperte:** non utilizzano uno o entrambi gli estremi.

Il vantaggio principale delle formule aperte è che possono essere utilizzate quando la funzione non è definita agli estremi, pur essendo l'integrale ben definito.

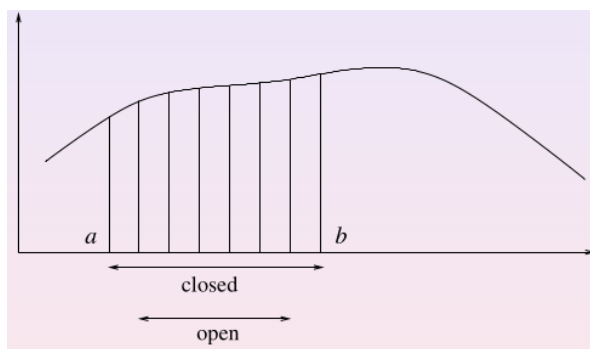


FIG. 3.1: differenza fra intervalli presi in considerazione da classi di formule chiuse e aperte

3.2 Formule di Newton-Cotes su intervallo singolo

Le formule di Newton-Cotes nascono dalla divisione in intervalli equispaziati e dalla successiva integrazione esatta di un polinomio che passa per quegli n punti (interpolazione).

La regola fondamentale dell'integrazione numerica, la formula di Newton-Cotes di grado più basso è la regola dei trapezi, che usa una retta tesa fra due punti successivi per interpolare:

$$\int_{x_1}^{x_2} f(x) dx = \frac{1}{2} h [f_1 + f_2] + \mathcal{O}(h^3 f'') , \quad (3.1)$$

questa è esatta per funzioni lineari, ad esempio $f(x) = x$.

Si ottiene interpolando $f(x)$ con il polinomio lineare che passa per i punti (x_1, f_1) e (x_2, f_2) :

$$f(x) = f_1 + (f_2 - f_1)x ,$$

e in seguito integrando sull'intervallo normalizzato $0 \leq x \leq 1$.

Formule di ordine superiore si ottengono interpolando con polinomi di grado più elevato. Ad esempio,

$$\int_{x_1}^{x_3} f(x) dx = h \left[\frac{1}{3} f_1 + \frac{4}{3} f_2 + \frac{1}{3} f_3 \right] + \mathcal{O}(h^5 f^{(4)}) , \quad (3.2)$$

che è la regola di Simpson, esatta per polinomi fino al terzo grado.

La regola di Simpson nasce dall'interpolazione con una parabola $P(x) = Ax^2 + Bx + C$ su tre punti equispaziati x_0, x_1, x_2 con funzione da integrare $f_0 = f(x_0), f_1, f_2$. Facciamo un cambio di variabile $t = x - x_i$ così che i tre punti diventino $t = 0, h, 2h$, riscrivendo la parabola come $P(t) = at^2 + bt + c$.

Si va ora ad imporre che $P(t = 0) = c = f_0$, mentre nel secondo punto $P(t = h) = ah^2 + bh + c = f_1$ e nel terzo $P(t = 2h) = 4ah^2 + 2bh + c = f_2$. Invertendo questo sistema di equazioni si trova che $c = f_0$, $a = \frac{f_2 - 2f_1 + f_0}{2h^2}$ mentre $b = \frac{f_1 - f_0}{h} - ah$.

A questo punto l'integrale $\int_{x_0}^{x_2} f(x) dx \approx \int_0^{2h} P(t) dt = \int_0^{2h} at^2 + bt + f_0 dt$ è facilmente sostituibile con le dovute sostituzioni di a e b nelle forme appena trovate e con una semplice integrazione del polinomio si ottiene proprio Simpson: $\int_{x_0}^{x_2} f(x) dx \approx \frac{h}{3} [f_0 + 4f_1 + f_2]$.

Analogamente, per cinque punti la formula di Newton-Cotes diventa:

$$\int_{x_1}^{x_5} f(x) dx = h \left[\frac{14}{45} f_1 + \frac{64}{45} f_2 + \frac{24}{45} f_3 + \frac{64}{45} f_4 + \frac{14}{45} f_5 \right] + \mathcal{O}(h^7 f^{(6)}) .$$

3.2.1 Alcune formule locali

È utile osservare anche relazioni locali su intervalli elementari:

$$\int_{x_0}^{x_1} f(x) dx = h[f_1] + \mathcal{O}(h^2 f') ,$$

$$\int_{x_0}^{x_1} f(x) dx = h \left[\frac{3}{2}f_1 - \frac{1}{2}f_2 \right] + \mathcal{O}(h^3 f'') .$$

La prima è un semplice rettangolo, una Newton-Cotes aperta di ordine 0 con errore di ordine h^2 , mentre la seconda formula è una Newton-Cotes aperta di ordine 1 e si ottiene interpolando linearmente $f(x) = f_1 + x(f_2 - f_1)$, e integrando nell'intervallo $-1 \leq x \leq 0$:

$$\int_{-1}^0 f(x) dx = -(-f_1) - \frac{(-1)^2}{2} (f_2 - f_1) = \left[\frac{3}{2}f_1 - \frac{1}{2}f_2 \right] .$$

Formule di ordine superiore si ottengono usando approssimazioni polinomiali di grado crescente.

3.3 Intervalli estesi

Passando a intervalli estesi, dalla regola dei trapezi in eq.(3.1) segue una regola dei trapezi estesa:

$$\int_{x_1}^{x_n} f(x) dx = h \left[\frac{1}{2}f_1 + f_2 + f_3 + \dots + \frac{1}{2}f_n \right] + \mathcal{O}\left(\frac{(x_n - x_1)^3}{N^2} f''\right) , \quad (3.3)$$

dove N è il numero di suddivisioni.

Dalla regola di Simpson in eq.(3.2), si ottiene invece:

$$\int_{x_1}^{x_n} f(x) dx = h \left[\frac{1}{3}f_1 + \frac{4}{3}f_2 + \frac{2}{3}f_3 + \frac{4}{3}f_4 \dots + \frac{4}{3}f_{n-1} + \frac{1}{3}f_n \right] + \mathcal{O}\left(\frac{1}{N^4}\right) . \quad (3.4)$$

Di seguito un'implementazione in C della regola dei trapezi e di Simpson:

```
1  #include <stdio.h>
2
3  // Queste le funzioni che andremo a definire a seguito del
   MAIN
4  double POWER(double numero, double elevazione); // la funzione
   da integrare: legge di potenza
5  double funzione(double varF); // routine che presa una x
   restituisce la y=f(x)
```

```
6 double areaTrapezio(double lato1,double lato2,double altezza);  
    // routine che calcola l'area di un singolo trapezio  
7 double trapezioIntegral(double a,double b,int colonne); //  
    integrazione con metodo dei trapezi  
8 double simpsonIntegral(double a,double b,int colonne); //  
    integrazione con Simpson  
9  
10 int main(int argc,char**argv){  
11     // main: qui si chiama TrapezioIntegral per fare un'  
    integrazione da a=1 a b=2  
12     // con l'intervallo diviso in 5000 parti uguali  
13     printf("[ trapezi: %lf ]\n", trapezioIntegral(1,2,5000));  
14     printf("[ simpson  %lf ]\n", simpsonIntegral(1,2,5000));  
15 }  
16  
17 double POWER(double numero,int elevazione){  
18     double ret = 1;  
19     for(int i=elevazione;i>0;i--){  
20         ret *= numero;  
21     }  
22     return ret;  
23 }  
24 double funzione(double varF){  
25     double y = POWER(varF,2);  
26     return y;  
27 }  
28  
29 double areaTrapezio(double lato1,double lato2,double altezza){  
30     return (lato1+lato2)*altezza/2;  
31 }  
32  
33 double trapezioIntegral(double a,double b,int colonne){  
34     double area = 0;  
35     double nPartiUguali = (b-a)/colonne;  
36  
37     for(int i=0;i<colonne-1;i++){  
38         area += areaTrapezio(funzione(a),funzione(a+  
            nPartiUguali),nPartiUguali);  
39         a += nPartiUguali;  
40     }  
41     area += areaTrapezio(funzione(a),funzione(b),b-a);  
42     return area;  
43 }  
44  
45 // Simpson composito su [a,b] con "colonne" sottointervalli (  
    deve essere pari)  
46 double simpsonIntegral(double a,double b,int colonne){  
47     if(colonne < 2) colonne = 2;  
48     if(colonne % 2 != 0) colonne -= 1;    // Simpson richiede  
        un numero pari di sottointervalli  
49 }
```

```

50     double h = (b-a)/colonne;
51     double somma = funzione(a) + funzione(b);
52
53     // termini con coefficiente 4: i dispari
54     for(int i=1; i<=colonne-1; i+=2){
55         double x = a + i*h;
56         somma += 4.0 * funzione(x);
57     }
58
59     // termini con coefficiente 2: i pari (esclusi estremi)
60     for(int i=2; i<=colonne-2; i+=2){
61         double x = a + i*h;
62         somma += 2.0 * funzione(x);
63     }
64
65     return (h/3.0) * somma;
66 }

```

Listing 3.1: Implementazione del metodo dei trapezi

Si può combinare una formula aperta locale (singolo intervallo) con una formula chiusa estesa di quelle appena scritte per ottenere una formula aperta su intervallo esteso.

Ad esempio, combinando

$$\int_{x_0}^{x_1} f(x) dx = hf_1 + \mathcal{O}(h^2 f')$$

con la regola dei trapezi estesa su $[x_1, x_n]$ vista in eq.3.3, si ottiene:

$$\int_{x_0}^{x_n} f(x) dx = h \left[\frac{3}{2}f_1 + f_2 + f_3 + \dots + \frac{3}{2}f_{n-1} \right] + \mathcal{O}\left(\frac{1}{N^2}\right) .$$

Analogamente, combinando la formula aperta di ordine superiore

$$\int_{x_0}^{x_1} f(x) dx = h \left[\frac{23}{12}f_1 - \frac{16}{12}f_2 + \frac{5}{12}f_3 \right] + \mathcal{O}(h^4 f^{(3)}) ,$$

con la regola di Simpson estesa eq.(3.4), si ottiene una formula complessiva di ordine $\mathcal{O}(1/N^4)$.

$$\int_{x_0}^{x_n} f(x) dx = h \left[\frac{27}{12}f_1 + 0 + \frac{13}{12}f_3 + \frac{4}{3}f_4 + \dots + \frac{4}{3}f_{n-4} + \frac{13}{12}f_{n-3} + 0 + \frac{27}{12}f_{n-1} \right] + \mathcal{O}\left(\frac{1}{N^2}\right) .$$

Il problema pratico fondamentale rimane: quanti passi bisogna utilizzare?

Se si usa la regola dei trapezi come blocco base, l'errore scala come $\boxed{K/N^2}$, raddoppiando N , l'errore si riduce di un fattore 4. Questo comportamento è alla base dei metodi di raffinamento successivi.

3.3.1 Valutazione dell'errore e formula esatta dei trapezi

L'errore può essere valutato facendo la differenza fra l'integrale calcolato con N steps e quello calcolato con $2N$ steps.

Per stimare in modo più preciso l'errore della regola dei trapezi estesa, si può utilizzare la sua espressione esatta in termini dei numeri di Bernoulli:

$$\int_{x_1}^{x_n} f(x) dx = h \left[\frac{1}{2}f_1 + f_2 + \dots + \frac{1}{2}f_n \right] - \frac{B_2 h^2}{2!} (f'_n - f'_1) - \frac{B_{2k} h^{2k}}{(2k)!} (f_n^{(2k-1)} - f_1^{(2k-1)}) - \dots$$

dove B_k sono i numeri di Bernoulli, definiti tramite

$$\frac{t}{e^t - 1} = \sum_{n=0}^{\infty} B_n \frac{t^n}{n!}.$$

Il termine dominante dell'errore è proporzionale a h^2 , cioè a $1/N^2$. Se indichiamo con S_N l'integrale calcolato con N suddivisioni e con S_{2N} quello con $2N$, possiamo eliminare il termine in h^2 combinando linearmente i due risultati:

$$S = \frac{4}{3}S_{2N} - \frac{1}{3}S_N.$$

In questo modo l'errore dominante diventa dell'ordine h^4 .

3.3.2 Integrazione di Romberg

L'idea precedente può essere iterata. Se una combinazione elimina il termine in h^2 , si può ripetere il procedimento per eliminare anche il termine in h^4 , poi quello in h^6 , e così via.

Il metodo di Romberg consiste nel:

- calcolare l'integrale con una sequenza di suddivisioni $N, 2N, 4N, \dots$;
- costruire combinazioni lineari che eliminano progressivamente i termini dominanti dell'errore;
- interpretare il risultato come un'estrapolazione razionale nel limite $N \rightarrow \infty$.

Il metodo produce una tabella triangolare di approssimazioni con ordine crescente di accuratezza.

Di seguito un'implementazione in C dell'integrazione di Romberg presa dal *Numerical Recipes in C: The art of scientific programming*:

```
1  #include <math.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  #define EPS    1.0e-6f    // accuratezza relativa desiderata
```

```

6  #define JMAX 20          // massimo numero di raffinamenti (
    livelli trapezio)
7  #define K 5             // punti usati nellestrapolazione (
    Romberg "ordine" ~ 2K)
8
9  // Prototipi: definiscili da qualche parte nel tuo progetto
10 float trapzd(float (*func)(float), float a, float b, int j);
11 void polint(const float xa[], const float ya[], int n, float
    x, float *y, float *dy);
12
13 static void nrerror(const char *msg) {
14     fprintf(stderr, "%s\n", msg);
15     exit(EXIT_FAILURE);
16 }
17
18 // Romberg via: trapezi successivi + estrapolazione
    polinomiale in  $h^2$  verso 0
19 float qromb(float (*func)(float), float a, float b)
20 {
21     float s[JMAX];        // s[j] = stima trapezoidale al
    livello j (0-based)
22     float h[JMAX];        // h[j] = variabile proporzionale a (
    passo)2
23     float ss = 0.0f;      // stima extrapolata dellintegrale (
    valore a  $h^2=0$ )
24     float dss = 0.0f;     // stima errore dellestrapolazione
25
26     // h[0] è solo una scala: conta il rapporto tra livelli.
27     // Ad ogni raffinamento, il passo h_geom dimezza => (passo
    )2 /4.
28     h[0] = 1.0f;
29
30     for (int j = 0; j < JMAX; ++j) {
31
32         // trapezio al livello (j+1) per restare compatibili
    con NR
33         s[j] = trapzd(func, a, b, j + 1);
34
35         // quando ho almeno K valori, estrapolo in  $h^2$  fino a
    0
36         if (j + 1 >= K) {
37             // uso gli ultimi K punti: h[j-K+1 .. j] e s[j-K+1
    .. j]
38             polint(&h[j - K + 1], &s[j - K + 1], K, 0.0f, &ss,
    &dss);
39
40             // criterio di arresto: errore stimato relativo
    sotto EPS
41             if (fabsf(dss) <= EPS * fabsf(ss)) {
42                 return ss;
43             }

```

```

44     }
45
46     // preparo la "x" per il prossimo livello: h^2 scala
      di 1/4 quando il passo scala di 1/2
47     if (j + 1 < JMAX) {
48         h[j + 1] = 0.25f * h[j];
49     }
50 }
51
52 nrerror("Too many steps in routine qromb");
53 return 0.0f; // non ci arriva mai
54 }

```

Listing 3.2: Implementazione del metodo di Romberg

3.3.3 Quadrature Gaussian

Vogliamo approssimare integrali del tipo

$$\int W(x)f(x) dx = \sum_{j=1}^N w_j f(x_j) \quad ,$$

dove: $W(x) > 0$ è una funzione peso, $\int W(x) dx = 1$ (normalizzazione) ed i nodi x_j non sono necessariamente equispaziati.

Introduciamo il prodotto scalare pesato

$$\langle f | g \rangle = \int W(x) f(x) g(x) dx \quad .$$

Si definisce una successione di polinomi ortogonali $\{p_k(x)\}$ tramite la ricorrenza a tre termini:

$$p_{-1}(x) = 0, \quad p_0(x) = 1 \quad ,$$

$$p_{j+1}(x) = (x - a_j) p_j(x) - b_j p_{j-1}(x) \quad ,$$

con coefficienti:

$$a_j = \frac{\langle x p_j | p_j \rangle}{\langle p_j | p_j \rangle}, \quad b_j = \frac{\langle p_j | p_j \rangle}{\langle p_{j-1} | p_{j-1} \rangle} \quad .$$

Proprietà fondamentali:

- p_j è monico e di grado j ,
- p_j possiede j zeri reali semplici,

- gli zeri sono interlacciati con quelli di p_{j-1} .

Assumiamo che $f(x)$ sia un polinomio di grado al massimo $2N - 1$. Può essere scritto in generale come $f(x) = p_N(x)q_f(x) + r_f(x)$ con $q_f(x)$ e $r_f(x)$ polinomi di grado $< N$. In particolare:

$$q_f(x) = \sum_{N-1} a_k p_k(x) , \quad a_k = \frac{\langle p_k | q_f \rangle}{\langle p_k | p_k \rangle} ,$$

$$r_f(x) = \sum_{N-1} b_k p_k(x) , \quad b_k = \frac{\langle p_k | r_f \rangle}{\langle p_k | p_k \rangle} ,$$

Vale che:

$$\int W(x) f(x) dx = \int W(x) (P_N(x)q_f(x) + r_f(x)) dx = \sum w_j (P_N(x)q_f(x) + r_f(x)) ,$$

che, nel caso in cui le x_j siano radici di p_N , diventa:

$$\int W(x) f(x) dx = \sum w_j(x) r_f(x_j) = \int W(x) r_f(x) dx ,$$

Ma per ipotesi, r_f era di un grado in meno rispetto ad N , quindi l'integrale qui sopra è esatto.

Sappiamo che le x_j sono radici di p_n , cosa dire per le w_j ?

Grazie all'ortogonalità si ottiene il sistema lineare:

$$\begin{pmatrix} p_0(x_1) & p_0(x_2) & \dots & p_0(x_N) \\ p_1(x_1) & p_1(x_2) & \dots & p_1(x_N) \\ \vdots & \vdots & & \vdots \\ p_{N-1}(x_1) & p_{N-1}(x_2) & \dots & p_{N-1}(x_N) \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{pmatrix} = \begin{pmatrix} \int W(x) p_0(x) dx \\ 0 \\ \vdots \\ 0 \end{pmatrix} .$$

Se scegliamo x_j come zeri di $p_N(x)$, il sistema è ben posto e la formula risulta esatta fino al grado $2N - 1$.

I pesi si possono esprimere direttamente come:

$$w_j = \frac{\langle p_{N-1} | p_{N-1} \rangle}{p_{N-1}(x_j) p'_N(x_j)} .$$

Intervallo	$W(x)$	"Recurrence"	Nome
$(-1, 1)$	1	$(i+1)P_{i+1} = (2i+1)xP_i + iP_{i-1}$	Legendre
$(-1, 1)$	$\frac{1}{\sqrt{1-x^2}}$	$T_{i+1} = 2xT_i - T_{i-1}$	Chebyshev
$(0, \infty)$	$x^c e^{-x}$	$(i+1)L_{i+1}^c = (-x+2i+c+1)L_i^c - (i+c)L_{i-1}^c \quad c=0, 1, \dots$	Laguerre
$(-\infty, \infty)$	e^{-x^2}	$H_{i+1} = 2xH_i - 2iH_{i-1}$	Hermite

Tabella GaussLegendre

Per $W(x) = 1$ su $(-1, 1)$:

n	x_j	w_j
2	± 0.577350269189626	1.0
3	± 0.774596669241483 0	0.5555555555555556 0.8888888888888889
4	± 0.861136311594053 ± 0.339981043584856	0.347854645137454 0.652145154862546

Per integrare su (a, b) mappiamo l'intervallo in $(-1, 1)$, grazie al cambio di variabile:

$$y = \frac{2(x-a)}{b-a} - 1 \quad , \quad x = a + \frac{(y+1)(b-a)}{2} \quad .$$

Allora si ottiene:

$$\int_a^b f(x) dx = \frac{b-a}{2} \int_{-1}^1 f\left(a + \frac{(y+1)(b-a)}{2}\right) dy.$$

3.3.4 Integrali impropri

Un integrale è improprio se:

- uno dei limiti è infinito,
- la funzione diverge in un estremo ma è integrabile,
- è presente una singolarità interna.

Limiti infiniti

Trattando un integrale improprio con limiti infiniti (che può voler dire anche solo molto molto grandi), per $ab > 0$ si può fare un cambio di variabile del tipo:

$$\int_a^b f(x) dx = \int_{1/a}^{1/b} \frac{1}{t^2} f\left(\frac{1}{t}\right) dt \quad ,$$

Chiaro che un cambio di variabile del genere significa che rimaniamo a debita distanza dal punto $x = 0$. Se $ab < 0$, si divide l'integrale in due parti in modo che l'integrale che contiene lo zero non sia trasformato.

Singolarità integrabile all'estremo

Supponiamo che l'integrale abbia una singolarità integrabile di tipo *power law*:

$$f(x) \sim (x-a)^{-\gamma} \quad , \quad 0 \leq \gamma < 1 \quad ,$$

si usa la mappatura $t = (x-a)^{1-\gamma}$ così che si ottenga:

$$\int_a^b f(x) dx = \frac{1}{1-\gamma} \int_0^{(b-a)^{1-\gamma}} t^{\frac{\gamma}{1-\gamma}} f\left(t^{\frac{1}{1-\gamma}} + a\right) dt ,$$

con $b > a$. Similmente se la singolarità è in b :

$$\int_a^b f(x) dx = \frac{1}{1-\gamma} \int_0^{(b-a)^{1-\gamma}} t^{\frac{\gamma}{1-\gamma}} f\left(b - t^{\frac{1}{1-\gamma}}\right) dt ,$$

Alternativamente un metodo più banale ma più situazionale è aggiungere e sottrarre una funzione con stessa singolarità di cui si conosce l'integrale.

Singularità interna

Se la divergenza è in un punto interno $x = y$ vi sono due casi:

- L'integrando è tale che la divergenza è integrabile su (a, y) e (y, b) . In tal caso si divide l'intervallo e si utilizza la prescrizione per integrale improprio con limite infinito;
- Se l'integrale esiste solo come valore principale, si riorganizza l'integrando per ottenere cancellazione numerica.

Singularità di tipo Cauchy

Un integrale che andiamo spesso a risolvere in fisica è nella forma:

$$I(y) = P \int_a^b \frac{f(x)}{x-y} dx ,$$

Uno dei modi di risolvere quest'integrale, che assume che $f(x)$ sia continua nella sua derivata prima e seconda, usa l'integrazione per parti:

$$I(y) = [\ln|x-y| f(x)]_a^b - \int_a^b \ln|x-y| f'(x) dx .$$

Se necessario, si integra nuovamente:

$$\begin{aligned} I(y) &= [\ln|x-y| f(x)]_a^b - [(x-y) \ln|x-y| - (x-y)] f'(x) \Big|_a^b + \dots \\ &\dots + \int_a^b [(x-y) \ln|x-y| - (x-y)] f''(x) dx . \end{aligned}$$

Ora l'integrando finale è regolare salvo discontinuità di f'' .

Formule aperte

Una formula aperta utile è:

$$\begin{aligned} \int_{x_1}^{x_n} f(x) dx &= h [f_{3/2} + f_{5/2} + \cdots + f_{n-1/2}] + \cdots \\ &\quad \cdots + \sum_{k=1}^{\infty} \frac{B_{2k} h^{2k}}{(2k)!} (1 - 2^{-2k+1}) \left(f_n^{(2k-1)} - f_1^{(2k-1)} \right) . \end{aligned}$$

Qui i B_{2k} sono numeri di Bernoulli.

In questo caso la semplice riduzione del passo non permette il riuso diretto dei punti precedenti; tuttavia triplicando la suddivisione si possono costruire procedure di raffinamento analoghe a Romberg.

Capitolo 4

Integrazione numerica delle Equazioni Differenziali Ordinarie (ODE)

Il problema che andiamo a studiare in questa parte è la soluzione numerica di un problema del tipo $y' = f(x, y)$ con condizione iniziale $y(x_0) = y_0$ in un dominio $x_0 \leq x \leq x_f$, ridurremo anche ogni equazione differenziale ordinaria (da ora in avanti ODE) di ordine superiore ad un sistema di ODE di primo ordine:

$$y'' = f(x, y, y') \Rightarrow \begin{cases} y' = v \\ v' = f(x, y, v) \end{cases} .$$

Data una ODE del tipo $y'' = f(x, y, y')$, abbiamo due diversi problemi:

1. **Initial value problems** (IVPs): $y(x_0)$ e $y'(x_0)$ sono dati ed y e y' per x e x_f vanno trovati;
2. **Two-point value problems** (TVPs): le condizioni ai bordi sono date per più di una x , ad esempio conosciamo $y(x_0)$ e $y'(x_0)$ e vogliamo trovare y e y' per ogni x .

Per risolvere numericamente un'ODE si introduce una discretizzazione dell'asse x , costruendo una mesh del tipo $x_0 < x_1 < \dots < x_i < \dots < x_f$. L'integrazione consiste nell'avanzare la soluzione da x_i a x_{i+1} utilizzando un passo h adeguato. È quindi necessario un algoritmo che:

1. Dato $y(x_i)$, fornisca una stima di $y(x_{i+1})$. Questo costituisce il **metodo di integrazione**;
2. Uno **stepper** che adatta il passo, valuta l'errore locale associato al metodo di integrazione e decide se mantenere, aumentare o ridurre il passo h per garantire stabilità ed efficienza;
3. Un codice che gestisce l'inizio e la fine dell'integrazione fra x_0 e x_f .

4.0.1 Metodi multi-step

Data la ODE $y' = f(x, y)$ e un *time mesh* $\{x_0, x_1, \dots, x_n\}$, con $x_i = ih$, un metodo multistep di ordine p esprime y_{n+1} come combinazione lineare di alcuni valori già calcolati $y_n, y_{n-1}, \dots$ e delle corrispondenti derivate $y'_n, y'_{n-1}, \dots$ moltiplicate per lo step h (vedi [metodi lineari multistep](#)):

$$y_{n+1} = \sum_{j=0}^p a_j y_{n-j} + h \sum_{j=-1}^p b_j y'_{n-j} \quad (4.1)$$

Stiamo facendo un'espansione '*Taylor-like*', come utilizzarla? Iniziamo derivando un semplice schema esplicito al secondo ordine **Adams' type** ($\equiv a_j = 0 \ \forall \ j > 1$):

$$y_{n+1} = a_0 y_n + h [b_0 y'_n + b_1 y'_{n-1}] \quad (4.2)$$

Scegliamo una funzione polinomiale di prova, $y_{n+j} = (t + jh)^3$ e la sostituiamo in eq.4.2 ottenendo l'identità:

$$(t + h)^3 = a_0 t^3 + h[3b_0 t^2 + 3b_1(t - h)^2] \quad ,$$

espandendo e matchando i coefficienti fra membro sinistro e destro si trovano: $a_0 = 1$, $b_1 = -1/2$, $b_0 = 3/2$ ed un errore $\propto 6h^3/2$ dato che non compare un termine h^3 nel membro destro.

Metodi impliciti

I metodi impliciti sono quelli per cui $b_{-1} \neq 0$:

$$y_{n+1} = \sum_{j=0}^p a_j y_{n-j} + h \sum_{j=-1}^p b_j y'_{n-j} \quad (4.3)$$

Quindi, mentre in un metodo esplicito non comparirebbe a destra un termine con y'_{n+1} , nei metodi impliciti compare. Normalmente, si fa un guess del nuovo y_{n+1} via un metodo esplicito (in cui $b_{-1} = 0$, quindi calcoliamo y_{n+1} tramite valori conosciuti) per poi fare un certo numero di iterazioni utilizzando un metodo implicito.

Famiglie di metodi multistep di tipo Adams

Abbiamo definito gli integratori Adams' type come quelli per cui $a_j = 0 \ \forall \ j > 0$ di seguito in tabella si vedono i coefficienti per metodi di due diverse famiglie derivate da questa prima ipotesi:

- **Adams-Bashforth** (AB(0) meglio noto come **metodo di Eulero**, AB(1), ..., AB(4), ...), questi sono dei metodi espliciti che assumono la forma:

$$y_{n+1} = y_n + h \sum_{j=0}^p b_j y'_{n-j} \quad (4.4)$$

- Mentre similmente possiamo includere il termine b_{-1} per avere una forma di tipo Adams ma implicita, detta di **Adams-Moulton** (AM(-1), AM(0) o meglio noto come **metodo dei trapezi**, ..., AM(3), ...):

$$y_{n+1} = y_n + h \sum_{j=-1}^p b_j y'_{n-j} \quad (4.5)$$

ordine p	b_0	b_1	b_2	b_3	b_4
0	1				
1	3/2	-1/2			
2	23/12	-16/12	5/12		
3	55/24	-59/24	37/24	-9/24	
4	1901/720	-2774/720	2626/720	-1274/720	251/720

TAB. 4.1: Adams-Bashforth

ordine p	b_{-1}	b_0	b_1	b_2	b_3
-1	1				
0	1/2	1/2			
1	5/12	8/12	-1/12		
2	9/24	19/24	-5/24	1/24	
3	251/720	646/720	-264/720	106/720	-19/720

TAB. 4.2: Adams-Moulton

analisi dell'ordine di errore

L'idea è imporre che lo schema riproduca esattamente l'andamento di una funzione polinomiale di grado noto, in modo da determinare i coefficienti che compaiono nella combinazione lineare. Il risultato finale è un metodo che utilizza y'_n e y'_{n-1} per approssimare y_{n+1} , con un errore globale dell'ordine di h^3 .

Per la funzione $y' = 2x$, con condizione iniziale $y(0) = 0$, la soluzione esatta $y = x^2$ ma risolvendola con uno schema $y_{n+1} = y_n + 2hx_n$ vediamo che ridurre il passo di integrazione h non porta ad una precisione arbitrariamente alta. Esiste quindi un valore ottimale del passo h che minimizza l'errore totale come si vede in fig.4.1. Infatti, l'errore totale è composto da:

- l'errore di troncamento, che decresce all'aumentare della risoluzione,
- l'errore numerico di roundoff, che invece cresce se h è troppo piccolo, poiché aumenta il numero totale di operazioni.

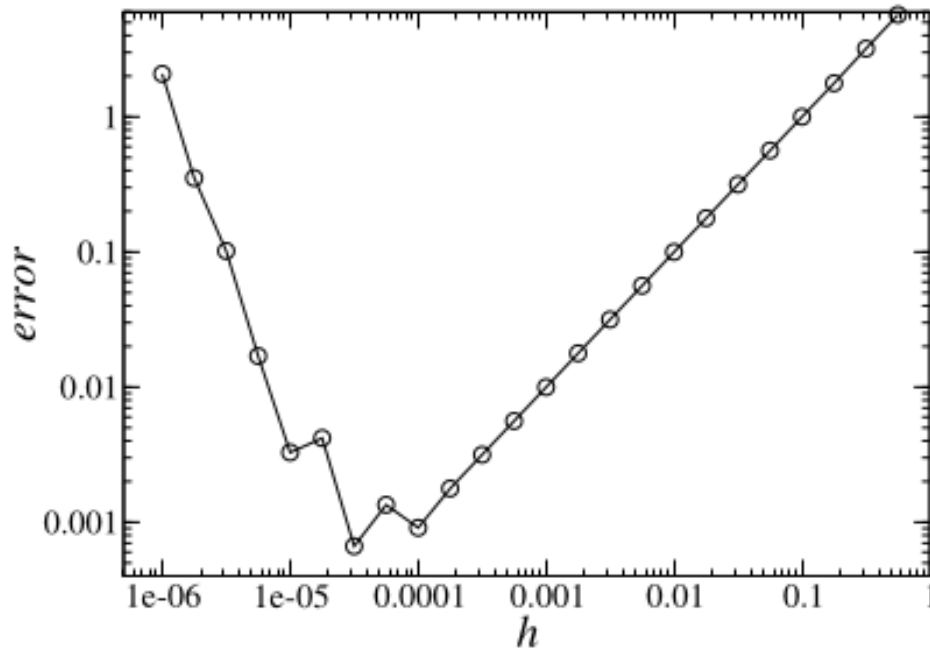


FIG. 4.1: errore di un metodo di Eulero $y_{n+1} = y_n + 2hx_n$ in funzione di h , usato per integrare la funzione $y' = 2x$ con condizione iniziale $y(0) = 0$

stabilità, radici caratteristiche, regioni di A-stabilità

Sia $\{y_i\}$ la soluzione numerica corrispondente ai punti della mesh $x_0, x_1, \dots, x_n$, con passo h . Il metodo è *stabile* se per ogni perturbazione iniziale $|z_i - y_i| < \varepsilon$ per $i < p$, la differenza rimane limitata:

$$|z_i - y_i| < C\varepsilon \quad \text{per ogni } i \geq p.$$

Il metodo **converge** se y_i è tale che $\text{Max}_{i < p} |Y_i - y_i| \rightarrow 0$ se $h \rightarrow 0$ e vale anche $\text{Max}_{i > p} |Y_i - y_i| \rightarrow 0$, allora y_i si dice convergere a Y_i .

Un metodo **consistente** ($\sum_{j=0}^p a_j = 1$) è *convergente se e solo se è stabile*.

Il problema della stabilità è difficile, notiamo però che in generale integrare è un problema legato al trovare le radici di un polinomio, esprimiamo ancora il punto y_{n+1} come combinazione lineare dei punti vicini e le derivate:

$$y_{n+1} = \sum_{j=0}^p a_j y_{n-j} + h \sum_{j=-1}^p b_j y'_{n-j},$$

dove in generale vale $\sum_{j=0}^p a_j = 1$ mentre $\sum_{j=0}^p j a_j = \sum_{j=-1}^p b_j$

La stabilità e la convergenza sono assicurate dalla *root condition*:

$$\rho(r) = r^{p+1} - \sum_{j=0}^p a_j r^{p-j}$$

questo polinomio ha $p + 1$ radici, notiamo inoltre che $\rho(1) = 0$ cioè 1 è sempre una radice. La condizione ci dice che:

- $|r_j| \leq 1$ per $j = 0, 1, \dots, p$
- $|r_j| = 1$ se r_j è una radice semplice

e - se soddisfatto - implica che il metodo sia stabile. Richiamiamo il generico *multi-step* di eq.4.1, in questo caso $p = 1$ e $a_0 = 1$, le radici di $0 = r^2 - r$ sono 0 ed 1 quindi il metodo è stabile.

Si ottengono condizioni più stringenti se si considera un'equazione differenziale test $y' = \lambda y(x)$, che ha soluzione di tipo se ci chiediamo le radici in questo caso abbiamo:

$$\rho^{n+1} = \sum_{j=0}^p a_j \rho_{n-j} + \lambda h \sum_{j=-1}^p b_j \rho_{n-j} \quad ,$$

da cui ricaveremmo $p + 1$ radici $\rho_i(\lambda h)$:

- Il metodo è **relativamente stabile** se $|\rho_j(\lambda h)| \leq \rho_0(\lambda h)$ per $j = 1, \dots, p$;
- Il metodo è **debolmente stabile** altrimenti.

Per il nostro multi-step : $a_0 = 1$, $b_0 = 3/2$, $b_1 = -1/2$, $p = 1$:

$$\rho^2 - (1 + 3\lambda h/2)\rho + \lambda h/2 = 0 \quad \Rightarrow \quad \rho = \frac{1 + 3\lambda h/2 \pm \sqrt{(1 + 3\lambda h/2)^2 - 2\lambda h}}{2}$$

$$\Rightarrow \quad \rho_{0,1} \sim 1 + \lambda h, \quad \frac{\lambda h}{2} \quad \text{il metodo è relativamente stabile, ma } \lambda < 0$$

Invece il metodo è **assolutamente stabile** se $|\rho_j(\lambda h)| < 1$ per $j = 0, 1, \dots, p$. Nel caso del nostro multi-step abbiamo scritto sopra i valori e stavolta facciamo una considerazione senza approssimazione su

$$\rho_{0,1} = \frac{1 + 3\lambda h/2 \pm \sqrt{(1 + 3\lambda h/2)^2 - 2\lambda h}}{2} \quad ,$$

che è A-stabile per $-1 < \lambda h < 0$

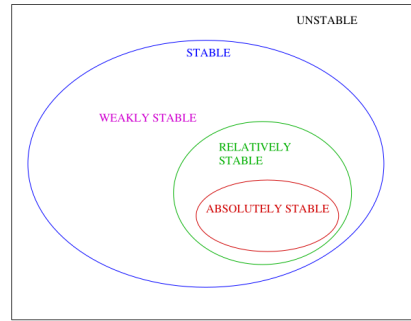


FIG. 4.2: Schema visivo per comprendere i diversi tipi di stabilità

Per un metodo numerico, la soluzione approssimata soddisfa

$$y_{n+1} = \rho(z) y_n, \quad z = h\lambda,$$

dove $\rho(z)$ è il fattore di amplificazione numerico. Una regione di stabilità è il luogo geometrico dei valori complessi $z = h\lambda$ per cui il metodo numerico NON amplifica errori quando la soluzione vera è smorzante, la *regione di stabilità assoluta* è l'insieme dei valori $z \in \mathbb{C}$ tali che $|\rho(z)| < 1$.

La figura mostra un esempio regioni di stabilità:

- in rosso: i metodi Adams–Bashforth (AB), espliciti;
- in verde: i metodi Adams–Moulton (AM), impliciti.

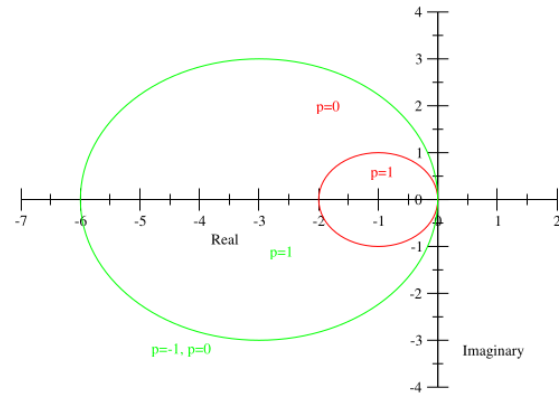


FIG. 4.3: Esempio di una regione di stabilità

Confronto di metodi multi-step

Guardiamo un primo confronto fra alcuni metodi che possiamo scrivere viste le cose affrontate:

- **metodo di Eulero**: $y_{n+1} = y_n + hy'_n + O(h^2)$, relativamente stabile;
- metodo del punto medio (**midpoint**): $y_{n+1} = y_{n-1} + 2hy'_n + O(h^3)$, debolmente stabile;
- Adams-Bashforth (**AB(2)**): $y_{n+1} = y_n + h(3y'_n - y'_{n-1})/2 + O(h^3)$, assolutamente stabile;
- **metodo dei trapezi**: $y_{n+1} = y_n + h(y'_n - y'_{n+1})/2 + O(h^3)$, stabile, per $\lambda < 0$ è assolutamente stabile.

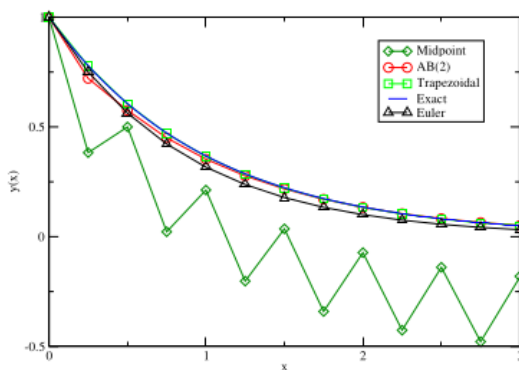


FIG. 4.4: Integrazione di $y' = -y$ con $h=0.25$ e condizioni al contorno $y(0) = 1$

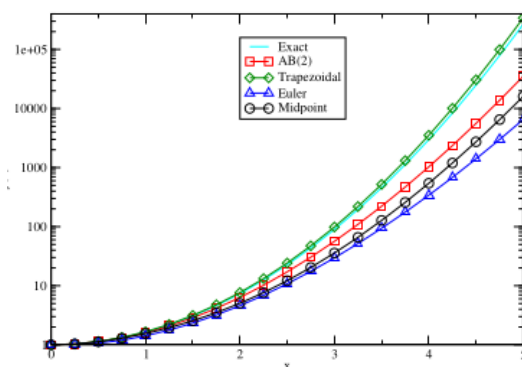


FIG. 4.5: Integrazione di $y' = xy$ con $h=0.25$ e condizioni al contorno $y(0) = 1$

Eulero che è relativamente stabile e di ordine h^2 è da considerarsi migliore del midpoint che sarà anche di ordine h^3 ma è solamente debolmente stabile. Però, in un sistema già instabile - ad esempio $y' = xy$ in cui la soluzione è un esponenziale - conta invece solo l'ordine di errore.

4.0.2 Metodi one-step

Nei metodi one-step, y_{n+1} dipende unicamente da y_n , si capisce bene come il vantaggio più grande sia non dover tenere in memoria le y_{n-j} che usavamo nei metodi multi-step.

Il metodo one-step più banale è una comune espansione di Taylor della soluzione:

$$y(t+h) = y(t) + hy'(t) + \frac{h^2}{2}y''(t) + \dots,$$

fermandosi alla derivata n -esima si ottiene un metodo numerico di ordine desiderato. Questo metodo non è generalmente utilizzato ma è molto facile da usare e appropriato quando la soluzione è di tipo polinomiale.

Poiché $y' = f(t, y(t))$, la seconda derivata si ricava tramite la regola della catena:

$$y'' = \frac{\partial}{\partial t}f(t, y) + \frac{\partial}{\partial y}f(t, y) \frac{\partial}{\partial y}y(t) = f_t(t, y) + f_y(t, y)f(t, y).$$

Un metodo di Taylor del secondo ordine è quindi dato da:

$$y_{n+1} = y_n + hf(t_n, y_n) + \frac{h^2}{2} [f_t(t_n, y_n) + f_y(t_n, y_n)f(t_n, y_n)].$$

In molti casi, metodi come il trapezoidale implicito o i metodi di Runge-Kutta permettono di raggiungere lo stesso ordine di accuratezza con un numero minore di derivate richieste.

metodi Runge-Kutta

L'idea dei metodi Runge-Kutta (RK) è quella di approssimare l'integrale

$$y(t+h) = y(t) + \int_t^{t+h} f(\tau, y(\tau)) d\tau,$$

costruendo una stima accurata della pendenza media della soluzione nel tratto $[t, t+h]$. A differenza dei metodi multistep, i metodi RK utilizzano soltanto le informazioni al punto (t, y) corrente, senza ricorrere ai valori precedenti. Per comprendere come si arriva a tali schemi, possiamo ricostruire passo per passo la procedura intuitiva che porta dapprima a un metodo di ordine due (RK2) e poi al classico metodo di ordine quattro (RK4).

Prendiamo ad esempio la funzione $y(x)$ soluzione dell'ODE $y' = f(x, y(x)) = 2\sqrt{-y}$ con condizione iniziale $y(0) = -1$, costruiamo l'approssimazione supponendo di essere arrivati ad integrare fino ad un punto x_0 e vogliamo proseguire di h :

1. come prima cosa si calcola la derivata nel punto x_0 , $y'(x_0) = f(x, y(x))|_{x=x_0}$, come si vede in verde in figura;
2. ci si sposta di $k_1 = hf(t, y)$ arrivando nel punto di ascissa $x_f = x_0 + h$ ma non lungo la soluzione poichè si è proseguiti linearmente lungo la retta verde, qui si calcola $k_2 = hf(t_n + h, y_n + k_1)$, come si vede in magenta in figura;
3. si prende la media dei due spostamenti sulle ordinate k_1 e k_2 :

$$y_{n+1} = y_n + \frac{k_1 + k_2}{2}.$$

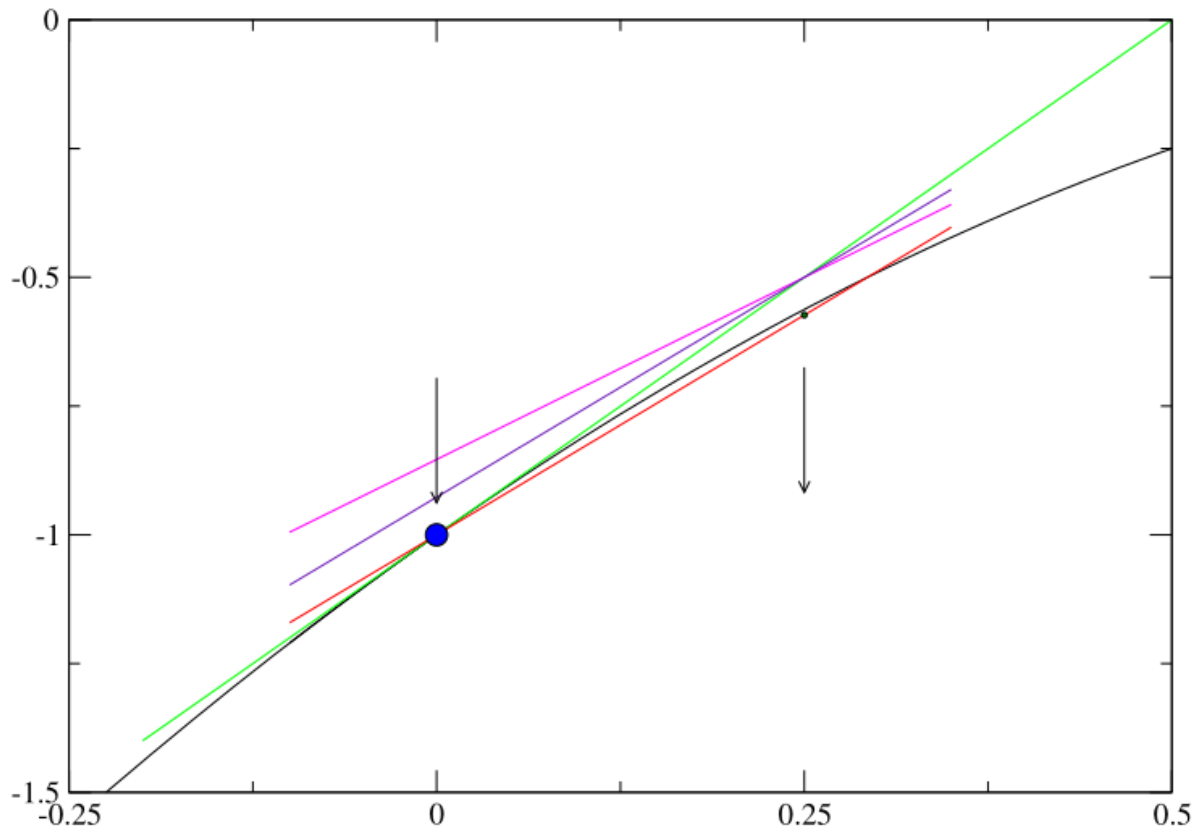


FIG. 4.6: esempio visuale di applicazione di Runge-Kutta

I passaggi spiegati sono la base del Runge-Kutta e corrispondono ad un metodo del secondo ordine.

Per ottenere ordini superiori, si introducono valutazioni intermedie della derivata. L'idea è approssimare sempre meglio il valore della pendenza media integrale, scegliendo opportunamente ascisse intermedie e combinazioni lineari delle derivate.

Un metodo RK di ordine quattro può utilizzare quattro valutazioni:

$$k_1 = hf(t_n, y_n),$$

$$k_2 = hf\left(t_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right),$$

$$k_3 = hf\left(t_n + \frac{h}{2}, y_n + \frac{k_2}{2}\right),$$

$$k_4 = hf(t_n + h, y_n + k_3).$$

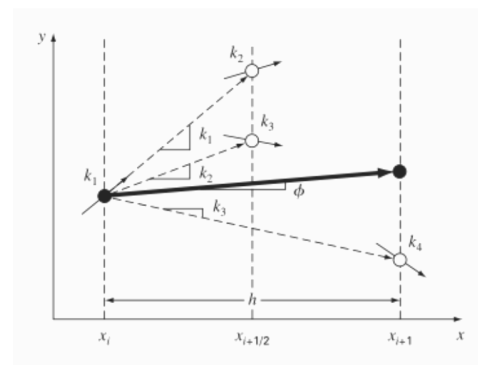


FIG. 4.7: Visualizzazione delle 4 valutazioni da fare in un RK4

Ogni k_i è una stima della derivata in un punto diverso dell'intervallo, costruita per compensare l'errore di espansione di Taylor del passo precedente. Il valore numerico della soluzione viene aggiornato combinando i contributi:

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad . \quad (4.6)$$

Questa particolare combinazione lineare è scelta in modo che l'errore globale sia dell'ordine h^4 , e che l'espansione in serie di Taylor del metodo coincida con quella della soluzione esatta fino ai termini di ordine h^4 . Il metodo risultante è il molto utilizzato *Runge-Kutta del quarto ordine* (**RK4**).

Ricorda le parole del Mannella: Runge-Kutta va usato con passo temporale variabile, a passo fisso fa schifo!

Controllo del passo mediante Richardson

Una tecnica classica per stimare l'errore locale di un metodo one-step consiste nell'applicare lo stesso algoritmo con due passi diversi, h e $2h$, e confrontare i risultati. Sia $Y(x_n)$ la soluzione esatta e $y_h(x_n)$ l'approssimazione ottenuta con passo h e $y_{2h}(x_n)$ quella ottenuta con passo $2h$. Per un metodo di ordine due (come il trapezoidale), vale lo sviluppo:

$$Y(x_n) - y_h(x_n) = D(x_n)h^2 + O(h^3), \quad Y(x_n) - y_{2h}(x_n) = 4D(x_n)h^2 + O(h^3).$$

Eliminando il coefficiente ignoto $D(x_n)$ si ottiene un'approssimazione più precisa:

$$Y(x_n) \approx \frac{4y_h(x_n) - y_{2h}(x_n)}{3} + O(h^3) \quad ,$$

quindi una stima dell'errore è data da:

$$Y(x_n) - y_h(x_n) \approx \frac{y_h(x_n) - y_{2h}(x_n)}{3} \quad .$$

Questa informazione permette di regolare dinamicamente il passo di integrazione per mantenere l'errore entro una tolleranza prefissata. L'unico limite pratico è che, per stimare l'errore, occorre disporre sia dell'integrazione a passo h sia di quella a passo $2h$.

x	$y_h(x)$	$y_{2h}(x)$	Errore stimato	$Y(x) - y_h(x)$
1.0	0.49602	0.48314	0.00429	0.00398
2.0	0.33099	0.32361	0.00246	0.00234
3.0	0.24852	0.24389	0.00154	0.00148
4.0	0.19899	0.19484	0.00105	0.00101
5.0	0.16594	0.16366	0.00076	0.00073

TAB. 4.3: Stima dell'errore tramite Richardson (Atkinson, $y' = -y^2$, $y(0) = 1$, $h = 0.25$).

Il metodo del midpoint modificato

Il metodo del midpoint modificato è un algoritmo one-step composto che permette di muovere la soluzione da x a $x + H$ suddividendo l'intervallo in n sotto-passi di ampiezza $h = H/n$. Si introducono le variabili ausiliarie z_m , con:

$$z_0 = y(x), \quad z_1 = z_0 + hf(x, z_0),$$

e la ricorrenza centrale:

$$z_{m+1} = z_{m-1} + 2h f(x + mh, z_m), \quad m = 1, 2, \dots, n-1.$$

Il valore finale è ottenuto tramite:

$$y(x + H) \approx y_n = \frac{1}{2}(z_n + z_{n-1} + hf(x + H, z_n)).$$

L'aspetto notevole di questo metodo è che l'errore dipende soltanto da potenze pari di h , cioè:

$$y(x + H) - y_n = \sum_{k=1}^{\infty} \alpha_k h^{2k}.$$

Questa proprietà lo rende particolarmente adatto all'accoppiamento con metodi di estrapolazione, riducendo drasticamente il numero di valutazioni della funzione richiesta per ottenere alta accuratezza.

Richardson extrapolation e metodo di Burlisch-Stoer

La *Richardson extrapolation* si basa sull'idea che l'approssimazione ottenuta da un metodo one-step su un intervallo di ampiezza H dipende analiticamente dal passo elementare $h = H/n$. Applicando l'algoritmo elementare con $n = 1, 2, 4, 8, \dots$ sotto-passi, si ottengono approssimazioni $y_1, y_2, y_4, y_8, \dots$, che possono essere estrapolate nel limite $n \rightarrow \infty$ (ossia $h \rightarrow 0$) utilizzando una funzione razionale

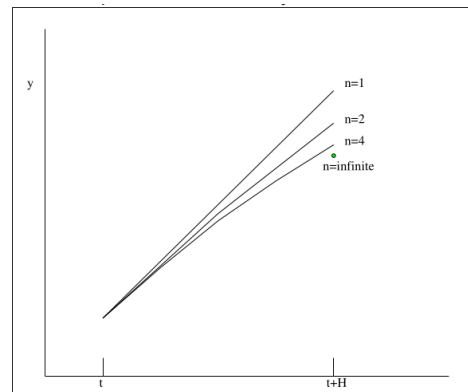


FIG. 4.8: Idea dietro l'estrapolazione di Richardson

$$R_{i(i+1)\dots(i+m)} = \frac{p_0 + p_1 x + \dots + p_\mu x^\mu}{1 + q_1 x + \dots + q_\nu x^\nu}, \quad x = \frac{H}{n}.$$

Questo è il risultato del **metodo di Burlisch-Stoer** che combina queste due idee:

1. usa il *midpoint modificato* come metodo elementare di integrazione su $[x, x + H]$;
2. applica la Richardson extrapolation ai risultati ottenuti per $n = 1, 2, 4, \dots$;
3. costruisce una matrice di estrapolazione (tabella di Richardson) da cui selezionare la stima più accurata.

La scelta del passo H e del numero di sotto-passaggi n è adattata dinamicamente per minimizzare il costo computazionale mantenendo al contempo l'errore entro limiti prestabiliti. Il metodo Burlisch-Stoer è estremamente efficiente per funzioni sufficientemente regolari ed è in grado di raggiungere ordini molto elevati, pur senza ricorrere a valutazioni multiple di derivate alte.

Metodi Runge-Kutta Hamiltoniani

Consideriamo un sistema Hamiltoniano del tipo: $H(q, p) = \frac{p^2}{2} + V(q)$, con equazioni del moto:

$$\dot{q} = \frac{\partial H}{\partial p}, \quad \dot{p} = -\frac{\partial H}{\partial q},$$

e date certe condizioni al contorno $p(0)$ and $q(0)$. Vista la forma di RK4 pensiamo di potervi costruire una *contact transformation*, ovvero una mappa da q, p a Q, P dove questi ultimi possono essere pensati come un'evoluzione a un tempo successivo di q, p .

L'evoluzione generata da tali equazioni è una trasformazione *simplettica*, cioè preserva la struttura geometrica del flusso Hamiltoniano, come le trasformazioni canoniche studiate a meccanica classica (vedi capitolo settimo di 'Fisica teorica I: meccanica' di Landau-Lifšits).

I metodi numerici come i Runge-Kutta classici, non sono in generale simplettici e possono introdurre deriva energetica nelle integrazioni di lungo periodo.

Idea generale. Un metodo Runge-Kutta Hamiltoniano (o simplettico) cerca di costruire una mappa

$$(q_n, p_n) \mapsto (q_{n+1}, p_{n+1})$$

che sia a sua volta una trasformazione simplettica e che approssimi il flusso esatto dopo un passo h .

Per un metodo RK con coefficienti $\{a_i, b_i\}$, la trasformazione RK simplettica di ordine n assume la forma:

$$p_j = p_{j-1} + hb_j F(q_{j-1}), \quad q_j = q_{j-1} + ha_j p_j,$$

che non sono nient'altro che l'implementazione delle equazioni di Hamilton, dove $F(q) = -V'(q)$ e ricordiamo di avere delle condizioni iniziali p_0, q_0 con cui andiamo a valutare p_n, q_n come punti evoluti dopo un tempo h , l'aggiornamento 'sfalsato' per cui p_j dipende dai p_{j-1}, q_{j-1} precedenti ma q_j dipende da q_{j-1}, p_j serve a mantenere la simpletticità.

'Velocity Verlet' / 'Leapfrog' L'esempio più semplice è quello in cui prendiamo la forma appena scritta per q_1, p_1, q_2, p_2 ed inseriamo i coefficienti $a_1 = a_2 = \frac{1}{2}$, $b_1 = 0, b_2 = 1$:

$$\begin{aligned} p_1 &= p_0 + h \times 0 \times F(q_0) = p_0 \\ q_1 &= q_0 + h \times \frac{1}{2} \times p_1 = q_0 + \frac{h}{2} p_1 \\ p_2 &= p_1 + h \times 1 \times F(q_1) = p_0 + hF(q_1) \\ q_2 &= q_1 + h \times \frac{1}{2} \times p_2 = q_1 + \frac{h}{2} p_2 \end{aligned}$$

Interpretazione: q avanza in due metà simmetriche ($a_1 = a_2 = \frac{1}{2}$), mentre la p viene aggiornata solo a seguito di un mezzo passo in avanti.

Esempio di schema symplettico. Una scelta possibile di coefficienti *magici* è:

i	(a_i, b_i)
1	(0.515352837431123, 0.134496199277431)
2	(-0.085782019412974, -0.224819803079421)
3	(0.441583023616467, 0.756320000515668)
4	(0.128846158365384, 0.334003603286321)

Esempio pratico. Per il sistema $H(q, p) = \frac{p^2}{2} + \frac{q^4}{4} - \frac{q^2}{2}$, la comparazione tra un RK standard e un RK Hamiltoniano mostra che il metodo symplettico conserva l'Hamiltoniana in modo molto più accurato nel tempo, evitando il drift energetico tipico dei metodi non-symplettici.

4.0.3 Equazioni stiff

Un'equazione differenziale si dice *stiff* quando presenta scale temporali molto diverse: la soluzione varia lentamente su larga scala, ma la dinamica interna dell'equazione contiene componenti che decadono estremamente velocemente. In questi casi, i metodi espliciti richiedono passi h molto piccoli per ragioni di stabilità, anche quando la soluzione vera non lo richiederebbe.

Esempio semplice. Immaginiamo di dover risolvere un problema come un sistema

$$\begin{cases} u'(x) = 998 u(x) + 1998 v(x), & u(0) = 1 \\ v'(x) = -999 u(x) - 1999 v(x), & v(0) = 0 \end{cases}$$

Si può notare che le soluzioni analitiche esistono e sono $u(x) = 2e^{-x} - e^{-1000x}$ e $v(x) = -e^{-x} + e^{-1000x}$, ma dovendo risolvere tramite metodi numerici finiremmo per usare time step troppo piccoli per sopperire alla presenza di modi con tempi scala molto lontani fra loro, lo si vede subito scrivendo la matrice del sistema

$$A = \begin{pmatrix} 998 & 1998 \\ -999 & -1999 \end{pmatrix} \quad \lambda_{1,2} = -1, -1000$$

Come affrontare il problema? Affrontiamo il problema grazie a metodi impliciti/semi-impliciti ad ordine basso, in particolare presa $y' = f(x, y(x))$, si linearizza un metodo di Euler implicito:

$$y_{n+1} = y_n + h f(x_{n+1}, y_{n+1})$$

$$y_{n+1} = y_n + h \left[f(x_{n+1}, y_n) + \frac{\partial f}{\partial y_n}(y_{n+1} - y_n) \right]$$

Portando a sinistra il termine con y_{n+1} e dividendo i due membri rimane:

$$y_{n+1} = \left[1 - h \frac{\partial f}{\partial y_n} \right]^{-1} \left(y_n + h f(x_{n+1}, y_n) - h \frac{\partial f}{\partial y_n} y_n \right)$$

Formalmente, definita $\mathbf{A}$ la matrice del nostro sistema lineare (matrice definita positiva, $\lambda_i > 0 \forall i$) e dato $\mathbf{y}' = -\mathbf{A}\mathbf{y}$, utilizzando un metodo di eulero esplicito abbiamo:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{y}'_n = (1 - h\mathbf{A})\mathbf{y}_n \equiv \mathbf{B}\mathbf{y}_n$$

e $\mathbf{B}^n \rightarrow 0$ per $n \rightarrow \infty$ solo se gli autovalori $(1 - \lambda_i h) \equiv \mu_i$ di $\mathbf{B}$ sono < 1 e questo tipicamente non accade per le *stiff equations*.

Tuttavia, usando un metodo implicito che assume la forma $(Id + h\mathbf{A})\mathbf{y}_{n+1} = \mathbf{y}_n$ da non confondere con il metodo esplicito appena sviluppato sopra, abbiamo una diversa $\mathbf{B}$: $\mathbf{y}_{n+1} = (Id + h\mathbf{A})^{-1}\mathbf{y}_n \equiv \mathbf{B}\mathbf{y}_n$. Se gli autovalori di $\mathbf{A}$ sono $\lambda > 0$, allora gli autovalori di $(Id + h\mathbf{A})^{-1}$ sono $\mu_i \equiv (1 + h\lambda)^{-1} \forall h$ quindi la soluzione si comporta bene ($0 < \mu_i < 1$).

4.0.4 Boundary Value Problems (BVP)

Prendiamo di esempio un'equazione di un oscillatore armonico smorzato $y'' = -\gamma y' - \omega_0^2 y$ con una condizione iniziale $y(0) = 1$ ed un'altra condizione al tempo finale $t_f = k\pi/\omega$, dove $\omega^2 \equiv \omega_0^2 - \gamma^2/4$.

Il problema che si verifica è che data la tipica soluzione $y(t) = e^{-\gamma t/2}(A \sin(\omega t) + B \cos(\omega t))$, mentre la condizione iniziale ci porta ad imporre $A = 0$, la condizione sul punto finale non porta alcuna informazione in più per B perché abbiamo scelto il punto finale in un nodo dell'oscillazione portandoci ad avere $y(t_f) = e^{-\gamma t_f/2}(-1)^k$ che non contiene né B né A quindi non ci aiuta a trovare una soluzione univoca.

Abbiamo quindi un grosso problema: *la soluzione non è unica!*

Vediamo due diversi modi per affrontare il problema.

Shooting method

Come si può immaginare dal nome, qui *spariamo* a caso.

Ripartiamo dallo stesso problema di oscillatore armonico smorzato e immaginiamo di voler imporre la condizione al tempo finale $y(t_f) = y(5.02) = 0.418$.

Diamo dei valori alle costanti, come $\gamma = 0.3$ ed $\omega_0 = 1$, a questo punto siamo costretti per costruire una simil-soluzione a imporre un'altra condizione iniziale, come la derivata $y'(0) = s$, fatto ciò integriamo fino al punto finale.

Ripetiamo la costruzione per diversi valori di s fino ad averne uno che approssimi bene la condizione che avremmo voluto imporre: $y(5.02, s) \approx 0.418$.

metodo di rilassamento / differenze finite

Il metodo consiste nel mettere un tentativo di soluzione, *initial guess*, fra x_0 ed x_f e 'rilassarlo' fino a trovare la soluzione corretta.

Partiamo ad esempio da una $y'' = f(x, y, y')$ con condizioni al bordo $y(x_0) = g_0$ e $y(x_n) = g_n$. Si assume una mesh di n punti spaziali di un passo h , usando le formule delle differenze finite possiamo scrivere:

$$\frac{y_{n-1} - 2y_n + y_{n+1}}{h^2} = f(x_n, y_n, \frac{y_{n+1} - y_{n-1}}{2h}) \equiv f_n$$

dove y_0 e y_n sono condizioni date in precedenza, riscriviamo in forma matriciale il problema:

$$\frac{1}{h^2} \begin{pmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \cdots & 0 \\ 0 & 1 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & -2 & 1 \\ 0 & \cdots & 0 & 1 & -2 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_{N-1} \end{pmatrix} - \begin{pmatrix} g_0/h^2 \\ 0 \\ \vdots \\ 0 \\ g_n/h^2 \end{pmatrix}$$

che può essere scritto come $\frac{1}{h^2} \mathbf{A} \mathbf{Y} \equiv \mathbf{F}(\mathbf{Y}) + \mathbf{G}$ e da questa forma si risolve con metodo di Newton dopo aver linearizzato $\mathbf{F}(\mathbf{Y})$:

$$\mathbf{y}^{(m+1)} = \mathbf{y}^{(m)} - \left[\frac{\mathbf{A}}{h^2} - \frac{\partial \mathbf{F}_i}{\partial y_j} \right]^{-1} \left[\frac{\mathbf{A} \mathbf{y}^{(m)}}{h^2} - \mathbf{F}(\mathbf{y}^{(m)}) - \mathbf{G} \right]$$

A questo punto possiamo dare un guess iniziale $y^{(0)}$ ed iterare.

Capitolo 5

Integrazione numerica di equazioni differenziali alle derivate parziali (PDEs)

5.1 Introduzione

Esistono principalmente tre tipi di equazioni differenziali alle derivate parziali (PDE, *partial differential equation*):

- **Iperboliche:** come ad esempio l'equazione delle onde

$$\boxed{\frac{\partial^2 u}{\partial t^2} - v^2 \frac{\partial^2 u}{\partial x^2} = 0} \quad ;$$

- **Paraboliche:** come ad esempio l'equazione di diffusione

$$\boxed{\frac{\partial P}{\partial t} - \frac{\partial}{\partial x} \left(D \frac{\partial}{\partial x} P \right) = 0} \quad ;$$

- **Ellittiche:** come ad esempio l'equazione di Poisson

$$\boxed{\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \rho(x, y)} \quad .$$

La differenza computazionalmente sta nel fatto che il problema può essere:

- *Initial value problem*;
- *Boundary value problem*.

Nel primo caso dobbiamo stare ben attenti alla *stabilità*, mentre nel secondo il problema maggiore è l'*efficienza*.

Applicazione

Mettiamo che io prenda $\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \rho(x, y)$, prima cosa che vorremmo fare è introdurre uno *spacing* Δ in modo che $x_j = x_0 + j\Delta$ (con $j=0, \dots, J$) e $y_\ell = y_0 + j\Delta$ (con $\ell=0, \dots, L$). Possiamo esprimere le derivate come spiegato nel paragrafo sulle differenze finite:

$$\frac{u_{j+1,\ell} - 2u_{j,\ell} + u_{j-1,\ell}}{\Delta^2} + \frac{u_{j,\ell+1} - 2u_{j,\ell} + u_{j,\ell-1}}{\Delta^2} = \rho_{j,\ell}$$

Il modo di soluzione che introduciamo ora passa attraverso l'introdurre un indice $i = j(L+1) + \ell$ (per $j=0$ va da 0 ad L , per $j=1$ va da $L+1$ a $2L+1$ e così via...) così che u diventi un vettore ed il problema diventi un problema di algebra lineare:

$$u_{i+L+1} + u_{i-(L+1)} + u_{i+1} + u_{i-1} - 4u_i = \Delta^2 \rho_i, \quad ,$$

$$\mathbf{A} \vec{u} = \vec{b}$$

dove in questo caso ci si può convincere che

$$\mathbf{A} = \left[\begin{array}{ccc|ccc|ccc} -4 & 1 & & 1 & & & & & \\ & 1 & -4 & 1 & & & & & \\ & & 1 & -4 & & & & & \\ \hline & 1 & & & -4 & 1 & & 1 & \\ & & 1 & & 1 & -4 & 1 & & 1 \\ & & & 1 & & 1 & -4 & & \\ \hline & & & 1 & & & & -4 & 1 \\ & & & & 1 & & & 1 & -4 & 1 \\ & & & & & 1 & & & 1 & -4 \end{array} \right]$$

Come risolvere $\mathbf{A}\vec{u} = \vec{b}$?

- quando $\mathbf{A}$ è costante con *fast Fourier methods*;
- direttamente, usando ad esempio il *gradiente coniugato* (molto accurato, non molto efficiente quando $\mathbf{A}$ è grande perché 'sparsa');
- *Rilassamento*, scrivendo $\mathbf{A} = \mathbf{E} - \mathbf{F}$ dove $\mathbf{E}$ è invertibile, si inizia da un guess $\vec{u}^0$ e si itera risolvendo $\mathbf{E}\vec{u}^{(r+1)} = \vec{b} + \mathbf{F}\vec{u}^{(r)}$

5.2 Differenze finite

Il metodo delle differenze finite è una strategia utilizzata per risolvere numericamente equazioni differenziali che, nelle sue varianti, si basa sull'approssimazione delle derivate con equazioni alle differenze finite. Viene utilizzato prevalentemente per equazioni differenziali ordinarie, anche se il metodo viene sfruttato come schema di avanzamento nel tempo per problemi alle derivate parziali.

$$u_{i+1} = u_i + u'_i dx + \frac{1}{2} u''_i dx^2 + \dots$$

$$\frac{u_{i+1} - u_i}{dx} = u'_i + \frac{1}{2}u''_i dx + \dots$$

$$u_{i+1} - u_{i-1} = 2u'_i dx + \frac{1}{6}u'''_i dx^3$$

Che cosa vuol dire? che la derivata nel punto i -esimo può essere in qualche modo ricavata attraverso valori della funzione u in i e nei punti vicini $i+1$, $i-1$, etc. .

Possiamo poi stimare l'errore di queste stime grazie alle serie di Taylor.

$$u'_{i=0} = \frac{1}{2dx} (-u_{-1} + u_1) + \dots dx^2 u'''_0 \quad u'_0 = \frac{1}{12dx} [u_{-2} - 8u_{-1} - 8u_1 - u_2] + \dots dx^4 u^{(5)}_0$$

$$\text{con al bordo: } u'_0 = \frac{1}{dx} (-u_0 + u_1) \quad \text{oppure} \quad u'_0 = \frac{1}{2dx} (-3u_0 + 4u_1 - u_2)$$

$$u''_0 = \frac{1}{dx^2} (u_{-1} - 2u_0 + u_1) + \dots dx^2 u^{(4)}_0 \quad u''_0 = \frac{1}{12dx^2} (-u_{-2} + 16u_{-1} - 30u_0 + 16u_1 - u_2) + \dots dx^2 u^{(6)}_0$$

$$\text{con al bordo: } u''_0 = \frac{1}{dx^2} (-2u_0 - 5u_1 + 4u_2 - u_3)$$

5.3 Flux-conservative PDEs

Molte equazioni differenziali sono nella forma di un'equazione che conserva il flusso:

$$\boxed{\frac{\partial}{\partial t} \mathbf{u} = - \frac{\partial}{\partial x} \mathbf{F}(\mathbf{u})} \quad , \quad (5.1)$$

dove $\mathbf{F}(u)$ è il flusso della quantità.

Ad esempio, anche l'equazione standard iperbolica $\frac{\partial^2 u}{\partial t^2} = v^2 \frac{\partial^2 u}{\partial x^2}$ può essere scritta, definendo $s \equiv \frac{\partial}{\partial t} u$ e $r \equiv v \frac{\partial}{\partial x} u$, come un'equazione del tipo sopra definito:

$$\frac{\partial}{\partial t} s = v \frac{\partial}{\partial x} r \quad , \quad \frac{\partial}{\partial t} r = v \frac{\partial}{\partial x} s \quad ,$$

definendo $s \equiv \frac{\partial}{\partial t} u$ e $r \equiv v \frac{\partial}{\partial x} u$.

5.3.1 Forward Time, Centered Space

Altro esempio di PDS che possiamo scrivere in forma flux-conservative è $\boxed{\frac{\partial u}{\partial t} = v \frac{\partial u}{\partial x}}$ se considerato $\mathbf{F}(u) = vu$. Prendiamo appunto $\frac{\partial u}{\partial t} = v \frac{\partial u}{\partial x}$, usando $x_j = x_0 + j\Delta x$ ($j=0, \dots, J$) e $t_n = t_0 + n\Delta t$ ($n=0, \dots, N$) andiamo a discretizzare $u_j^n \equiv u(t_n, x_j)$ e ancora una volta esprimiamo le derivate a meno di un errore:

$$\frac{\partial u}{\partial t}|_{j,n} = \frac{u_j^{n+1} - u_j^n}{\Delta t} + O(\Delta t) \quad \frac{\partial u}{\partial x}|_{j,n} = \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} + O(\Delta x^2)$$

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = -v \left(\frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} \right) \quad (5.2)$$

A partire da eq.5.2 sviluppiamo la soluzione chiamata *forward time centered space* (**FTCS**), forma esplicita in cui esprimiamo i valori 'forward' u^{n+1} nel tempo in funzione di u^n e centrata nello spazio:

$$u_j^{n+1} = u_j^n - v \Delta t \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} \quad (5.3)$$

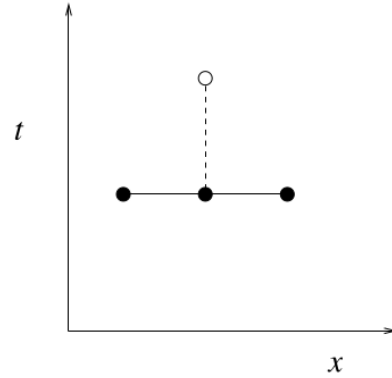


FIG. 5.1: Visualizzazione del metodo *forward time centered space*, in cui vengono usati 3 punti $j-1, j, j+1$ al tempo n -esimo per il calcolo al tempo $n+1$ -esimo in posizione j .

Analisi di stabilità

Possiamo fare un'analisi di stabilità di Von Neumann arrivati a questo punto: l'idea è quella di studiare la stabilità lineare di un'onda, assumendo $u_j^n = e^{i(kx - \omega t)}$ e definito $\xi \equiv e^{i\omega \Delta t}$ si ha $u_j^n = \xi^n e^{ikj\Delta x}$ dove ξ^n rappresenta l'ampiezza al tempo t_n (quindi $e^{-i\omega t} = (e^{-i\omega \Delta t})^n = \xi^n$).

Sostituendo u_j^n in eq.5.3 si ottiene:

$$\xi = 1 + i \frac{v \Delta t}{\Delta x} \sin(k \Delta x) \quad .$$

Risulta chiaro che $|\xi| > 1$ quindi secondo quest'analisi per ogni scelta di Δx e Δt , il metodo è instabile $\forall k$.

5.3.2 Metodo di Lax

Il metodo di Lax consiste nel sostituire

$$u_j^n \rightarrow \frac{u_{j+1}^n - u_{j-1}^n}{2}$$

all'interno di eq.(5.3) così da fare un'analisi di stabilità di Von Neumann sulla forma:

$$u_j^{n+1} = \frac{u_{j+1}^n - u_{j-1}^n}{2} + v\Delta t \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x}, \quad (5.4)$$

$$\xi = \cos(k\Delta x) - i \frac{v\Delta t}{\Delta x} \sin(k\Delta x).$$

dove abbiamo usato ancora $u_j^n = \xi^n e^{ikj\Delta x}$.

Ora per la stabilità si ottiene $|\xi|^2 = \cos^2(k\Delta x) + \left(\frac{v\Delta t}{\Delta x}\right)^2 \sin^2(k\Delta x)$, che può essere stabile ($|\xi| < 1$) se viene rispettato il vincolo $\frac{|v|\Delta t}{\Delta x} \leq 1$

Cosa c'è dietro il metodo di Lax

Lax in più dimensioni

Errori del metodo di Lax

5.3.3 Metodo staggered leap-frog

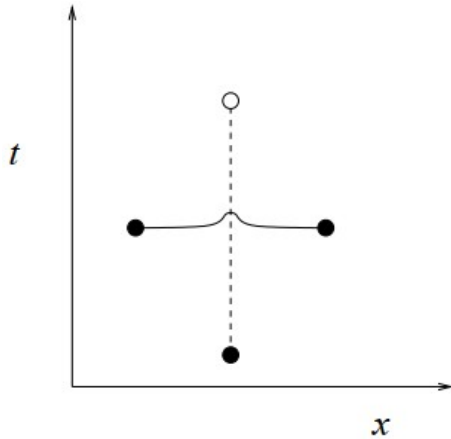


FIG. 5.3: Visualizzazione del metodo *staggered leap frog*.

5.3.4 Metodo two-step Lax-Wendroff

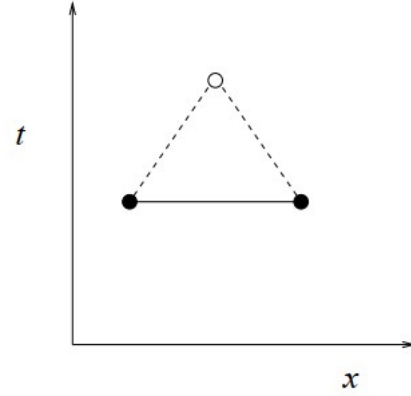


FIG. 5.2: Visualizzazione del metodo di Lax, per avanzare nel tempo a $(n+1, j)$ si usano i punti $(n, j-1)$ e $(n, j+1)$, non più il punto (n, j) che veniva usato in FTCS.

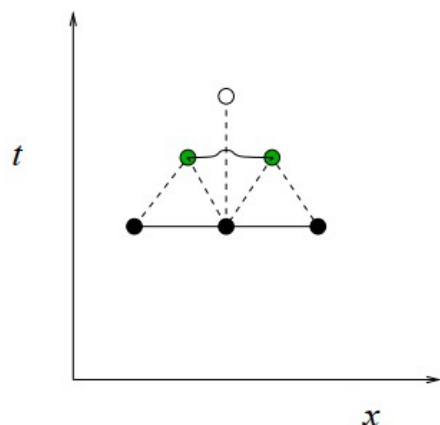


FIG. 5.4: Visualizzazione dello schema *Two-Step Lax-Wendroff*.

5.3.5 Schroedinger e operatori split

5.4 Diffusive initial value problems

Schemi impliciti

Second'ordine

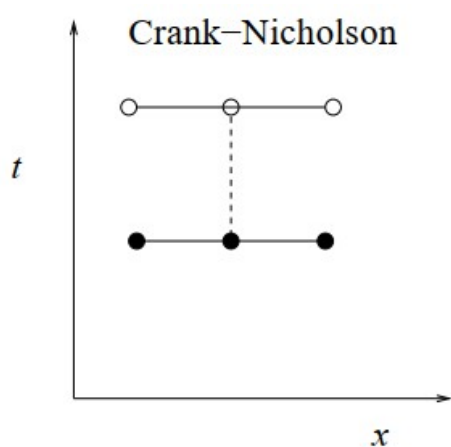


FIG. 5.5: Visualizzazione dello schema *Crank-Nicholson*.

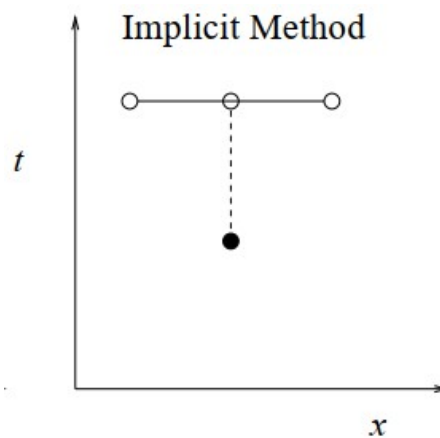


FIG. 5.6: Visualizzazione dello schema *implicit*.

In più dimensioni

5.5 Metodi di Fourier

5.6 Cyclic Reduction

5.7 Metodi di rilassamento

5.7.1 Una soluzione pratica

Partiamo dall'idea di risolvere un'equazione differenziale come ad esempio l'importante **equazione di Burgers**

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = 0 \quad ,$$

alla quale in più potremmo aggiungere un termine $\nu \frac{\partial^2 u}{\partial x^2}$ che rappresenti la viscosità. Immaginiamo di poter linearizzare $u = u_0 + \delta u$ tale che $\langle u \rangle = u_0$ e $|\delta u| \ll |u_0|$.

$$u \frac{\partial u}{\partial x} = (u_0 + \delta u) \frac{\partial}{\partial x} (u_0 + \delta u) = u_0 u'_0 + u_0 \delta u' + \delta u u'_0 + \delta u \delta u'$$

Riconduciamo l'equazione a:

$$\frac{\partial(\delta u)}{\partial t} + u_0 \frac{\partial(\delta u)}{\partial x} + \cancel{\delta u \frac{\partial u_0}{\partial x}} = 0$$

Cosa significherebbe risolvere questa equazione differenziale? innanzitutto nella pratica divideremo lo spazio discretamente e di conseguenza la u allo stesso modo.

Andiamo a supporre una soluzione $u(x, t) = U(x - ct)$ che si propaga a velocità costante.

Se noi andiamo a scrivere un programma per far propagare un'onda potremmo usare ciò che abbiamo visto nella sezione su *Forward Time Centered Space*.

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = -u \left(\frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} \right)$$

Vediamo un esempio di programma in c++ con $u=1$ di fronte la derivata spaziale:

```

1  #include <iostream>
2  #include <fstream>
3  #include <cmath>
4
5  using namespace std;
6
7  // EQUAZIONE DEL TIPO:    du/dt = -c * du/dx
8
9  int main()  {
10
11     const int N = 200;
12     const int M = 2000;
13     const double pi = 3.14159265358979323846;
14     //float XX[N], tt[M];
15     double ff[M][N];
16     const double L = 2*pi, dx= L / N;
17     const double T = 10.0, dt= T / M;
18
19     for(int j=0; j<N; j++){                                // carico
        inizialmente la condizione iniziale u(x, t=0)
    }

```

```

20         ff[0][j] = sin( 2* j * dx ) + sin( 10 * j * dx ) / 8
21         ;
22     }
23     /* questa è la nostra funzione al tempo t=0 per ogni x
24     andiamo a calcolare l'evoluzione temporale con
25     forward time e centered space*/
26
27     for (int k = 1; k < M-1; k++){
28
29
30         for (int i = 1; i < N-1; i++){
31             // Ho spostato la derivata direttamente qui dentro
32             double ksucc = (ff[k-1][i+1] - ff[k-1][i-1]) / (2
33                 * dx);
34
35             ff[k][i] = ff[k-1][i] + dt*ksucc;
36         }
37
38         // condizioni al contorno
39         ff[k][0] = ff[k-1][0] + dt * ( ff[k-1][1] - ff[k-1][N
40             -1] ) / ( 2 * dx );
41         ff[k][N-1] = ff[k-1][N-1] + dt * ( ff[k-1][0] - ff[k
42             -1][N-2] ) / ( 2 * dx );
43     }
44
45     // SALVATAGGIO SU FILE PER PYTHON
46     ofstream file("output.txt");
47     if (!file) {
48         cout << "Errore apertura file!\n";
49         return 1;
50     }
51
52     for (int k = 0; k <= M; ++k) {
53         for (int i = 0; i < N; ++i) {
54             file << ff[k][i] << "\n";
55         }
56         file << "\n";
57     }
58
59     file.close();
60     cout << "File output.txt generato!\n";
61
62     return 0;
63 }

```

Listing 5.1: Burgers: applicazione all'evoluzione di un'ode con FTCS

Aggiungiamo il termine di dissipazione per ottenere $\frac{\partial u}{\partial t} = -u \frac{\partial u}{\partial x} + \nu \frac{\partial^2 u}{\partial x^2}$ così che la

eq.5.3 diventi questa volta:

$$u_j^{n+1} = u_j^n + \Delta t \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} + \nu \Delta t \frac{u_{j-1}^n - 2u_j^n + u_{j+1}^n}{\Delta x^2} .$$

```

1  #include <iostream>
2  #include <fstream>
3  #include <cmath>
4
5  // EQUAZIONE DEL TIPO:    du/dt + c * du/dx = d^2u/dx^2
6
7  using namespace std;
8  const int N = 50;
9
10 // Derivata prima / differenze finite / condizioni al contorno
    periodiche
11 inline double derivata_prima(double ff[][N], int k, int i,
    double dx, int N){
12     int ip = (i + 1) % N;
13     int im = (i - 1 + N) % N;
14
15     return (ff[k][ip] - ff[k][im]) / (2.0 * dx);
16 }
17
18 // Derivata seconda / differenze finite / condizioni al
    contorno periodiche
19 inline double derivata_seconda(double ff[][N], int k, int i,
    double dx, int N){
20     int ip = (i + 1) % N;
21     int im = (i - 1 + N) % N;
22
23     return (ff[k][ip] - 2.0 * ff[k][i] + ff[k][im]) / (dx * dx
        );
24 }
25
26 // Avanzamento in avanti nel tempo
27 inline double forward_time(double u, double dudx, double dt,
    double c ){
28     return u + dt * dudx;
29 } // il segno meno per il termine du/dx è stato incluso nella
    chiamata della funzione
30
31 int main() {
32     const int M = 1000;
33     const double pi = 3.14159265358979323846;
34
35     static double ff[M][N];
36     const double L = 2*pi, dx= L / N;
37     const double T = 10, dt= T / M;
38     double v = 0.008; /*viscosità cinematica*/
39

```

```

40     for(int j=0; j<N; j++){
41         ff[0][j] = sin( 2* j * dx );
42     }
43
44     for (int k = 1; k < M; k++){
45
46         for (int i = 0; i < N; i++){
47             double ksucc = (-derivata_prima(ff, k-1, i, dx, N)
48                 + v*derivata_seconda(ff, k-1, i, dx, N));
49
50             ff[k][i] = forward_time(ff[k-1][i], ksucc, dt,
51                 1.0);
52         }
53
54         // SALVATAGGIO SU FILE PER PYTHON
55         ofstream file("output.txt");
56         if (!file) {
57             cout << "Errore apertura file!\n";
58             return 1;
59         }
60
61         for (int k = 0; k < M; ++k) {
62             for (int i = 0; i < N; ++i) {
63                 file << ff[k][i] << "\n";
64             }
65             file << "\n";
66         }
67
68         file.close();
69         cout << "File output.txt generato!\n";
70
71         return 0;
72     }

```

Listing 5.2: upgrade dell'applicazione precedente

Capitolo 6

Metodo delle differenze finite

Introduzione

Il metodo delle differenze finite è una tecnica numerica utilizzata per approssimare le derivate di una funzione mediante valori della stessa in punti discreti.

Conosciamo bene la definizione di derivata come rapporto incrementale, possiamo quindi usare punti discreti della funzione per svolgere questo rapporto ed approssimare la funzione:

$$\frac{d\varphi}{dx} = \lim_{h \rightarrow 0} \frac{\varphi(x+h) - \varphi(x)}{h} \quad \text{rapporto incrementale in avanti}$$

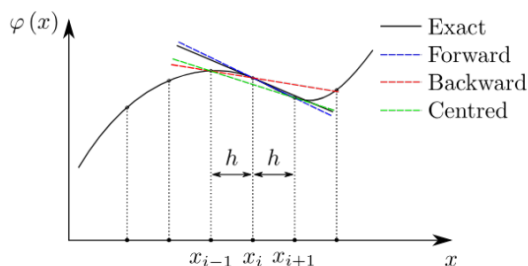
$$\frac{d\varphi}{dx} = \lim_{h \rightarrow 0} \frac{\varphi(x) - \varphi(x-h)}{h} \quad \text{rapporto incrementale all'indietro}$$

$$\frac{d\varphi}{dx} = \lim_{h \rightarrow 0} \frac{\varphi(x+h) - \varphi(x-h)}{2h} \quad \text{rapporto incrementale centrato}$$

Questo metodo ci dice che considerando il dominio $x \in [0, X]$ e ponendo la *mesh* di N punti, tale che l'intervallo sia $h = X/N$, l'approssimazione della derivata calcolata con punti discreti della funzione può essere calcolata con:

$$\frac{d\varphi}{dx}|_i \approx \frac{\varphi_{i+1} - \varphi_i}{h}, \quad \frac{d\varphi}{dx}|_i \approx \frac{\varphi_i - \varphi_{i-1}}{h}, \quad \frac{d\varphi}{dx}|_i \approx \frac{\varphi_{i+1} - \varphi_{i-1}}{2h}. \quad (6.1)$$

Nessuna di queste formule è migliore delle altre in ogni occasione, per ogni funzione che si voglia derivare.



Nel caso in figura si vede come il rapporto in avanti è qui il migliore per approssimare la derivata prima.

FIG. 6.1: Prova di applicazione del metodo delle differenze finite al calcolo della derivata prima di una generica funzione polinomiale.

Accuratezza delle approssimazioni

Per studiare l'errore utilizziamo lo sviluppo di Taylor.

$$\varphi(x) = \varphi(x_0) + (x - x_0) \left. \frac{d\varphi}{dx} \right|_{x_0} + \frac{(x - x_0)^2}{2} \left. \frac{d^2\varphi}{dx^2} \right|_{x_0} + \frac{(x - x_0)^3}{6} \left. \frac{d^3\varphi}{dx^3} \right|_{x_0} + \dots$$

Ponendo $x_i = x_0$ otteniamo per φ_{i+1} , φ_{i-1} :

$$\varphi_{i+1} = \varphi_i + (x_{i+1} - x_i) \left. \frac{d\varphi}{dx} \right|_i + \frac{(x_{i+1} - x_i)^2}{2} \left. \frac{d^2\varphi}{dx^2} \right|_i + \frac{(x_{i+1} - x_i)^3}{6} \left. \frac{d^3\varphi}{dx^3} \right|_i + \dots$$

$$\varphi_{i-1} = \varphi_i + (x_{i-1} - x_i) \left. \frac{d\varphi}{dx} \right|_i + \frac{(x_{i-1} - x_i)^2}{2} \left. \frac{d^2\varphi}{dx^2} \right|_i + \frac{(x_{i-1} - x_i)^3}{6} \left. \frac{d^3\varphi}{dx^3} \right|_i + \dots$$

Poiché $x_{i+1} - x_i = h$, si ottiene

$$\begin{aligned} \varphi_{i+1} &= \varphi_i + h \left. \frac{d\varphi}{dx} \right|_i + \frac{h^2}{2} \left. \frac{d^2\varphi}{dx^2} \right|_i + \frac{h^3}{6} \left. \frac{d^3\varphi}{dx^3} \right|_i + \dots \\ \varphi_{i-1} &= \varphi_i - h \left. \frac{d\varphi}{dx} \right|_i + \frac{h^2}{2} \left. \frac{d^2\varphi}{dx^2} \right|_i - \frac{h^3}{6} \left. \frac{d^3\varphi}{dx^3} \right|_i + \dots \end{aligned} \quad (6.2)$$

Da queste espansioni in eq.(6.2) possiamo riottenere la derivata con metodo forward e backward:

$$\begin{aligned} \left. \frac{d\varphi}{dx} \right|_i &= \frac{\varphi_{i+1} - \varphi_i}{h} - \frac{h}{2} \left. \frac{d^2\varphi}{dx^2} \right|_i - \frac{h^2}{6} \left. \frac{d^3\varphi}{dx^3} \right|_i + \dots = \\ &= \frac{\varphi_{i+1} - \varphi_i}{h} + O(h) \quad (\text{forward}) \\ \left. \frac{d\varphi}{dx} \right|_i &= \frac{\varphi_i - \varphi_{i-1}}{h} + \frac{h}{2} \left. \frac{d^2\varphi}{dx^2} \right|_i - \frac{h^2}{6} \left. \frac{d^3\varphi}{dx^3} \right|_i + \dots = \\ &= \frac{\varphi_i - \varphi_{i-1}}{h} + O(h) \quad (\text{backward}) \end{aligned} \quad (6.3)$$

Sommando le due espressioni in eq.(6.3) si ottiene

$$\left. \frac{d\varphi}{dx} \right|_i = \frac{\varphi_{i+1} - \varphi_{i-1}}{2h} - \frac{h^2}{6} \left. \frac{d^3\varphi}{dx^3} \right|_i + \dots = \frac{\varphi_{i+1} - \varphi_{i-1}}{2h} + O(h^2)$$

Questa formula ha un errore di ordine h^2 , quindi è più accurata rispetto alle approssimazioni forward e backward che hanno errore $O(h)$.

Approssimazione della derivata seconda

Sottraendo le due espansioni in eq.(6.3) si ottiene

$$0 = \frac{\varphi_{i+1} - \varphi_i}{h} - \frac{\varphi_i - \varphi_{i-1}}{h} - h \left. \frac{d^2\varphi}{dx^2} \right|_i + O(h^3)$$

da cui segue la formula standard della derivata seconda calcolata con il metodo delle differenze finite:

$$\left. \frac{d^2\varphi}{dx^2} \right|_i = \frac{\varphi_{i+1} - 2\varphi_i + \varphi_{i-1}}{h^2} + O(h^2) \quad (6.4)$$

La stessa espressione può essere ottenuta partendo dalla definizione della derivata seconda, approssimando la derivata prima nei punti intermedi e sostituendo le differenze finite:

$$\begin{aligned} \left. \frac{d^2\varphi}{dx^2} \right|_i &= \frac{d}{dx} \left(\left. \frac{d\varphi}{dx} \right|_i \right) = \lim_{h \rightarrow 0} \frac{\left. \frac{d\varphi}{dx} \right|_{i+1/2} - \left. \frac{d\varphi}{dx} \right|_{i-1/2}}{h} = \\ &\approx \frac{\frac{\varphi_{i+1} - \varphi_i}{h} - \frac{\varphi_i - \varphi_{i-1}}{h}}{h} = \frac{\varphi_{i+1} - 2\varphi_i + \varphi_{i-1}}{h^2} \end{aligned}$$

Capitolo 7

Algoritmo di Thomas

L'algoritmo di Thomas è un metodo numerico molto efficiente per la risoluzione di sistemi lineari la cui matrice dei coefficienti è **tridiagonale**. Questo tipo di sistemi compare frequentemente nella discretizzazione di equazioni differenziali mediante metodi alle differenze finite.

Consideriamo dunque il seguente sistema di equazioni lineari:

$$\begin{aligned}v_1\phi_1 + w_1\phi_2 &= S_1 \\u_2\phi_1 + v_2\phi_2 + w_2\phi_3 &= S_2 \\ \dots \quad u_3\phi_2 + v_3\phi_3 + w_3\phi_4 &= S_3 \\ \dots &\end{aligned}$$

dove u_i rappresentano i coefficienti sotto la diagonale principale, v_i rappresentano i coefficienti sulla diagonale principale, w_i rappresentano i coefficienti sopra la diagonale principale, S_i rappresentano i termini noti del sistema.

Eliminazione della prima variabile

Il primo passo dell'algoritmo consiste nell'eliminare progressivamente le variabili procedendo lungo il sistema.

Si sottrae la seconda equazione, moltiplicata per v_1 , dalla prima moltiplicata per u_2 .

Successivamente si procede a normalizzare le equazioni dividendo la prima equazione per v_1 e la seconda per $(v_2v_1 - u_2w_1)$.

In questo modo si ottiene un sistema equivalente nel quale i coefficienti delle variabili ϕ_1 e ϕ_2 risultano normalizzati a uno:

$$\begin{aligned}\phi_1 + \frac{w_1}{v_1}\phi_2 &= \frac{S_1}{v_1} \\ \phi_2 + \frac{w_2v_1}{(v_2v_1 - u_2w_1)}\phi_3 &= \frac{S_2v_1 - S_1u_2}{(v_2v_1 - u_2w_1)} \\ u_3\phi_2 + v_3\phi_3 + w_3\phi_4 &= S_3\end{aligned}$$

Riscrittura del sistema

Le espressioni precedenti possono essere riscritte in una forma più compatta osservando che $\frac{w_2 v_1}{v_2 v_1 - u_2 w_1} = \frac{w_2}{v_2 - u_2 \frac{w_1}{v_1}}$ e analogamente per il termine noto $\frac{S_2 - S_1 u_2 / v_1}{(v_2 - u_2 w_1 / v_1)}$.

Il sistema diventa quindi:

$$\begin{aligned}\phi_1 + \frac{w_1}{v_1} \phi_2 &= \frac{S_1}{v_1} \\ \phi_2 + \frac{w_2}{\left(v_2 - u_2 \frac{w_1}{v_1}\right)} \phi_3 &= \frac{S_2 - S_1 \frac{u_2}{v_1}}{\left(v_2 - u_2 \frac{w_1}{v_1}\right)} \\ u_3 \phi_2 + v_3 \phi_3 + w_3 \phi_4 &= S_3\end{aligned}$$

Introduzione dei coefficienti ricorsivi

A questo punto è conveniente introdurre nuovi coefficienti che permettono di riscrivere il sistema in forma ricorsiva.

Definiamo:

$$\alpha_1 = \frac{w_1}{v_1} \quad , \quad \beta_1 = \frac{S_1}{v_1}$$

e successivamente:

$$\alpha_2 = \frac{w_2}{\left(v_2 - u_2 \alpha_1\right)} \quad , \quad \beta_2 = \frac{S_2 - u_2 \beta_1}{\left(v_2 - u_2 \alpha_1\right)}$$

Utilizzando queste definizioni il sistema assume la forma

$$\begin{aligned}\phi_1 + \alpha_1 \phi_2 &= \beta_1 \\ \phi_2 + \alpha_2 \phi_3 &= \beta_2 \\ u_3 \phi_2 + v_3 \phi_3 + w_3 \phi_4 &= S_3 \\ &\dots\end{aligned}$$

Passo ricorsivo

Il procedimento può essere ripetuto anche per le equazioni successive. Si sottrae dall'ultima equazione scritta la precedente moltiplicata per u_3 , poi si divide il risultato per il coefficiente di ϕ_3 . Si ottiene:

$$\phi_3 + \frac{w_3}{v_3 - u_3 \alpha_2} \phi_4 = \frac{S_3 - u_3 \beta_2}{v_3 - u_3 \alpha_2}$$

Confrontando questa espressione con le definizioni precedenti si osserva che è possibile introdurre una definizione ricorsiva generale dei coefficienti.

Per $i = 2, \dots, N$:

$$\alpha_i = \frac{w_i}{(v_i - u_i \alpha_{i-1})} \quad (16)$$

$$\beta_i = \frac{S_i - u_i \beta_{i-1}}{(v_i - u_i \alpha_{i-1})} \quad (17)$$

Queste relazioni permettono di determinare progressivamente tutti i coefficienti α_i e β_i .

Forma finale del sistema

Dopo aver applicato questo procedimento a tutte le equazioni del sistema, assume la forma

$$\begin{aligned} \phi_1 &= \beta_1 - \alpha_1 \phi_2 \\ \phi_2 &= \beta_2 - \alpha_2 \phi_3 \\ &\dots \\ \phi_{N-1} &= \beta_{N-1} - \alpha_{N-1} \phi_N \\ \phi_N &= \beta_N \end{aligned}$$

L'ultima equazione fornisce immediatamente il valore di ϕ_N .

Una volta noto ϕ_N è possibile determinare tutte le altre incognite tramite una procedura di sostituzione all'indietro: $\phi_{N-1}, \phi_{N-2}, \dots, \phi_1$.

Costo computazionale

Il numero di operazioni in virgola mobile (floating point operations, FLOP) richiesto dall'algoritmo di Thomas per risolvere un sistema lineare di N equazioni è $\boxed{10N - 11}$.

Questo significa che il costo computazionale cresce linearmente con la dimensione del sistema.

Prestazioni

Consideriamo ad esempio un processore in grado di eseguire 600 GFLOPS cioè 600×10^9 .

operazioni floating point al secondo (come ad esempio il processore AMD Ryzen Threadripper 2990WX).

In queste condizioni il tempo necessario per risolvere un sistema lineare con $N = 1000$ equazioni è dell'ordine di 17 ns.

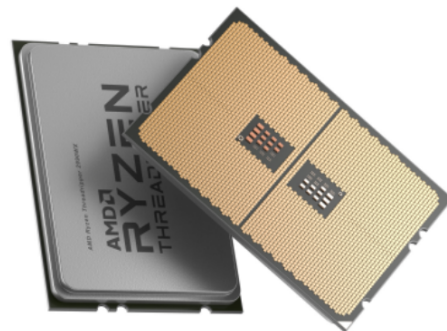


FIG. 7.1: catzo e palle

Capitolo 8

Metodo di Jacobi

8.1 Introduzione

Consideriamo il sistema lineare

$$\mathbf{A}\boldsymbol{\phi} = \mathbf{s} \quad (8.1)$$

dove $A \in \mathbb{R}^{n \times n}$ è una matrice assegnata, $\boldsymbol{\phi}$ il vettore incognito e $\mathbf{s}$ il termine noto.

Il metodo di Jacobi si basa sulla decomposizione della matrice A nella forma

$$\mathbf{A} = \mathbf{D} - \mathbf{E}$$

dove: D è la matrice diagonale contenente gli elementi diagonali di A mentre E contiene l'opposto degli elementi fuori diagonale di A .

Sostituendo nella relazione iniziale si ottiene:

$$D\boldsymbol{\phi} = E\boldsymbol{\phi} + \mathbf{s}$$

Assumendo che D sia invertibile (cioè $a_{ii} \neq 0$ per ogni i), si può scrivere

$$\boldsymbol{\phi} = \mathbf{D}^{-1}\mathbf{E}\boldsymbol{\phi} + \mathbf{D}^{-1}\mathbf{s}$$

Introducendo le definizioni $\mathbf{H} = \mathbf{D}^{-1}\mathbf{E}$ e $\mathbf{q} = \mathbf{D}^{-1}\mathbf{s}$ il sistema assume la forma

$$\boldsymbol{\phi} = \mathbf{H}\boldsymbol{\phi} + \mathbf{q}$$

8.1.1 Struttura della matrice di iterazione

Dalla struttura di D ed E si ottiene che la matrice H ha forma

$$H = \begin{pmatrix} 0 & -\frac{a_{12}}{a_{11}} & -\frac{a_{13}}{a_{11}} & \dots & -\frac{a_{1n}}{a_{11}} \\ -\frac{a_{21}}{a_{22}} & 0 & -\frac{a_{23}}{a_{22}} & \dots & -\frac{a_{2n}}{a_{22}} \\ \vdots & \vdots & \vdots & & \vdots \\ -\frac{a_{n1}}{a_{nn}} & -\frac{a_{n2}}{a_{nn}} & -\frac{a_{n3}}{a_{nn}} & \dots & 0 \end{pmatrix} \quad (8.2)$$

Gli elementi diagonali sono nulli.

Se la matrice A è diagonalmente dominante, vale

$$\sum_{j=1}^n |h_{ij}| < 1 \quad \forall i \quad \rightarrow \quad \|H\|_{\infty} < 1 \quad \rightarrow \quad \boxed{\rho(H) < 1} \quad .$$

Quindi la matrice H è convergente.

8.1.2 Procedimento iterativo

Si introduce il processo iterativo:

$$\phi^{(m+1)} = H\phi^{(m)} + \mathbf{q} \quad ,$$

con condizione iniziale

$$\phi^{(1)} = \mathbf{q} \quad .$$

Esplicitando le iterazioni:

$$\begin{aligned} \phi^{(2)} &= H\phi^{(1)} + \mathbf{q} = H\mathbf{q} + \mathbf{q} \\ \phi^{(3)} &= H^2\mathbf{q} + H\mathbf{q} + \mathbf{q} \\ &\vdots \\ \phi^{(m+1)} &= H^m\mathbf{q} + H^{m-1}\mathbf{q} + \dots + \mathbf{q} \end{aligned}$$

8.1.3 Convergenza del metodo

Moltiplicando per $(I - H)$ si ottiene

$$(I - H)\phi^{(m+1)} = \dots = (I - H^{m+1})\mathbf{q}$$

e quindi invertendo $(I - H)$:

$$\phi^{(m+1)} = (I - H)^{-1}(I - H^{m+1})\mathbf{q}$$

Passando al limite

$$\lim_{m \rightarrow \infty} \phi^{(m+1)} = \lim_{m \rightarrow \infty} (\mathbf{I} - \mathbf{H})^{-1} (\mathbf{I} - \mathbf{H}^{m+1}) \mathbf{q} \stackrel{\rightarrow 0}{=} (\mathbf{I} - \mathbf{H})^{-1} \mathbf{q}$$

D'altra parte, la soluzione esatta soddisfa

$$\phi = H\phi + \mathbf{q}$$

da cui

$$(\mathbf{I} - \mathbf{H})\phi = \mathbf{q} \quad \Rightarrow \quad \phi = (\mathbf{I} - \mathbf{H})^{-1} \mathbf{q}$$

Quindi

$$\lim_{m \rightarrow \infty} \phi^{(m+1)} = (\mathbf{I} - \mathbf{H})^{-1} \mathbf{q} = \phi \quad (\text{soluzione esatta}).$$

Secondo quanto appena trovato il metodo di Jacobi è convergente per matrici diagonalmente dominanti, tuttavia, abbiamo dato per scontato la convergenza del processo iterativo quando il raggio $\rho(H)$ è < 1 . Andiamo a verificarlo nel caso semplice di una matrice d'iterazione $\mathbf{H}$ simmetrica.

Nonostante la matrice del sistema, $\mathbf{A}$, sia simmetrica, lo stesso non vale per la matrice d'iterazione di Jacobi $\mathbf{H}$, a meno di casi particolari.

Sia $\mathbf{H}_{n \times n}$ reale e simmetrica. Allora esiste una matrice ortogonale $\mathbf{U}$ tale che

$$\mathbf{U}^T \mathbf{H} \mathbf{U} = \Lambda.$$

dove $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$. Segue che: $\mathbf{H} \mathbf{U} = \mathbf{U} \Lambda$, quindi

$$H \mathbf{u}_j = \lambda_j \mathbf{u}_j \quad (j = 1, \dots, n)$$

Con $\mathbf{u}_j$ corrispondenti alle j -esime colonne di $\mathbf{U}$ mentre λ_j sono gli elementi diagonali della j -esima riga di Λ .

Dall'ultima relazione si ricava che le colonne della matrice $\mathbf{U}$ sono gli autovettori di $\mathbf{H}$, con elementi diagonali di Λ che ne sono i rispettivi autovalori. Inoltre, l'insieme degli autovettori è una base ortonormale di $\mathbb{R}_n$.

Da qui in avanti, quindi, chiamiamo le colonne di $\mathbf{U}$ come una base: $\varphi_1, \varphi_2, \dots, \varphi_n$.

8.2 Condizione per la convergenza

Teorema sulla convergenza:

Condizione necessaria e sufficiente affinché la matrice $\mathbf{H}$ converga è che:

$$\rho(\mathbf{H}) < 1.$$

Dimostrazione:

Per dimostrare questo teorema si parte dall'equazione di iterazione:

$$\phi^{(m+1)} = \mathbf{H}\phi^{(m)} + \mathbf{q} ,$$

definiamo anche il vettore errore come

$$\epsilon^{(m)} = \phi^{(m)} - \phi .$$

Dove ϕ è la soluzione esatta, mentre $\phi^{(m)}$ è un'approssimazione all' m -esima iterazione.

La definizione di errore può essere riscritta nella forma:

$$\epsilon^{(m)} = \mathbf{H}^m \epsilon^{(m-1)} \rightarrow \epsilon^{(m)} = \mathbf{H}^m \epsilon^{(0)}$$

Dove $\epsilon^{(0)}$ è l'errore iniziale.

Sviluppando l'errore sull'autobase di $\mathbf{H}$:

$$\begin{aligned} \epsilon^{(0)} &= \sum_{h=1}^n c_h^{(0)} \phi_h \\ \epsilon^{(m)} &= \mathbf{H}^m \sum_{h=1}^n c_h^{(0)} \phi_h = \sum_{h=1}^n c_h^{(0)} \mathbf{H}^m \phi_h = \sum_{h=1}^n c_h^{(0)} \lambda_h^m \phi_h \end{aligned}$$

La convergenza avviene se e solo se

$$\lim_{m \rightarrow \infty} \|\epsilon^{(m)}\| = \lim_{m \rightarrow \infty} \sum_{h=1}^n c_h^{(0)} |\lambda_h^m| \|\phi_h\| = 0 \rightarrow |\lambda_h| < 1 \quad \forall h$$

cioè: $\boxed{\rho(\mathbf{H}) < 1}$.

□

La proprietà $\rho(\mathbf{H}) < 1$ ci assicura che l'operatore matriciale $\mathbf{H}$ contragga, nel senso della norma, ogni vettore al quale venga applicato.

Ancora sulle condizioni di convergenza

Una condizione sufficiente per la convergenza è che la matrice A sia *strettamente diagonalmente dominante* ($>$) oppure *irriducibilmente diagonalmente dominante* ($\geq$):

$$|a_{ii}| \geq \sum_{j=1, j \neq i}^n |a_{ij}|$$

Nel secondo caso è necessario che per almeno una riga vi sia dominanza diagonale stretta:

$$\exists i : |a_{ii}| > \sum_{j \neq i} |a_{ij}|.$$

Definizione di irriducibilità: Una matrice è detta irriducibile se non è riducibile. Una matrice $\mathbf{A}_{n \times n}$ è riducibile quando esiste una matrice di permutazione P tale che la matrice $P\mathbf{A}P^T$ è triangolare a blocchi

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots \\ 0 & a_{22} & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix} \rightarrow P\mathbf{A}P^T = \begin{pmatrix} \hat{A}_{11} & \mathbf{A}_{12} & \cdots \\ 0 & \hat{A}_{22} & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}$$

Teorema [irriducibilità e grafi]: Una matrice $\mathbf{A}_{n \times n} \in \mathbb{C}$ è irriducibile se e solo se il grafo orientato $G(\mathbf{A})$ associato è fortemente connesso.

Il grafo orientato (*directed graph*) è l'insieme dei cammini orientati ("*directed paths*") che connettono due nodi, si dicano i e j , per i quali la matrice $\mathbf{A}$ ha un coefficiente a_{ij} non nullo.

Un grafo orientato è fortemente connesso se per ogni scelta di due indici esiste una sequenza di collegamenti (coefficienti non nulli) tale che i e j siano connessi da $\{a_{i,\ell_1}, a_{\ell_1,\ell_2}, \dots, a_{\ell_r,j}\}$, c.d. cammino orientato.

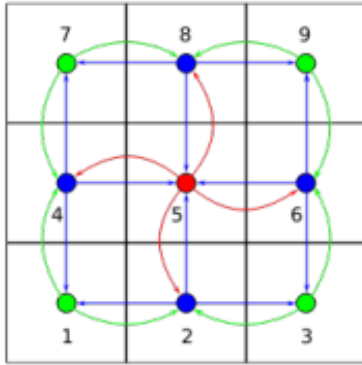


FIG. 8.1: Esempio visivo di grafo connesso

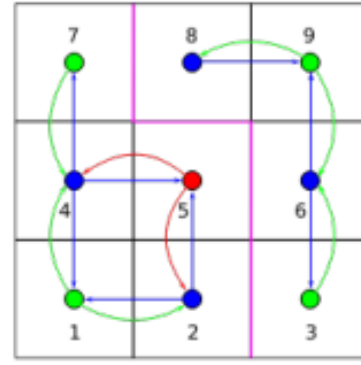


FIG. 8.2: Esempio di grafo non connesso

Considerazioni fisiche sul grafo orientato:

- secondo la nostra discretizzazione dell'equazione di diffusione dei neutroni dobbiamo considerare che un coefficiente non nullo a_{ij} è equivalente a dire che i due nodi i e j si influenzano mutualmente;
- L'irriducibilità è assicurata dal *neutron leakage* fra i nodi, solo in correnti nulle potremmo avere una matrice riducibile.

Immaginiamo che nel nostro dominio vi sia un'*absorbing curtain* (come quella in viola in fig.(??)). Il sistema in matrice $\mathbf{A}$ allora è riducibile perché non è possibile tracciare un percorso orientato presa ciascuna coppia di nodi del sistema.

Mostiamo in che forma appare la matrice riducibile dell'esempio in fig.(??):

$$\mathbf{A}\phi = \mathbf{b} \Rightarrow \begin{bmatrix} a_{11} & a_{12} & 0 & a_{14} & 0 & 0 & 0 & 0 & 0 \\ a_{21} & a_{22} & 0 & 0 & a_{25} & 0 & 0 & 0 & 0 \\ 0 & 0 & a_{33} & 0 & 0 & a_{36} & 0 & 0 & 0 \\ a_{41} & 0 & 0 & a_{44} & a_{45} & 0 & a_{47} & 0 & 0 \\ 0 & a_{52} & 0 & a_{54} & a_{55} & 0 & 0 & 0 & 0 \\ 0 & 0 & a_{63} & 0 & 0 & a_{66} & 0 & 0 & a_{69} \\ 0 & 0 & 0 & a_{74} & 0 & 0 & a_{77} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & a_{88} & a_{89} \\ 0 & 0 & 0 & 0 & 0 & a_{96} & 0 & a_{98} & a_{99} \end{bmatrix} \begin{bmatrix} \phi_1 \\ \phi_2 \\ \phi_3 \\ \phi_4 \\ \phi_5 \\ \phi_6 \\ \phi_7 \\ \phi_8 \\ \phi_9 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \\ b_8 \\ b_9 \end{bmatrix}$$

Esiste una permutazione $\mathbf{P}$ che ci permette di riarrangiare le righe in modo che la matrice diventi diagonale a blocchi (caso speciale di matrice triangolare a blocchi), questo significa che si riconoscono due flussi di neutroni ben distinti che non si influenzano fra loro:

$$\mathbf{P} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix},$$

$$\mathbf{A}\phi = \mathbf{b} \rightarrow \mathbf{P}\mathbf{A}(\mathbf{P}^T\mathbf{P})\phi = \mathbf{P}\mathbf{b} \rightarrow \mathbf{P}\mathbf{A}\mathbf{P}^T\mathbf{P}\psi = \mathbf{P}\mathbf{b}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{14} & 0 & 0 & 0 & 0 & 0 & 0 \\ a_{21} & a_{22} & 0 & a_{25} & 0 & 0 & 0 & 0 & 0 \\ a_{41} & 0 & a_{44} & a_{45} & a_{47} & 0 & 0 & 0 & 0 \\ 0 & a_{52} & a_{54} & a_{55} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & a_{74} & 0 & a_{77} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & a_{33} & a_{36} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & a_{63} & a_{66} & 0 & a_{69} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & a_{88} & a_{89} \\ 0 & 0 & 0 & 0 & 0 & 0 & a_{96} & a_{98} & a_{99} \end{bmatrix} \begin{bmatrix} \phi_1 \\ \phi_2 \\ \phi_4 \\ \phi_5 \\ \phi_7 \\ \phi_3 \\ \phi_6 \\ \phi_8 \\ \phi_9 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_4 \\ b_5 \\ b_7 \\ b_3 \\ b_6 \\ b_8 \\ b_9 \end{bmatrix}$$

dove sulla diagonale vediamo che vi sono blocchi quadrati. Il sistema è quindi riducibile ad un primo sistema di 5 equazioni con soluzione $\phi_I = [\phi_1, \phi_2, \phi_4, \phi_5, \phi_7]^T$ ed un secondo sistema di 4 equazioni con soluzione $\phi_{II} = [\phi_3, \phi_6, \phi_8, \phi_9]^T$. Possiamo definirli flussi disaccoppiati (*decoupled fluxes*).

8.3 Flusso illimitato

Le considerazioni fisiche sull'irriducibilità della matrice garantiscono che:

- La dominanza diagonale stretta della matrice $\mathbf{A}$ del sistema è assicurata da $\Sigma_a > 0$ in ogni nodo. Se in qualche nodo $\Sigma_a = 0$ e non ci sono perdite verso i nodi adiacenti, il flusso neutronico diventerebbe illimitato, a meno che non ci siano interazioni con altri nodi.
- Lirriducibilità della matrice assicura che il flusso resti limitato, grazie alle influenze mutue tra tutti i nodi.

Un reattore nucleare è accoppiato debolmente quando il cammino libero medio dei neutroni è relativamente piccolo se comparata alla sua dimensione. In questo caso, nonostante ci sarà sempre un'interazione fra qualsiasi coppia di regioni scelte del reattore, questa interazione può spesso essere abbastanza piccola da permetterci di considerare come se i nodi fossero indipendenti fra loro.

8.4 Come funziona il metodo di Jacobi

Considerando la decomposizione di Jacobi della matrice $\mathbf{A}$ del sistema, l'idea alla base del metodo è la seguente:

$$\begin{cases} a_{11}\phi_1 + a_{12}\phi_2 + a_{13}\phi_3 + \cdots + a_{1n}\phi_n = b_1 \\ a_{21}\phi_1 + a_{22}\phi_2 + a_{23}\phi_3 + \cdots + a_{2n}\phi_n = b_2 \\ \vdots \\ a_{n1}\phi_1 + a_{n2}\phi_2 + a_{n3}\phi_3 + \cdots + a_{nn}\phi_n = b_n \end{cases}$$

Si adotta la seguente procedura iterativa:

$$\begin{cases} \phi_1^{(m+1)} = \left(b_1 - a_{12}\phi_2^{(m)} - a_{13}\phi_3^{(m)} - \cdots - a_{1n}\phi_n^{(m)} \right) / a_{11} \\ \phi_2^{(m+1)} = \left(b_2 - a_{21}\phi_1^{(m)} - a_{23}\phi_3^{(m)} - \cdots - a_{2n}\phi_n^{(m)} \right) / a_{22} \\ \vdots \\ \phi_n^{(m+1)} = \left(b_n - a_{n1}\phi_1^{(m)} - a_{n2}\phi_2^{(m)} - \cdots - a_{n,n-1}\phi_{n-1}^{(m)} \right) / a_{nn} \end{cases}$$

Un processo così brutale converge alla soluzione esatta del problema poiché la matrice di iterazione $\mathbf{H}$ di Jacobi, avendo raggio spettrale minore di 1, contrae sempre l'errore iniziale introdotto da qualsiasi stima iniziale scelta.

Matrice diagonalmente dominante (norma dell'errore in blu in figura):

$$\begin{bmatrix} 6 & 2 & 3 \\ 2 & 4 & -1 \\ 1 & -2 & 5 \end{bmatrix} \begin{bmatrix} \phi_1 \\ \phi_2 \\ \phi_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$$

La prima riga non rispetta la condizione di matrice diagonalmente dominante (norma dell'errore in rosso):

$$\begin{bmatrix} 5 & 2 & 3 \\ 2 & 4 & -1 \\ 1 & -2 & 5 \end{bmatrix} \begin{bmatrix} \phi_1 \\ \phi_2 \\ \phi_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$$

La prima e la seconda riga non rispettano la condizione di matrice diagonalmente dominante (norma dell'errore in viola):

$$\begin{bmatrix} 5 & 2 & 3 \\ 2 & 3 & -1 \\ 1 & -2 & 5 \end{bmatrix} \begin{bmatrix} \phi_1 \\ \phi_2 \\ \phi_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$$

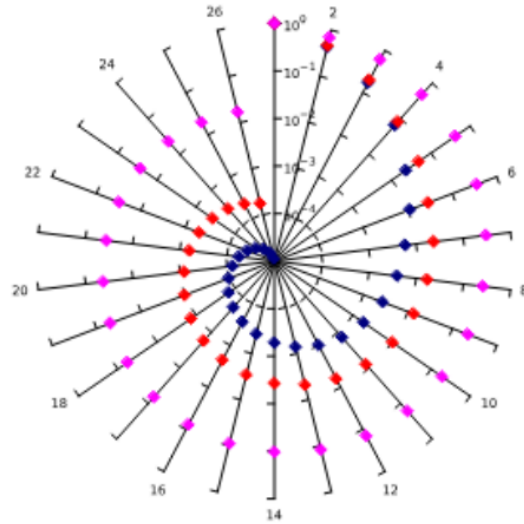


FIG. 8.3: error norm vs. iteration

8.5 Implementazione per equazione di diffusione

Nel caso bidimensionale rappresentato in figura che presenta sia indicizzazione singola che doppia:

$$-\frac{D}{h_x^2}(\phi_{i+1,j} - 2\phi_{i,j} + \phi_{i-1,j}) - \frac{D}{h_y^2}(\phi_{i,j+1} - 2\phi_{i,j} + \phi_{i,j-1}) + \Sigma_a \phi_{i,j} = Q_{i,j}$$

Definendo i coefficienti di seguito:

$$W_{i,j} = E_{i,j} = \frac{D/h_x^2}{\Sigma_a + 2D/h_x^2 + 2D/h_y^2}$$

$$S_{i,j} = N_{i,j} = \frac{D/h_y^2}{\Sigma_a + 2D/h_x^2 + 2D/h_y^2}$$

$$q_{i,j} = \frac{Q_{i,j}}{\Sigma_a + 2D/h_x^2 + 2D/h_y^2}$$

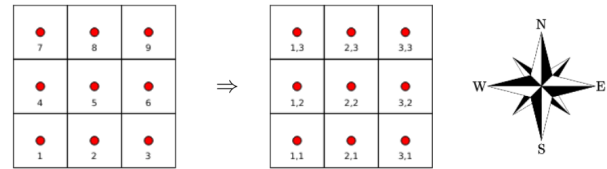


FIG. 8.4: indicizzazione

lo schema iterativo diventa:

$$\phi_{i,j}^{(m+1)} = W_{i,j} \phi_{i-1,j}^{(m)} + E_{i,j} \phi_{i+1,j}^{(m)} + S_{i,j} \phi_{i,j-1}^{(m)} + N_{i,j} \phi_{i,j+1}^{(m)} + q_{i,j}$$

Nel caso tridimensionale compaiono due coefficienti in più, $D_{i,j}$ e $U_{i,j}$, rispettivamente per i flussi $\phi_{i,j,k-1}^{(m)}$ e $\phi_{i,j,k+1}^{(m)}$.

8.6 Velocità di convergenza

Se la matrice $\mathbf{H}_{n \times n}$ ha un singolo autovalore di modulo al più λ_1 ed l'insieme di autovettori φ forma una base di $\mathbb{R}_n$, allora:

$$\epsilon^{(m)} = \mathbf{H}^m \epsilon^{(0)} = \mathbf{H}^m \sum_{h=1}^n c_h^{(0)} \varphi_h = \sum_{h=1}^n c_h^{(0)} \lambda_h^m \varphi_h ,$$

$$\epsilon^{(m)} = \lambda_1^m \left(c_1^{(0)} \varphi_1 + \sum_{h=2}^n c_h^{(0)} \frac{\lambda_h^m}{\lambda_1^m} \varphi_h \right) .$$

La somma svenisce mandando $m \rightarrow \infty$ quindi l'errore si comporta come:

$$\|\epsilon^{(m)}\| \underset{m \rightarrow \infty}{=} |\lambda_1^m| \|c_1^{(0)} \varphi_1\| = C \rho(\mathbf{H})^m$$

Se la matrice $\mathbf{H}_{n \times n}$ avesse invece più di un autovalore di modulo massimo, $\rho(\mathbf{H})$, è ancora possibile raggiungere lo stesso risultato di cui sopra. Infatti, assunto che gli autovettori siano normalizzati, segue:

$$\begin{aligned} \|\epsilon^{(m)}\|^2 &= (\epsilon^{(m)}, \epsilon^{(m)}) = \left(\sum_{h=1}^n c_h^{(0)} \lambda_h^m \varphi_h, \sum_{h=1}^n c_h^{(0)} \lambda_h^m \varphi_h \right) = \\ &= \sum_{h=1}^n |c_h^{(0)} \lambda_h^m|^2 = \rho(\mathbf{H})^{2m} \left(\sum_{h: |\lambda_h| = \rho(\mathbf{H})} |c_h^{(0)}|^2 + \sum_{h: |\lambda_h| < \rho(\mathbf{H})} |c_h^{(0)}|^2 \frac{|\lambda_h^m|^2}{\rho(\mathbf{H})^{2m}} \right) \end{aligned}$$

Che porta nuovamente a $\|\epsilon^{(m)}\| \underset{m \rightarrow \infty}{=} C \rho(\mathbf{H})^m$. Quindi vale che:

$$\lim_{m \rightarrow \infty} \frac{\|\epsilon^{(m+1)}\|}{\|\epsilon^{(m)}\|} = \rho(\mathbf{H})$$

Che non vuol dire altro che il raggio spettrale corrisponde al fattore di decrescita dell'errore ad ogni iterazione (per questo potremmo anche chiamarlo fattore asintotico di convergenza).

Il tasso di convergenza medio dopo m iterazioni è

$$R_m(\mathbf{H}) = -\frac{1}{m} \ln \|\mathbf{H}^m\|$$

Si può mostrare che $\lim_{m \rightarrow \infty} \|\mathbf{H}^m\|^{\frac{1}{m}} = \rho(\mathbf{H})$ per cui dal tasso di convergenza medio possiamo dare una definizione di tasso asintotico di convergenza:

$$R_\infty(\mathbf{H}) = -\ln \rho(\mathbf{H}) .$$

Numero di iterazioni

Ci chiediamo quale sia il numero di iterazioni necessario per ridurre l'errore di un fattore σ :

$$\rho(\mathbf{H})^n < \sigma$$

$$n > \frac{\ln \sigma}{\ln \rho(\mathbf{H})} = -\frac{\ln \sigma}{R_\infty(\mathbf{H})}$$

Osservazioni finali

Per avere una convergenza veloce è necessario che

$$\rho(\mathbf{H}) \ll 1$$

Si osserva che:

- se $\Sigma_a \rightarrow 0$, la dominanza diagonale diminuisce e $\rho(\mathbf{H}) \rightarrow 1$;
- raffinando la griglia ($h_x, h_y \rightarrow 0$), $\rho(H) \rightarrow 1$;

quindi una maggiore risoluzione spaziale comporta una convergenza più lenta.

Capitolo 9

Gauss-Siedel, SOR & LOR

9.1 Metodo di Gauss-Seidel

Il metodo di Jacobi calcola il flusso in ogni nodo all'iterazione $m + 1$ sulla base dei flussi ottenuti all'iterazione m . Tuttavia, quando si aggiorna il flusso nel nodo di coordinate (i, j) , i nodi con indici i e j minori sono già stati aggiornati.

Il metodo di Gauss-Seidel cerca di sfruttare immediatamente i flussi già aggiornati nell'iterazione corrente. Per fare ciò, si immagina di decomporre ulteriormente la matrice di iterazione di Jacobi $\mathbf{H}$ come segue:

$$\mathbf{H} = \mathbf{L} + \mathbf{U} \quad (1)$$

dove

$$\mathbf{L} = \begin{bmatrix} 0 & 0 & 0 & \circ & 0 \\ -\frac{a_{21}}{a_{22}} & 0 & 0 & \circ & 0 \\ -\frac{a_{31}}{a_{33}} & -\frac{a_{32}}{a_{33}} & 0 & \circ & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -\frac{a_{n1}}{a_{nn}} & -\frac{a_{n2}}{a_{nn}} & -\frac{a_{n3}}{a_{nn}} & \circ & 0 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} 0 & -\frac{a_{12}}{a_{11}} & -\frac{a_{13}}{a_{11}} & \circ & -\frac{a_{1n}}{a_{11}} \\ 0 & 0 & -\frac{a_{23}}{a_{22}} & \circ & -\frac{a_{2n}}{a_{22}} \\ 0 & 0 & 0 & \circ & -\frac{a_{3n}}{a_{33}} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \circ & 0 \end{bmatrix}.$$

L'equazione di iterazione assume la nuova forma:

$$\phi = \mathbf{H}\phi + \mathbf{q} \quad \Rightarrow \quad \phi = \mathbf{L}\phi + \mathbf{U}\phi + \mathbf{q} \quad (2)$$

che, scritta in termini di componenti,

$$\phi_i = \sum_{j=1}^{i-1} l_{i,j} \phi_j + \sum_{j=i+1}^n u_{i,j} \phi_j + q_i \quad (3)$$

permette di impostare il seguente schema iterativo:

$$\phi^{(m+1)} = \mathbf{L}\phi^{(m+1)} + \mathbf{U}\phi^{(m)} + \mathbf{q}. \quad (4)$$

È un dato di fatto che i flussi nella prima somma dell'Eq.(3) appartengono a nodi che sono già stati aggiornati nel corso dell'iterazione $(m + 1)$.

Partendo dall'Eq.(4), si ottiene:

$$\begin{aligned}\phi^{(m+1)} &= \mathbf{L}\phi^{(m+1)} + \mathbf{U}\phi^{(m)} + \mathbf{q} , \\ (\mathbf{I} - \mathbf{L})\phi^{(m+1)} &= \mathbf{U}\phi^{(m)} + \mathbf{q} , \\ \phi^{(m+1)} &= (\mathbf{I} - \mathbf{L})^{-1}\mathbf{U}\phi^{(m)} + (\mathbf{I} - \mathbf{L})^{-1}\mathbf{q} .\end{aligned}$$

La matrice di iterazione di Gauss-Seidel è quindi: $\boxed{\mathbf{H}_{GS} = (\mathbf{I} - \mathbf{L})^{-1}\mathbf{U}}$.

È possibile dimostrare che il raggio spettrale di $\mathbf{H}_{GS}$ è legato a quello della matrice di Jacobi $\mathbf{H}_J$ dalla relazione:

$$\boxed{\rho(\mathbf{H}_{GS}) = [\rho(\mathbf{H}_J)]^2} .$$

Tasso di convergenza

Nel caso del metodo di Gauss-Seidel, il tasso asintotico di convergenza è data da:

$$R_{\infty}^{GS} = -\ln \rho(\mathbf{H}_{GS}) = -2 \ln \rho(\mathbf{H}_J) = 2 R_{\infty}^J,$$

dove R_{∞}^J è il tasso asintotico di convergenza del metodo di Jacobi.

Di conseguenza, il metodo di Gauss-Seidel converge più rapidamente del metodo di Jacobi, richiedendo circa la metà delle iterazioni.

Implementazione pratica del metodo di Gauss-Seidel

Il calcolo del flusso in ogni nodo coinvolge sia i valori dell'iterazione precedente che quelli dell'iterazione corrente. Ad esempio come si vede in figura di fianco, quando si aggiorna il flusso del nodo 10, per i nodi 4 e 9 si utilizzano i valori già aggiornati nell'iterazione corrente, mentre per i nodi 11 e 16 si adottano quelli dell'iterazione precedente.

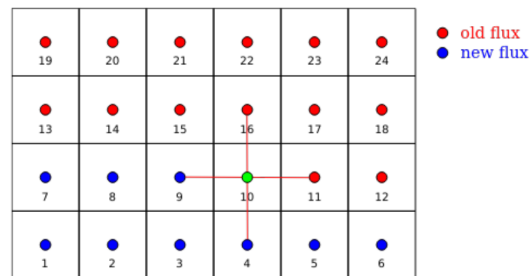


FIG. 9.1: visualizzazione di Gauss-Siedel

Introducendo come per il metodo di Jacobi i coefficienti $W_{i,j}$, $E_{i,j}$, $S_{i,j}$, $N_{i,j}$ e il termine sorgente $q_{i,j}$, lo schema iterativo può essere espresso come:

$$\phi_{i,j}^{(m+1)} = W_{i,j} \phi_{i-1,j}^{(m+1)} + E_{i,j} \phi_{i+1,j}^{(m)} + S_{i,j} \phi_{i,j-1}^{(m+1)} + N_{i,j} \phi_{i,j+1}^{(m)} + q_{i,j} .$$

9.2 Metodo di Successive Overrelaxation (SOR)

Il concetto di questo metodo è quello di estrapolare la previsione di Gauss-Seidel utilizzando un opportuno fattore di overrelaxation $\omega > 1$. Partendo dalla stima di Gauss-Seidel all'iterazione $(m + 1)$, Eq.(4), si sottrae $\phi^{(m)}$ da entrambi i membri:

$$\phi^{(m+1)} - \phi^{(m)} = \mathbf{L}\phi^{(m+1)} + (\mathbf{U} - \mathbf{I})\phi^{(m)} + \mathbf{q} \quad .$$

Il flusso alla nuova iterazione $(m + 1)$ viene quindi calcolato come:

$$\phi^{(m+1)} = \phi^{(m)} + \omega [\phi^{(m+1)} - \phi^{(m)}]_{GS} \quad ,$$

$$\phi^{(m+1)} = \phi^{(m)} + \omega [\mathbf{L}\phi^{(m+1)} + (\mathbf{U} - \mathbf{I})\phi^{(m)} + \mathbf{q}] \quad .$$

$$\phi^{(m+1)} = \omega \mathbf{L}\phi^{(m+1)} + [(1 - \omega)\mathbf{I} + \omega \mathbf{U}] \phi^{(m)} + \omega \mathbf{q} \quad .$$

9.2.1 Matrice di iterazione del metodo SOR

Dalle equazioni precedenti è possibile ricavare la matrice di iterazione dello schema iterativo SOR:

$$\phi^{(m+1)} = (\mathbf{I} - \omega \mathbf{L})^{-1} [(1 - \omega)\mathbf{I} + \omega \mathbf{U}] \phi^{(m)} + \omega (\mathbf{I} - \omega \mathbf{L})^{-1} \mathbf{q} \quad .$$

La matrice di iterazione $\mathbf{H}$ del metodo SOR è quindi:

$$\mathbf{H} = (\mathbf{I} - \omega \mathbf{L})^{-1} [(1 - \omega)\mathbf{I} + \omega \mathbf{U}] \quad .$$

È possibile dimostrare che esiste un valore ottimale di ω che minimizza il raggio spettrale di $\mathbf{H}$, dato da:

$$\omega_{opt} = \frac{2}{1 + \sqrt{1 - \rho^2(\mathbf{H}_J)}}, \quad 1 \leq \omega_{opt} \leq 2 \quad .$$

Per calcolare ω_{opt} è necessario conoscere il raggio spettrale della matrice di iterazione di Jacobi $\mathbf{H}_J$.

Stima del raggio spettrale di Jacobi

Prima di avviare il processo iterativo SOR, è consigliabile eseguire un certo numero di iterazioni di Jacobi per ottenere una stima del suo raggio spettrale, $\rho_{\mathbf{H}_J}$.

Consideriamo una matrice simmetrica $\mathbf{H}_J$. Il vettore noto $\mathbf{q}$ può essere espresso in termini degli autovettori di $\mathbf{H}_J$ (che rappresentano una base ortogonale completa):

$$\mathbf{q} = \sum_{h=1}^n q_h \varphi_h \quad .$$

Inoltre, ricordiamo che gli autovettori sono ortonormali, poiché $\mathbf{U}^T \mathbf{U} = \mathbf{I}$.

Derivazione della stima

Si ha quindi:

$$\begin{aligned} \|\mathbf{H}_J^m \mathbf{q}\|^2 &= (\mathbf{H}_J^m \mathbf{q}, \mathbf{H}_J^m \mathbf{q}) = \left(\sum_{h=1}^n q_h \lambda_h^m \varphi_h, \sum_{k=1}^n q_k \lambda_k^m \varphi_k \right) = \\ &= \sum_{h=1}^n |q_h|^2 \lambda_h^{2m} = \sum_{\substack{h: \\ |\lambda_h| = \rho_{\mathbf{H}_J}}}^n |q_h|^2 \rho_{\mathbf{H}_J}^{2m} + \sum_{\substack{h: \\ |\lambda_h| < \rho_{\mathbf{H}_J}}}^n |q_h|^2 \lambda_h^{2m} = \\ &= \rho_{\mathbf{H}_J}^{2m} \left[\sum_{\substack{h: \\ |\lambda_h| = \rho_{\mathbf{H}_J}}}^n |q_h|^2 + \sum_{\substack{h: \\ |\lambda_h| < \rho_{\mathbf{H}_J}}}^n |q_h|^2 \frac{\lambda_h^{2m}}{\rho_{\mathbf{H}_J}^{2m}} \right] . \end{aligned}$$

Consideriamo ora il seguente limite:

$$\lim_{m \rightarrow \infty} \frac{\|\mathbf{H}_J^{m+1} \mathbf{q}\|^2}{\|\mathbf{H}_J^m \mathbf{q}\|^2} .$$

Dall'Equazione di $\|\mathbf{H}_J^m \mathbf{q}\|^2$, per $m \rightarrow \infty$, i rapporti nella sommatoria per $h : |\lambda_h| < \rho_{\mathbf{H}_J}$ tenderanno tutti a zero, così che il limite si riduce a:

$$\lim_{m \rightarrow \infty} \frac{\|\mathbf{H}_J^{m+1} \mathbf{q}\|^2}{\|\mathbf{H}_J^m \mathbf{q}\|^2} = \lim_{m \rightarrow \infty} \frac{\rho_{\mathbf{H}_J}^{2m+2} \sum_{h: |\lambda_h| = \rho_{\mathbf{H}_J}} |q_h|^2}{\rho_{\mathbf{H}_J}^{2m} \sum_{h: |\lambda_h| = \rho_{\mathbf{H}_J}} |q_h|^2} = \rho_{\mathbf{H}_J}^2 .$$

9.2.2 Applicazione pratica

In pratica, ricordando che:

$$\phi^{(m+1)} = \mathbf{H}_J \phi^{(m)} + \mathbf{q} = \mathbf{H}_J^m \mathbf{q} + \mathbf{H}_J^{m-1} \mathbf{q} + \dots + \mathbf{q} ,$$

si ha:

$$\|\phi^{(m+1)} - \phi^{(m)}\| = \|\mathbf{H}_J^m \mathbf{q}\| .$$

Dall'equazione di iterazione, segue che:

$$\lim_{m \rightarrow \infty} \frac{\|\mathbf{H}_J^{m+1} \mathbf{q}\|}{\|\mathbf{H}_J^m \mathbf{q}\|} = \lim_{m \rightarrow \infty} \frac{\|\phi^{(m+2)} - \phi^{(m+1)}\|}{\|\phi^{(m+1)} - \phi^{(m)}\|} = \rho_{\mathbf{H}_J} .$$

Pertanto, vengono eseguite diverse iterazioni di Jacobi fino a quando il rapporto $\frac{\|\phi^{(m+2)} - \phi^{(m+1)}\|}{\|\phi^{(m+1)} - \phi^{(m)}\|}$ non viene considerato una stima sufficientemente accurata del raggio spettrale.

Inoltre si può mostrare che $\omega = \omega_{opt}$:

$$\rho_{\mathbf{H}_{SOR}} = \frac{2}{1 + \sqrt{1 - \rho_{\mathbf{H}_J}^2}} - 1$$

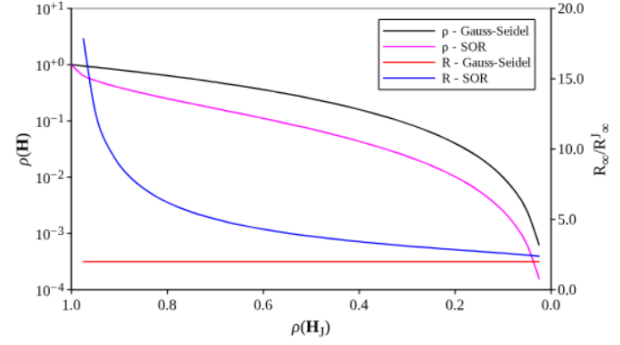


FIG. 9.2: Gauss-Siedel e Successive OverRelaxation comparati al raggio di convergenza di Jacobi.

9.3 Implementazione pratica del metodo SOR

Riprendendo l'equazione:

$$\phi^{(m+1)} = \omega \mathbf{L} \phi^{(m+1)} + [(1 - \omega) \mathbf{I} + \omega \mathbf{U}] \phi^{(m)} + \omega \mathbf{q},$$

e riscrivendola in termini di componenti, si ottiene:

$$\phi_i^{(m+1)} = \omega \sum_{j=1}^{i-1} l_{i,j} \phi_j^{(m+1)} + (1 - \omega) \phi_i^{(m)} + \omega \sum_{j=i+1}^n u_{i,j} \phi_j^{(m)} + \omega q_i .$$

Per un dominio 2D, utilizzando una notazione a doppio indice e introducendo i coefficienti $W_{i,j}$, $E_{i,j}$, $S_{i,j}$, $N_{i,j}$, si ha:

$$\begin{aligned} \phi_{i,j}^{(m+1)} &= (1 - \omega) \phi_{i,j}^{(m)} + \omega W_{i,j} \phi_{i-1,j}^{(m+1)} + \omega S_{i,j} \phi_{i,j-1}^{(m+1)} \\ &\quad + \omega E_{i,j} \phi_{i+1,j}^{(m)} + \omega N_{i,j} \phi_{i,j+1}^{(m)} + \omega q_{i,j} . \end{aligned}$$

9.4 Strategia iterativa Red & Black

Sia il metodo di Gauss-Seidel che il metodo SOR possono implementare una strategia di scansione particolare chiamata **Red & Black**.

Passo 1: Aggiornamento dei nodi rossi In questa fase, vengono aggiornati solo i nodi *rossi*:

$$\phi_{i,j}^{(m+1)} = W_{i,j} \phi_{i-1,j}^{(m)} + E_{i,j} \phi_{i+1,j}^{(m)} + S_{i,j} \phi_{i,j-1}^{(m)} + N_{i,j} \phi_{i,j+1}^{(m)} + q_{i,j} .$$

Passo 2: Aggiornamento dei nodi neri In questa fase, vengono aggiornati solo i nodi *neri*:

$$\phi_{i,j}^{(m+1)} = W_{i,j}\phi_{i-1,j}^{(m+1)} + E_{i,j}\phi_{i+1,j}^{(m+1)} + S_{i,j}\phi_{i,j-1}^{(m+1)} + N_{i,j}\phi_{i,j+1}^{(m+1)} + q_{i,j}.$$

Questa strategia consente un trattamento più coerente di ogni nodo, evitando la generazione di componenti asimmetriche spurie nell'errore. Nella valutazione del flusso in ogni nodo, i flussi dei nodi vicini appartengono tutti alla stessa iterazione.

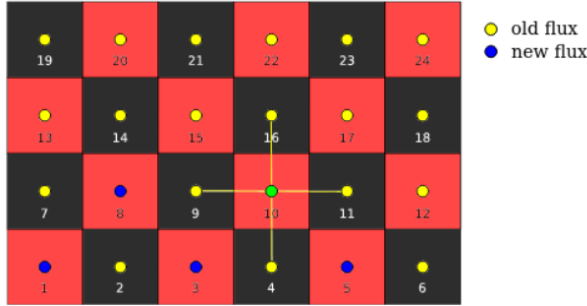


FIG. 9.3: step 1 del metodo Red & Black

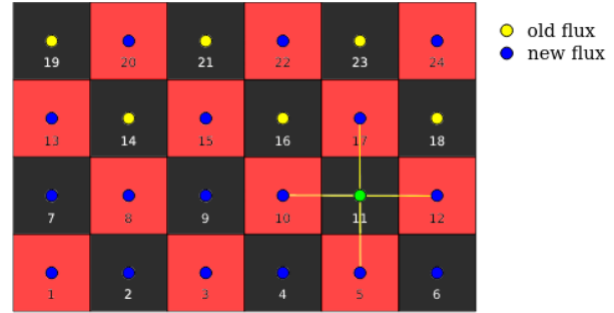


FIG. 9.4: step 2 del metodo Red & Black

9.5 Metodo Line Overrelaxation (LOR)

Facciamo riferimento al dominio 2D mostrato in figura. La matrice dei coefficienti $\mathbf{A}$ del sistema di equazioni nodali può essere letta come una matrice a blocchi (3x3), i cui blocchi diagonali (in blu) sono matrici tridiagonali.

$$\begin{bmatrix} a_{11} & a_{12} & 0 & a_{14} & 0 & 0 & 0 & 0 & 0 \\ a_{21} & a_{22} & a_{23} & 0 & a_{25} & 0 & 0 & 0 & 0 \\ 0 & a_{32} & a_{33} & 0 & 0 & a_{36} & 0 & 0 & 0 \\ a_{41} & 0 & 0 & a_{44} & a_{45} & 0 & a_{47} & 0 & 0 \\ 0 & a_{52} & 0 & a_{54} & a_{55} & a_{56} & 0 & a_{58} & 0 \\ 0 & 0 & a_{63} & 0 & a_{65} & a_{66} & 0 & 0 & a_{69} \\ 0 & 0 & 0 & a_{74} & 0 & 0 & a_{77} & a_{78} & 0 \\ 0 & 0 & 0 & 0 & a_{85} & 0 & a_{87} & a_{88} & a_{89} \\ 0 & 0 & 0 & 0 & 0 & a_{96} & 0 & a_{98} & a_{99} \end{bmatrix} \begin{bmatrix} \phi_1 \\ \phi_2 \\ \phi_3 \\ \phi_4 \\ \phi_5 \\ \phi_6 \\ \phi_7 \\ \phi_8 \\ \phi_9 \end{bmatrix} = \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \\ s_6 \\ s_7 \\ s_8 \\ s_9 \end{bmatrix}.$$

La matrice può essere riscritta in forma a blocchi:

$$\begin{bmatrix} \hat{\mathbf{A}}_{11} & \hat{\mathbf{A}}_{12} & 0 \\ \hat{\mathbf{A}}_{21} & \hat{\mathbf{A}}_{22} & \hat{\mathbf{A}}_{23} \\ 0 & \hat{\mathbf{A}}_{32} & \hat{\mathbf{A}}_{33} \end{bmatrix} \begin{bmatrix} \hat{\phi}_1 \\ \hat{\phi}_2 \\ \hat{\phi}_3 \end{bmatrix} = \begin{bmatrix} \hat{s}_1 \\ \hat{s}_2 \\ \hat{s}_3 \end{bmatrix}.$$

Equazione generica del sistema

L'equazione generica del sistema, scritta in forma a blocchi, è:

$$\hat{\mathbf{A}}_{i,i-1}\hat{\phi}_{i-1} + \hat{\mathbf{A}}_{i,i}\hat{\phi}_i + \hat{\mathbf{A}}_{i,i+1}\hat{\phi}_{i+1} = \hat{\mathbf{s}}_i \quad .$$

Il primo termine riguarda i flussi della riga $i - 1$, mentre il terzo termine riguarda i flussi della riga $i + 1$. Quando si trattano i nodi della i -esima riga, quelli sulla riga $i - 1$ sono già stati calcolati. Si può quindi adottare una strategia simile a Gauss-Seidel:

$$\hat{\mathbf{A}}_{i,i}\hat{\phi}_i^{(m+1)} = \hat{\mathbf{s}}_i - \overset{\text{line already updated}}{\hat{\mathbf{A}}_{i,i-1}\hat{\phi}_{i-1}^{(m+1)}} - \overset{\text{line not yet updated}}{\hat{\mathbf{A}}_{i,i+1}\hat{\phi}_{i+1}^{(m)}} \quad .$$

L'equazione sopra è un sistema tridiagonale che può essere risolto efficientemente con l'algoritmo di Thomas.

Il metodo

Come fatto per il metodo SOR, anche qui possiamo introdurre un'overrelaxation, scrivendo:

$$\hat{\mathbf{A}}_{i,i}\hat{\phi}_i^{(m+1)} = \hat{\mathbf{A}}_{i,i}\hat{\phi}_i^{(m)} + \omega \left[\hat{\mathbf{A}}_{i,i}\hat{\phi}_i^{(m+1)} - \hat{\mathbf{A}}_{i,i}\hat{\phi}_i^{(m)} \right]_{GS} \quad .$$

Grazie all'Eq.(30), possiamo sostituire il primo termine tra parentesi per ottenere:

$$\hat{\mathbf{A}}_{i,i}\hat{\phi}_i^{(m+1)} = \omega \hat{\mathbf{s}}_i - \omega \hat{\mathbf{A}}_{i,i-1}\hat{\phi}_{i-1}^{(m+1)} - \omega \hat{\mathbf{A}}_{i,i+1}\hat{\phi}_{i+1}^{(m)} + (1 - \omega)\hat{\mathbf{A}}_{i,i}\hat{\phi}_i^{(m)} \quad .$$

Anche in questo caso, il lato destro è un termine noto, mentre la matrice $\hat{\mathbf{A}}_{i,i}$ è tridiagonale.

Implementazione pratica del metodo LOR

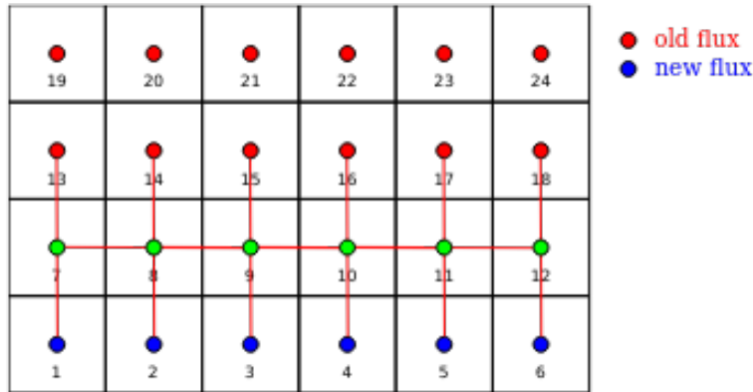


FIG. 9.5: implementazione LOR

Con riferimento alla figura precedente e adottando una notazione a doppio indice, possiamo scrivere:

$$\begin{aligned} W_{i,j}\phi_{i-1,j}^{(m+1)} + O_{i,j}\phi_{i,j}^{(m+1)} + E_{i,j}\phi_{i+1,j}^{(m+1)} &= -\omega S_{i,j}\phi_{i,j-1}^{(m+1)} - \omega N_{i,j}\phi_{i,j+1}^{(m)} \\ &+ \omega q_{i,j} + (1 - \omega) \left[W_{i,j}\phi_{i-1,j}^{(m)} + O_{i,j}\phi_{i,j}^{(m)} + E_{i,j}\phi_{i+1,j}^{(m)} \right] \quad . \end{aligned}$$

È un'equazione a tre punti.

Si noti che i coefficienti W , E , S e N non sono quelli dei metodi precedenti, qui stiamo parlando della matrice $\mathbf{A}$ più che $\mathbf{H}$. Nonostante il metodo si chiami "line overrelaxation", applicarlo per colonne o alternare fra righe e colonne funziona ugualmente.

Procedendo per colonne, l'equazione sopra diventa:

$$S_{i,j} \phi_{i,j-1}^{(m+1)} + O_{i,j} \phi_{i,j}^{(m+1)} + N_{i,j} \phi_{i,j+1}^{(m+1)} = -\omega W_{i,j} \phi_{i-1,j}^{(m+1)} - \omega E_{i,j} \phi_{i+1,j}^{(m)} + \omega q_{i,j} + (1 - \omega) \left[S_{i,j} \phi_{i,j-1}^{(m)} + O_{i,j} \phi_{i,j}^{(m)} + N_{i,j} \phi_{i,j+1}^{(m)} \right] .$$

Capitolo 10

Metodo alternate direction implicit (ADI)

10.1 La matrice di iterazione in ADI

Consideriamo il sistema lineare $\mathbf{A}\phi = \mathbf{s}$, dove la matrice $\mathbf{A}$ viene decomposta come:

$$\mathbf{A} = \mathbf{H} + \mathbf{V} + \mathbf{\Sigma} ,$$

dove:

- $\mathbf{H}$: contiene i coefficienti provenienti dalla discretizzazione della parte dipendente da x dell'operatore Laplaciano;
- $\mathbf{V}$: contiene i coefficienti provenienti dalla discretizzazione della parte dipendente da y dell'operatore Laplaciano;
- $\mathbf{\Sigma}$: contiene i coefficienti legati all'assorbimento.

Ad esempio, per un mezzo omogeneo con griglia uniforme ($h_{x,i} = h_x$, $h_{y,j} = h_y$), l'equazione di diffusione discretizzata è:

$$\underbrace{-\frac{D}{h_x^2}\phi_{i-1,j} + \frac{2D}{h_x^2}\phi_{i,j} - \frac{D}{h_x^2}\phi_{i+1,j}}_{(\mathbf{H}\phi)_{i,j}} - \underbrace{\frac{D}{h_y^2}\phi_{i,j-1} + \frac{2D}{h_y^2}\phi_{i,j} - \frac{D}{h_y^2}\phi_{i,j+1}}_{(\mathbf{V}\phi)_{i,j}} + \underbrace{\Sigma_{a,ij}\phi_{i,j}}_{(\mathbf{\Sigma}\phi)_{i,j}} = s_{i,j} .$$

10.1.1 Procedura iterativa a due passi

Tale decomposizione della matrice $\mathbf{A}$ viene utilizzata per impostare una procedura iterativa a due passi. Partendo dall'equazione:

$$(\mathbf{H} + \mathbf{V} + \mathbf{\Sigma})\phi = \mathbf{s} ,$$

si riscrive come:

$$\left(\mathbf{H} + \frac{1}{2}\mathbf{\Sigma}\right)\phi = -\left(\mathbf{V} + \frac{1}{2}\mathbf{\Sigma}\right)\phi + \mathbf{s} \quad .$$

Un parametro di rilassamento ω viene introdotto sommando a entrambi i membri dell'equazione sopra la stessa quantità $\omega\phi$, ottenendo:

$$\left(\mathbf{H} + \frac{1}{2}\mathbf{\Sigma} + \omega\mathbf{I}\right)\phi = \left(\omega\mathbf{I} - \mathbf{V} - \frac{1}{2}\mathbf{\Sigma}\right)\phi + \mathbf{s} \quad .$$

In modo equivalente, si può scrivere:

$$\left(\mathbf{V} + \frac{1}{2}\mathbf{\Sigma} + \omega\mathbf{I}\right)\phi = \left(\omega\mathbf{I} - \mathbf{H} - \frac{1}{2}\mathbf{\Sigma}\right)\phi + \mathbf{s} \quad .$$

10.1.2 Schema iterativo

Lo schema iterativo che ne segue è:

$$\begin{aligned} 1^\circ \text{ passo: } & \left(\mathbf{H} + \frac{1}{2}\mathbf{\Sigma} + \omega\mathbf{I}\right)\phi^{(m+\frac{1}{2})} = \left(\omega\mathbf{I} - \mathbf{V} - \frac{1}{2}\mathbf{\Sigma}\right)\phi^{(m)} + \mathbf{s} \quad , \\ 2^\circ \text{ passo: } & \left(\mathbf{V} + \frac{1}{2}\mathbf{\Sigma} + \omega\mathbf{I}\right)\phi^{(m+1)} = \left(\omega\mathbf{I} - \mathbf{H} - \frac{1}{2}\mathbf{\Sigma}\right)\phi^{(m+\frac{1}{2})} + \mathbf{s} \quad . \end{aligned}$$

Ogni iterazione m è quindi composta da due semi-iterazioni, date dalle equazioni precedenti. Una velocità di convergenza migliore si ottiene se il parametro ω viene cambiato ciclicamente, adottando lo schema:

$$\begin{aligned} \left(\mathbf{H} + \frac{1}{2}\mathbf{\Sigma} + \omega_m\mathbf{I}\right)\phi^{(m+\frac{1}{2})} &= \left(\omega_m\mathbf{I} - \mathbf{V} - \frac{1}{2}\mathbf{\Sigma}\right)\phi^{(m)} + \mathbf{s} \quad , \\ \left(\mathbf{V} + \frac{1}{2}\mathbf{\Sigma} + \omega_m\mathbf{I}\right)\phi^{(m+1)} &= \left(\omega_m\mathbf{I} - \mathbf{H} - \frac{1}{2}\mathbf{\Sigma}\right)\phi^{(m+\frac{1}{2})} + \mathbf{s} \quad . \end{aligned}$$

10.1.3 Struttura delle matrici

In base alle relazioni iniziali, ogni semi-iterazione ha la forma di un'equazione a tre punti, sebbene solo la prima sia tridiagonale. Considerando il sistema di 3×3 nodi mostrato in figura, la decomposizione adottata produce matrici con la seguente struttura:

$$\mathbf{A} = \mathbf{H} + \mathbf{V} + \mathbf{\Sigma}$$

$$\begin{bmatrix}
\times & \times & 0 & \times & 0 & 0 & 0 & 0 & 0 \\
\times & \times & \times & 0 & \times & 0 & 0 & 0 & 0 \\
0 & \times & \times & 0 & 0 & \times & 0 & 0 & 0 \\
\times & 0 & 0 & \times & \times & 0 & \times & 0 & 0 \\
0 & \times & 0 & \times & \times & \times & 0 & \times & 0 \\
0 & 0 & \times & 0 & \times & \times & 0 & 0 & \times \\
0 & 0 & 0 & \times & 0 & 0 & \times & \times & 0 \\
0 & 0 & 0 & 0 & \times & 0 & \times & \times & \times \\
0 & 0 & 0 & 0 & 0 & \times & 0 & \times & \times
\end{bmatrix} = \begin{bmatrix}
\times & \times & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
\times & \times & \times & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & \times & \times & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & \times & \times & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & \times & \times & \times & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & \times & \times & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & \times & \times & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & \times & \times & \times \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & \times & \times
\end{bmatrix} + \begin{bmatrix}
\times & 0 & 0 & \times & 0 & 0 & 0 & 0 & 0 \\
0 & \times & 0 & 0 & \times & 0 & 0 & 0 & 0 \\
0 & 0 & \times & 0 & 0 & \times & 0 & 0 & 0 \\
\times & 0 & 0 & \times & 0 & 0 & \times & 0 & 0 \\
0 & \times & 0 & 0 & \times & 0 & 0 & \times & 0 \\
0 & 0 & \times & 0 & 0 & \times & 0 & 0 & \times \\
0 & 0 & 0 & \times & 0 & 0 & \times & 0 & 0 \\
0 & 0 & 0 & 0 & \times & 0 & 0 & \times & 0 \\
0 & 0 & 0 & 0 & 0 & \times & 0 & 0 & \times
\end{bmatrix}$$

$\mathbf{H} + \frac{1}{2}\mathbf{\Sigma}$
(tridiagonal)

$\mathbf{V} + \frac{1}{2}\mathbf{\Sigma}$
(Band matrix)

L'aggiunta della matrice $\omega_m \mathbf{I}$ non modifica la struttura delle matrici a destra dell'uguale. Pertanto, $\mathbf{H} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I}$ è tridiagonale e può essere risolta con l'algoritmo di Thomas nella prima semi-iterazione.

La matrice $\mathbf{V} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I}$ è invece una matrice a banda. Tuttavia, è possibile trovare una matrice di permutazione $\mathbf{P}$ tale che:

$$\mathbf{P} \left(\mathbf{V} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I} \right) \mathbf{P}^T \Rightarrow \text{è tridiagonale,}$$

rendendo possibile l'uso dell'algoritmo di Thomas anche per la seconda semi-iterazione.

$$\begin{bmatrix}
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1
\end{bmatrix}
\begin{bmatrix}
\times & 0 & 0 & \times & 0 & 0 & 0 & 0 & 0 \\
0 & \times & 0 & 0 & \times & 0 & 0 & 0 & 0 \\
0 & 0 & \times & 0 & 0 & \times & 0 & 0 & 0 \\
\times & 0 & 0 & \times & 0 & 0 & \times & 0 & 0 \\
0 & \times & 0 & 0 & \times & 0 & 0 & \times & 0 \\
0 & 0 & \times & 0 & 0 & \times & 0 & 0 & \times \\
0 & 0 & 0 & \times & 0 & 0 & \times & 0 & 0 \\
0 & 0 & 0 & 0 & \times & 0 & 0 & \times & 0 \\
0 & 0 & 0 & 0 & 0 & \times & 0 & 0 & \times
\end{bmatrix}
\begin{bmatrix}
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0
\end{bmatrix} = \begin{bmatrix}
\times & \times & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
\times & \times & \times & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & \times & \times & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & \times & \times & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & \times & \times & \times & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & \times & \times & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & \times & \times & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & \times & \times & \times \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & \times & \times
\end{bmatrix}$$

$\mathbf{P} \quad \mathbf{V} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I} \quad \mathbf{P}^T$

FIG. 10.1: esempio

10.1.4 Risoluzione delle semi-iterazioni

Dall'equazione trovata per lo schema iterativo si ottiene:

$$\begin{aligned}\phi^{(m+\frac{1}{2})} &= \left(\mathbf{H} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I} \right)^{-1} \left[\left(\omega_m \mathbf{I} - \mathbf{V} - \frac{1}{2}\mathbf{\Sigma} \right) \phi^{(m)} + \mathbf{s} \right] = \\ &= \left(\mathbf{H} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I} \right)^{-1} \left(\omega_m \mathbf{I} - \mathbf{V} - \frac{1}{2}\mathbf{\Sigma} \right) \phi^{(m)} + \hat{\mathbf{s}} \ ,\end{aligned}$$

dove

$$\hat{\mathbf{s}} = \left(\mathbf{H} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I} \right)^{-1} \mathbf{s} \ .$$

Analogamente, sostituendo $\phi^{(m+\frac{1}{2})}$, si ottiene la matrice di iterazione corrispondente all'insieme delle due semi-iterazioni.

10.1.5 Matrice di iterazione del metodo ADI

$$\begin{aligned}\phi^{(m+1)} &= \left(\mathbf{V} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I} \right)^{-1} \left(\omega_m \mathbf{I} - \mathbf{H} - \frac{1}{2}\mathbf{\Sigma} \right) \phi^{(m+\frac{1}{2})} + \tilde{\mathbf{s}} = \\ &= \left(\mathbf{V} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I} \right)^{-1} \left(\omega_m \mathbf{I} - \mathbf{H} - \frac{1}{2}\mathbf{\Sigma} \right) \left[\left(\mathbf{H} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I} \right)^{-1} \right. \\ &\quad \cdot \left. \left(\omega_m \mathbf{I} - \mathbf{V} - \frac{1}{2}\mathbf{\Sigma} \right) \phi^{(m)} + \hat{\mathbf{s}} \right] + \tilde{\mathbf{s}} \\ &= \mathbf{J}_\omega \phi^{(m)} + \mathbf{Q},\end{aligned}$$

dove la matrice di iterazione $\mathbf{J}_\omega$ è definita come:

$$\mathbf{J}_\omega = \left(\mathbf{V} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I} \right)^{-1} \left(\omega_m \mathbf{I} - \mathbf{H} - \frac{1}{2}\mathbf{\Sigma} \right) \cdot \left(\mathbf{H} + \frac{1}{2}\mathbf{\Sigma} + \omega_m \mathbf{I} \right)^{-1} \left(\omega_m \mathbf{I} - \mathbf{V} - \frac{1}{2}\mathbf{\Sigma} \right) .$$

10.1.6 Convergenza del metodo ADI

È possibile dimostrare che, per qualsiasi $\omega > 0$, il raggio spettrale di $\mathbf{J}_\omega$ è minore di 1. La dimostrazione parte dalla considerazione che, se ovunque è $\Sigma_a > 0$ (come effettivamente accade), le matrici:

$$\mathbf{H} + \frac{1}{2}\mathbf{\Sigma} \quad \text{e} \quad \mathbf{V} + \frac{1}{2}\mathbf{\Sigma}$$

sono simmetriche e definite positive (SPD). Pertanto, i loro autovalori sono reali e positivi, e lo stesso vale per tutti gli autovalori λ_i di $\mathbf{J}_\omega$.

Si dimostra quindi che, se $\omega_m > 0$, tutti gli autovalori λ_i di $\mathbf{J}_\omega$ sono minori di 1, cioè:

$$0 < \lambda_i < 1 \quad \forall i.$$

Di conseguenza, $|\lambda_i| < 1$ per ogni i , garantendo la convergenza del metodo ADI.

10.2 ADI per problemi separabili

Consideriamo l'equazione di diffusione stazionaria 2D scritta nella forma:

$$\nabla^2 \phi(\mathbf{r}) + B^2 \phi(\mathbf{r}) = 0 \quad \Rightarrow \quad \frac{\partial^2 \phi(\mathbf{r})}{\partial x^2} + \frac{\partial^2 \phi(\mathbf{r})}{\partial y^2} + B^2 \phi(\mathbf{r}) = 0 \quad .$$

Analizziamo un problema separabile, in cui il flusso può essere espresso come prodotto di due funzioni di una sola variabile: $\phi(x, y) = X(x) Y(y)$. Sostituendo nell'equazione di diffusione, si ottiene:

$$\frac{1}{X} \frac{\partial^2 X}{\partial x^2} + \frac{1}{Y} \frac{\partial^2 Y}{\partial y^2} + B^2 = 0 \quad .$$

Poiché x e y sono variabili indipendenti, i primi due termini dell'equazione devono essere costanti. Pertanto:

$$\frac{1}{X_k} \frac{d^2 X_k}{dx^2} = -B_k^2 \quad ,$$

$$\frac{1}{Y_l} \frac{d^2 Y_l}{dy^2} = -B_l^2 \quad ,$$

con la condizione: $B_k^2 + B_l^2 = B^2$.

Le due equazioni sopra sono equazioni agli autovalori: X_k , B_k^2 e Y_l , B_l^2 rappresentano, rispettivamente, gli autofunzioni e gli autovalori corrispondenti ($k, l = 1, \dots, \infty$).

La soluzione generale $\phi_{k,l} = X_k Y_l$ è quindi un'autofunzione di entrambe, ma con autovalori diversi:

$$\frac{\partial^2 \phi_{k,l}}{\partial x^2} = -B_k^2 \phi_{k,l} \quad \text{e} \quad \frac{\partial^2 \phi_{k,l}}{\partial y^2} = -B_l^2 \phi_{k,l}.$$

Lo stesso vale per la versione discretizzata dell'equazione di diffusione, cioè per le matrici $\mathbf{H}$ e $\mathbf{V}$, nonché per $\mathbf{H} + \frac{1}{2}\mathbf{\Sigma}$ e $\mathbf{V} + \frac{1}{2}\mathbf{\Sigma}$, che condividono gli stessi autovettori $\phi_{k,l}$ ma con autovalori diversi:

$$\left(\mathbf{H} + \frac{1}{2}\mathbf{\Sigma} \right) \phi_{k,l} = \lambda_k \phi_{k,l},$$

$$\left(\mathbf{V} + \frac{1}{2}\mathbf{\Sigma} \right) \phi_{k,l} = \mu_l \phi_{k,l}, \quad (k = 1, \dots, n_x; \quad l = 1, \dots, n_y)$$

dove n_x e n_y sono il numero di nodi nelle direzioni x e y . Gli autovalori λ_k e μ_l hanno, rispettivamente, molteplicità n_y e n_x (ad esempio, gli n_y autovettori $\phi_{1,l}$ ($l = 1, \dots, n_y$) condividono lo stesso autovalore λ_1). Il numero totale di autovalori è $n_x \cdot n_y$.

10.2.1 Raggio spettrale della matrice di iterazione $\mathbf{J}_\omega$

Considerando la matrice di iterazione $\mathbf{J}_\omega$, le quattro matrici che la compongono hanno tutte lo stesso autovettore $\phi_{k,l}$. Pertanto:

$$\mathbf{J}_\omega \phi_{k,l} = \frac{\omega_m - \lambda_k}{\omega_m + \lambda_k} \frac{\omega_m - \mu_l}{\omega_m + \mu_l} \phi_{k,l} \quad ,$$

e il suo raggio spettrale è dato da:

$$\rho_{\mathbf{J}_\omega} = \max_{k,l} \left| \frac{\omega_m - \lambda_k}{\omega_m + \lambda_k} \frac{\omega_m - \mu_l}{\omega_m + \mu_l} \right|. \quad (19)$$

Poiché $\lambda_k, \mu_l > 0$ per ogni (k, l) , si ha:

$$\omega_m > 0 \quad \Rightarrow \quad \rho_{\mathbf{J}_\omega} < 1.$$

10.2.2 Scelta ottimale dei parametri ω

Immaginiamo ora di utilizzare due valori distinti del parametro ω , ad esempio ω_1 e ω_2 , e di eseguire una prima iterazione completa (cioè due semi-iterazioni) con $\omega = \omega_1$ e poi una seconda con $\omega = \omega_2$. La matrice di iterazione associata a questa iterazione composta è quindi uguale al prodotto $\mathbf{J}_{\omega_2} \mathbf{J}_{\omega_1}$, e si ha:

$$\mathbf{J}_{\omega_2} \mathbf{J}_{\omega_1} \phi_{k,l} = \frac{\omega_2 - \lambda_k}{\omega_2 + \lambda_k} \frac{\omega_2 - \mu_l}{\omega_2 + \mu_l} \frac{\omega_1 - \lambda_k}{\omega_1 + \lambda_k} \frac{\omega_1 - \mu_l}{\omega_1 + \mu_l} \phi_{k,l} \quad ,$$

quindi:

$$\rho_{\mathbf{J}_{\omega_2} \mathbf{J}_{\omega_1}} = \max_{k,l} \left| \frac{\omega_2 - \lambda_k}{\omega_2 + \lambda_k} \frac{\omega_2 - \mu_l}{\omega_2 + \mu_l} \frac{\omega_1 - \lambda_k}{\omega_1 + \lambda_k} \frac{\omega_1 - \mu_l}{\omega_1 + \mu_l} \right|.$$

Per una iterazione composta da p iterazioni complete, si ha:

$$\rho_{\mathbf{J}_{\omega_p} \cdots \mathbf{J}_{\omega_1}} = \max_{k,l} \left| \frac{\omega_p - \lambda_k}{\omega_p + \lambda_k} \frac{\omega_p - \mu_l}{\omega_p + \mu_l} \cdots \frac{\omega_2 - \lambda_k}{\omega_2 + \lambda_k} \frac{\omega_2 - \mu_l}{\omega_2 + \mu_l} \frac{\omega_1 - \lambda_k}{\omega_1 + \lambda_k} \frac{\omega_1 - \mu_l}{\omega_1 + \mu_l} \right|. \quad (21)$$

Se p è pari al numero n_x di autovalori distinti di $\mathbf{H} + \frac{1}{2}\mathbf{\Sigma}$ e i parametri ω sono scelti in modo che $\omega_1 = \lambda_1, \omega_2 = \lambda_2, \dots, \omega_{n_x} = \lambda_{n_x}$, il raggio spettrale $\rho_{\mathbf{J}_{\omega_p} \cdots \mathbf{J}_{\omega_1}}$ è uguale a zero. Infatti, nell'ultima equazione, per ogni coppia (k, l) esiste sempre un fattore nullo.

In queste condizioni, il processo iterativo raggiunge la soluzione esatta in un numero finito di iterazioni.

Esempio con sistema 2x2 nodi

Consideriamo un sistema di 2×2 nodi ($n_x = n_y = 2$) e poniamo $p = n_x = 2$. Gli autovalori δ_i della matrice di iterazione $\mathbf{J}_{\omega_2}, \mathbf{J}_{\omega_1}$ sono:

$$\begin{aligned}\delta_1 &= \frac{\omega_2 - \lambda_1}{\omega_2 + \lambda_1} \frac{\omega_2 - \mu_1}{\omega_2 + \mu_1} \frac{\omega_1 - \lambda_1}{\omega_1 + \lambda_1} \frac{\omega_1 - \mu_1}{\omega_1 + \mu_1}, \\ \delta_2 &= \frac{\omega_2 - \lambda_1}{\omega_2 + \lambda_1} \frac{\omega_2 - \mu_2}{\omega_2 + \mu_2} \frac{\omega_1 - \lambda_1}{\omega_1 + \lambda_1} \frac{\omega_1 - \mu_2}{\omega_1 + \mu_2}, \\ \delta_3 &= \frac{\omega_2 - \lambda_2}{\omega_2 + \lambda_2} \frac{\omega_2 - \mu_1}{\omega_2 + \mu_1} \frac{\omega_1 - \lambda_2}{\omega_1 + \lambda_2} \frac{\omega_1 - \mu_1}{\omega_1 + \mu_1}, \\ \delta_4 &= \frac{\omega_2 - \lambda_2}{\omega_2 + \lambda_2} \frac{\omega_2 - \mu_2}{\omega_2 + \mu_2} \frac{\omega_1 - \lambda_2}{\omega_1 + \lambda_2} \frac{\omega_1 - \mu_2}{\omega_1 + \mu_2}.\end{aligned}$$

Se scegliamo $\omega_1 = \lambda_1$ e $\omega_2 = \lambda_2$, si ottiene:

$$\delta_1 = \delta_2 = \delta_3 = \delta_4 = 0 \quad \Rightarrow \quad \rho_{\mathbf{J}_{\omega_2} \mathbf{J}_{\omega_1}} = 0.$$

Stima dell'intervallo $[\alpha, \beta]$ per gli autovalori

Tuttavia, la conoscenza di tutti gli autovalori della matrice $\mathbf{H} + \frac{1}{2}\mathbf{\Sigma}$ (o $\mathbf{V} + \frac{1}{2}\mathbf{\Sigma}$) è spesso un compito molto difficile. Inoltre, essendo il numero di autovalori molto elevato, potrebbe non essere conveniente eseguire anche una singola iterazione composta.

L'idea è quindi di stimare un intervallo $[\alpha, \beta]$ che contenga tutti gli autovalori delle suddette matrici, cioè tale che: $\alpha \leq \lambda_k, \mu_l \leq \beta \quad \forall k, l$.

Minimizzazione del raggio spettrale

Dall'equazione del raggio spettrale possiamo scrivere:

$$\begin{aligned}\rho_{\mathbf{J}_{\omega_p} \dots \mathbf{J}_{\omega_1}} &= \max_k \left(\left| \frac{\omega_p - \lambda_k}{\omega_p + \lambda_k} \dots \frac{\omega_2 - \lambda_k}{\omega_2 + \lambda_k} \frac{\omega_1 - \lambda_k}{\omega_1 + \lambda_k} \right| \right) \\ &\quad \cdot \max_l \left(\left| \frac{\omega_p - \mu_l}{\omega_p + \mu_l} \dots \frac{\omega_2 - \mu_l}{\omega_2 + \mu_l} \frac{\omega_1 - \mu_l}{\omega_1 + \mu_l} \right| \right) \\ &\leq \left[\max_{\alpha \leq x \leq \beta} \left| \frac{\omega_p - x}{\omega_p + x} \dots \frac{\omega_2 - x}{\omega_2 + x} \frac{\omega_1 - x}{\omega_1 + x} \right| \right]^2.\end{aligned}$$

Poiché $\alpha \leq x \leq \beta$, se x è tale da restituire il valore massimo, i due termini possono, al massimo, uguagliarlo, giustificando così l'ultima disuguaglianza.

Il problema si riduce quindi a minimizzare il contenuto tra parentesi quadrate, cioè:

$$\min_{\omega_p, \dots, \omega_2, \omega_1} \max_{\alpha \leq x \leq \beta} \left| \frac{\omega_p - x}{\omega_p + x} \dots \frac{\omega_2 - x}{\omega_2 + x} \frac{\omega_1 - x}{\omega_1 + x} \right|. \quad (24)$$

10.3 Considerazioni finali

- Senza l'uso di un parametro di rilassamento ω , il metodo ADI è generalmente meno performante del metodo LOR. Dalle equazioni di semi-iterazione si osserva che ogni semi-iterazione si basa sui flussi del passo precedente, anche se sarebbero disponibili valori aggiornati.
- Per problemi non separabili, non esiste una teoria rigorosa per la selezione dei parametri ω .
- Pertanto, il metodo ADI offre prestazioni ottimali per problemi separabili, mentre può essere molto meno efficiente per problemi non separabili.
- Per un problema quasi separabile, è possibile utilizzare il problema separabile più vicino come "problema modello" per calcolare i parametri ω . In generale, tali parametri forniscono buone prestazioni anche per problemi quasi separabili.

Capitolo 11

Metodi del gradiente

I metodi del gradiente costituiscono una classe di metodi iterativi utilizzati per risolvere sistemi lineari della forma

$$Ax = b,$$

dove $A \in \mathbb{R}^{n \times n}$ è una matrice simmetrica definita positiva (SPD).

Ricordiamo che una matrice simmetrica è definita positiva se vale la proprietà

$$v^T Av = (Av, v) > 0 \quad \forall v \neq \{0\} \quad .$$

Nota che la scrittura $v^T Av = (Av, v)$ è vera perché $\mathbf{A}$ è simmetrica, altrimenti se non lo fosse (Av, v) si svilupperebbe come $(Av)^T v = v^T A^T v$, che non è chiaramente la stessa cosa.

11.1 Formulazione variazionale del problema

Sia $x = \phi$ la soluzione del sistema lineare $Ax = b$.

Si può dimostrare che tale soluzione coincide con il minimo del funzionale quadratico

$$F(x) = \frac{1}{2}(Ax, x) - (b, x).$$

Per cui vale la relazione:

$$F(\phi) = \min_x F(x) \quad .$$

Dimostrazione

Poiché $b = A\phi$, il funzionale può essere riscritto come

$$F(x) = \frac{1}{2}(Ax, x) - (A\phi, x) \quad .$$

Valutando il funzionale nella soluzione esatta si ottiene

$$F(\phi) = \frac{1}{2}(A\phi, \phi) - (A\phi, \phi) = -\frac{1}{2}(A\phi, \phi) .$$

O, equivalentemente:

$$F(\phi) + \frac{1}{2}(A\phi, \phi) = 0 .$$

Aggiungendo e sottraendo opportunamente termini, si può riscrivere $F(x)$ come

$$\begin{aligned} F(x) &= \frac{1}{2}(A\phi, \phi) - \underbrace{(A\phi, x) + F(\phi) + \frac{1}{2}(A\phi, \phi)}_{=0} = \\ &= F(\phi) + \frac{1}{2}(Ax, x) - \frac{1}{2}(A\phi, x) - \frac{1}{2}(A\phi, x) + \frac{1}{2}(A\phi, \phi) = \\ &= F(\phi) + \frac{1}{2}(A(x - \phi), x) - \frac{1}{2}(A\phi, (x - \phi)) = \\ &= F(\phi) + \frac{1}{2}(A(x - \phi), x) - \frac{1}{2}(A(x - \phi), \phi) = \\ &= F(\phi) + \frac{1}{2}(A(x - \phi), x - \phi). \end{aligned}$$

Poiché A è definita positiva, il termine $(A(x - \phi), x - \phi)$ è sempre positivo per $x \neq \phi$. Ne segue che il minimo del funzionale è raggiunto esattamente per $x = \phi$.

Questo risultato mostra che la risoluzione del sistema lineare $Ax = b$ è equivalente alla minimizzazione del funzionale $F(x)$.

11.2 Interpretazione tramite errore

Data una soluzione approssimata x , si possono definire diverse misure dell'errore:

$$\begin{aligned} E_1(x) &= (\phi - x)^T (\phi - x) = \varepsilon^T \varepsilon , \\ E_2(x) &= (b - Ax)^T (b - Ax) = r^T r , \\ E_3(x) &= (\phi - x)^T (b - Ax) = \varepsilon^T A \varepsilon . \end{aligned} \tag{11.1}$$

Indicando con $\varepsilon = \phi - x$ e osservando che $b - Ax = A\phi - Ax = A(\phi - x) = A\varepsilon$.

Tutte queste quantità sono nulle se e solo se $x = \phi$.

11.2.1 Relazione tra errore e funzionale

Espandendo $E_3(x)$ si ottiene

$$\begin{aligned} E_3(x) &= \phi^T b - \phi^T Ax - x^T b + x^T Ax \\ &= \phi^T b + x^T Ax - 2x^T b . \end{aligned}$$

Utilizzando la definizione del funzionale $F(x)$, si ricava

$$E_3(x) = \phi^T b + 2F(x) .$$

Affinché l'errore si annulli nella soluzione esatta, deve valere

$$F(\phi) = -\frac{1}{2}\phi^T b = -\frac{1}{2}(A\phi, \phi) ,$$

che coincide con il valore minimo del funzionale.

In conclusione, la minimizzazione del funzionale $F(x)$ è completamente equivalente alla risoluzione del sistema lineare $Ax = b$, e fornisce la base teorica per lo sviluppo dei metodi iterativi basati sul gradiente.

11.3 Metodi di discesa

I metodi di discesa sono metodi iterativi che mirano a determinare il minimo del funzionale $F(x)$ costruendo una successione di approssimazioni $x^{(0)}, x^{(1)}, x^{(2)}, \dots$ a partire da una stima iniziale $x^{(0)}$.

L'idea fondamentale è che ogni nuova iterazione migliori la precedente, ovvero che il valore del funzionale decresca:

$$F(x^{(k+1)}) \leq F(x^{(k)}) ,$$

idealmente con disuguaglianza stretta.

Quando si raggiunge una configurazione tale che $Ax^{(k)} \approx b$, il processo può essere arrestato e $x^{(k)}$ viene considerata una buona approssimazione della soluzione.

Struttura generale del metodo

Il passaggio da $x^{(k)}$ a $x^{(k+1)}$ avviene tramite un processo in due fasi fondamentali:

1. **Scelta della direzione di ricerca** $p^{(k)}$;
2. **Scelta del passo** α_k , che determina quanto muoversi lungo tale direzione.

La nuova iterata è quindi definita come

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)} .$$

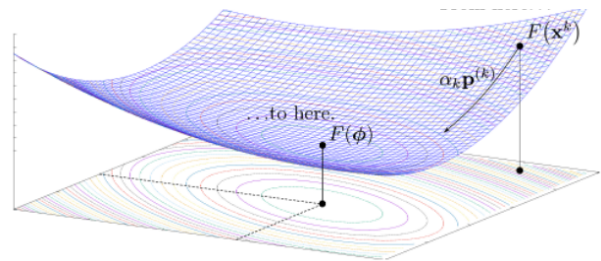


FIG. 11.1: Visualizzazione delle grandezze definite.

Dal punto di vista matematico: $p^{(k)} \in \mathbb{R}^n$ è il vettore direzione; $\alpha_k \in \mathbb{R}$ è un parametro scalare (step size).

Scelta della direzione

La scelta della direzione $p^{(k)}$ è cruciale per l'efficienza del metodo. Una scelta naturale per la direzione di discesa consiste nell'utilizzare il gradiente del funzionale. Infatti, la direzione in cui una funzione scalare varia più rapidamente è proprio quella del suo gradiente.

Si sceglie quindi come direzione di ricerca il negativo del gradiente:

$$\begin{aligned} p^{(k)} &= -\nabla F(x^{(k)}) = -\nabla \left[\frac{1}{2}(Ax^{(k)}, x) - (b, x^{(k)}) \right] = \\ &= -\nabla \left[\frac{1}{2}x^{(k)T} A x^{(k)} - x^{(k)T} b \right] = \\ &= - \left[\frac{1}{2}Ax^{(k)} + \frac{1}{2}x^{(k)T} A - b \right] . \end{aligned}$$

Poiché A è simmetrica ($A = A^T$), segue $\nabla F(x^{(k)}) = Ax^{(k)} - b$ e, pertanto, la direzione di discesa diventa:

$$\boxed{p^{(k)} = b - Ax^{(k)} = r^{(k)}} , \quad (11.2)$$

dove $r^{(k)}$ è il residuo, direzione lungo la quale il funzionale decresce maggiormente a partire da $x^{(k)}$.

Scelta del passo

Il parametro α_k viene scelto imponendo che il funzionale sia minimo lungo la direzione $r^{(k)}$, cioè:

$$F(x^{(k+1)}) = \min_{\alpha_k} F(x^{(k)} + \alpha_k r^{(k)}) . \quad (11.3)$$

Per determinare tale valore, si impone la condizione di stazionarietà $\frac{d}{d\alpha_k} F(x^{(k+1)}) = 0$ e sostituendo $x^{(k+1)} = x^{(k)} + \alpha_k r^{(k)}$ nel funzionale si ottiene:

$$F(x^{(k+1)}) = \frac{1}{2} (A(x^{(k)} + \alpha_k r^{(k)}), x^{(k)} + \alpha_k r^{(k)}) - (b, x^{(k)} + \alpha_k r^{(k)}) ,$$

ovvero, in forma matriciale:

$$F(x^{(k+1)}) = \frac{1}{2}(x^{(k)} + \alpha_k r^{(k)})^T A (x^{(k)} + \alpha_k r^{(k)}) - (x^{(k)} + \alpha_k r^{(k)})^T b .$$

Sviluppando i termini si ottiene:

$$F(x^{(k+1)}) = \frac{1}{2}x^{(k)T} A x^{(k)} + \alpha_k r^{(k)T} A x^{(k)} + \frac{1}{2}\alpha_k^2 r^{(k)T} A r^{(k)} - x^{(k)T} b - \alpha_k r^{(k)T} b .$$

Raccogliendo i termini:

$$F(x^{(k+1)}) = F(x^{(k)}) + \alpha_k r^{(k)T} (Ax^{(k)} - b) + \frac{1}{2}\alpha_k^2 r^{(k)T} A r^{(k)} .$$

Poiché $r^{(k)} = b - Ax^{(k)}$, segue:

$$F(x^{(k+1)}) = F(x^{(k)}) - \alpha_k r^{(k)T} r^{(k)} + \frac{1}{2}\alpha_k^2 r^{(k)T} A r^{(k)} .$$

Imponendo la condizione di minimo:

$$\frac{d}{d\alpha_k} F(x^{(k+1)}) = -r^{(k)T} r^{(k)} + \alpha_k r^{(k)T} A r^{(k)} = 0,$$

da cui si ricava

$$\alpha_k = \frac{r^{(k)T} r^{(k)}}{r^{(k)T} A r^{(k)}} = \frac{(r^{(k)}, r^{(k)})}{(A r^{(k)}, r^{(k)})} .$$

La condizione di minimo del funzionale lungo la direzione di ricerca può essere riscritta in forma più compatta utilizzando la regola della derivazione composta. Infatti, si ha:

$$\frac{d}{d\alpha_k} F(x^{(k+1)}) = \frac{dx^{(k+1)}}{d\alpha_k} \cdot \frac{dF(x^{(k+1)})}{dx^{(k+1)}} = 0 .$$

Dalla definizione dell'iterazione $x^{(k+1)} = x^{(k)} + \alpha_k r^{(k)}$, segue immediatamente che

$$\frac{dx^{(k+1)}}{d\alpha_k} = r^{(k)} .$$

D'altra parte, la derivata del funzionale rispetto alla variabile x è il suo gradiente, quindi:

$$\frac{dF(x^{(k+1)})}{dx^{(k+1)}} = \nabla F(x^{(k+1)}) .$$

Ricordando che $\nabla F(x) = Ax - b = -r$, si ottiene:

$$\nabla F(x^{(k+1)}) = -r^{(k+1)} .$$

Sostituendo nella condizione di cui sopra, $r^{(k)} \cdot (-r^{(k+1)}) = 0$, segue la proprietà fondamentale

$$r^{(k)} r^{(k+1)} = 0 \quad \rightarrow \quad r^{(k)} \perp r^{(k+1)} .$$

Interpretazione

La relazione precedente mostra che i residui di due iterazioni successive sono ortogonali tra loro.

Questa proprietà ha un significato geometrico molto importante: il passo α_k è scelto in modo tale che il nuovo punto $x^{(k+1)}$ sia il minimo del funzionale lungo la direzione $r^{(k)}$.

Di conseguenza:

- il gradiente nel punto $x^{(k+1)}$ è ortogonale alla direzione seguita;
- poiché il gradiente è opposto al residuo, si ha che $r^{(k+1)}$ è ortogonale a $r^{(k)}$.

In altre parole, ogni iterazione elimina completamente la componente dell'errore lungo la direzione appena utilizzata.

Tuttavia, in generale non vale

$$r^{(k)} r^{(j)} = 0 \quad \text{per } j \neq k \pm 1,$$

cioè i residui non sono tutti mutuamente ortogonali, ma solo quelli consecutivi.

11.3.1 Algoritmo del Steepest Descent Method

Ecco lo pseudocodice che implementa il metodo, con ogni passaggio commentato:

Listing 11.1: Algoritmo del Steepest Descent Method

```
% Inizializzazione
x = x0;           % Stima iniziale
r = b - A * x;    % Residuo iniziale

% Iterazione fino a convergenza
while ||r|| > tolleranza

    % Calcolo del parametro alpha_k
    alpha = (r^T * r) / (r^T * A * r);

    % Aggiornamento della soluzione x_k -> x_{k+1}
    x = x + alpha * r;

    % Aggiornamento del residuo in funzione di x_{k+1}
    r = b - A * x;
    % potrebbe essere calcolato anche come
    % b - A*x_k - alpha_k*A*r_k = r_k - alpha_k*A*r_k
end
```

- **x0**: Stima iniziale della soluzione.
- **r**: Residuo iniziale, calcolato come $\mathbf{b} - \mathbf{A}\mathbf{x}$.
- **alpha**: Parametro di passo calcolato come $\frac{\mathbf{r}^T \mathbf{r}}{\mathbf{r}^T \mathbf{A} \mathbf{r}}$.
- **while ||r|| > tolleranza**: Iterazione finché il residuo non scende sotto una soglia prefissata.
- **x = x + alpha * r**: Aggiornamento della soluzione lungo la direzione del residuo.
- **r = b - A * x**: Aggiornamento del residuo dopo ogni iterazione.

11.3.2 Interpretazione geometrica

Data l'eq.(11.1), dove $\epsilon = \phi - \mathbf{x}$, se espandiamo ϵ in termini degli autovettori di $\mathbf{A}$, $\epsilon = \mathbf{U}\mathbf{z}$, dove $\mathbf{U}$ è la matrice ortogonale degli autovettori e $\mathbf{z}$ è il vettore dei coefficienti, otteniamo:

$$E_3(\mathbf{z}) = (\mathbf{U}\mathbf{z})^T \mathbf{A} \mathbf{U}\mathbf{z} = \mathbf{z}^T \mathbf{U}^T \mathbf{A} \mathbf{U}\mathbf{z} = \mathbf{z}^T \mathbf{\Lambda} \mathbf{z},$$

con $\mathbf{\Lambda}$ matrice diagonale degli autovalori λ_i di $\mathbf{A}$.

Iperellissoidi e autovalori

L'equazione (??) può essere riscritta come: $E_3(\mathbf{z}) = \lambda_1 z_1^2 + \lambda_2 z_2^2 + \cdots + \lambda_n z_n^2$.

I luoghi a errore costante sono rappresentati da iperellissoidi i cui semiassi sono legati agli autovalori di $\mathbf{A}$:

$$r_i = \sqrt{\frac{E_3(\mathbf{z})}{\lambda_i}}.$$

La soluzione esatta ϕ corrisponde al centro degli iperellissoidi (dove $E_3(\mathbf{z}) = 0$).

Visualizzazione geometrica

Esempio tridimensionale ($n = 3$) di iperellissoide.

Il metodo di discesa sposta $\mathbf{x}^{(k)}$ sulla superficie $F(x)$ lungo la direzione di massima pendenza $\mathbf{r}^{(k)}$ perpendicolare a $F(x^{(k)})$ fino al punto $x^{(k+1)}$ dove il funzionale assume il suo minimo valore lungo la direzione di spostamento.

Siccome vale l'ortogonalità dei residui successivi $\mathbf{r}^{(k)} \perp \mathbf{r}^{(k+1)}$, implica che, al passo $k + 1$, la direzione di discesa $\mathbf{r}^{(k+1)}$ è perpendicolare all'iperellissoide corrispondente a $F(\mathbf{x}^{(k+1)})$ oltre che a $\mathbf{r}^{(k)}$.

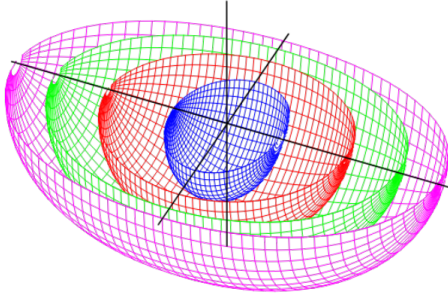


FIG. 11.2: Visualizzazione degli ellissoidi per $n=3$

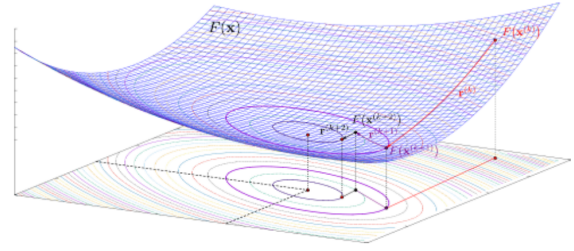


FIG. 11.3: Visualizzazione della discesa

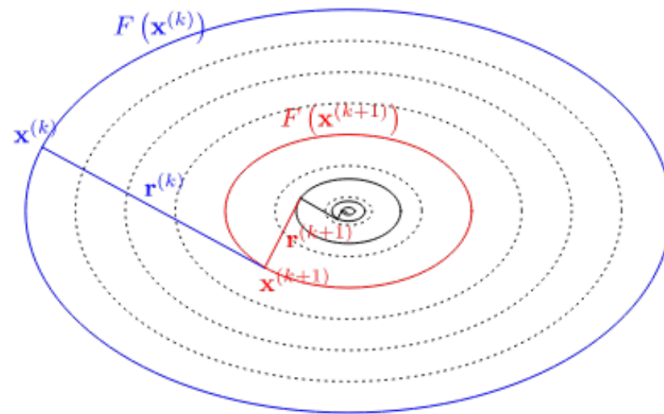


FIG. 11.4: Metodo dello *steepest descent* visualizzato nella sua totalità

Condition number e convergenza

Se la matrice $\mathbf{A}$ ha numero di condizione $\kappa(\mathbf{A}) = 1$ (cioè tutti gli autovalori sono uguali), gli iperellissoidi degenerano in ipersfere e il metodo converge in una sola iterazione, indipendentemente dal guess iniziale.

$$\kappa(\mathbf{A}) = \frac{|\lambda_{\max}(\mathbf{A})|}{|\lambda_{\min}(\mathbf{A})|}$$

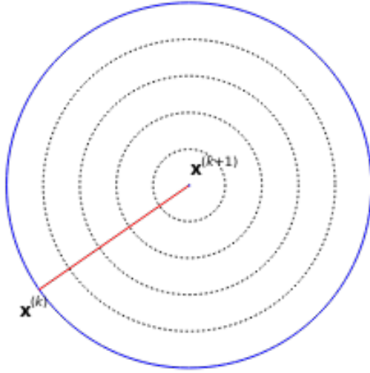


FIG. 11.5: Caso in cui gli iperellissoidi degenerano in ipersfere

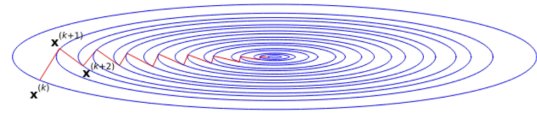


FIG. 11.6: Caso di matrice malcondizionata, che converge lentamente

Se gli autovalori sono raggruppati in un intervallo ristretto, la convergenza è veloce. Al contrario, se gli autovalori sono molto spazati (matrice mal condizionata), la convergenza è lenta.

Se $\lambda_1 \gg \lambda_2$, l'iperellissoide è molto allungato lungo la direzione corrispondente a λ_1 . Il metodo di discesa "oscilla" tra le direzioni principali, rallentando la convergenza, come si vede in fig.(??).

11.4 Conjugate Gradient Method

Il metodo di Steepest Descent può essere lento, soprattutto per matrici mal condizionate, nel senso che può tornare spesso a muoversi in direzioni già usate in step precedenti.

Il *Conjugate Gradient Method* è un algoritmo più efficiente che sfrutta direzioni di ricerca linearmente indipendenti. L'algoritmo è simile a quello di Steepest Descent, ma la direzione di ricerca $\mathbf{p}^{(k)}$ è aggiornata come:

$$\mathbf{p}^{(k)} = \mathbf{r}^{(k)} + \beta_k \mathbf{p}^{(k-1)},$$

dove β_k è scelto per garantire l' $\mathbf{A}$ -ortogonalità: $(\mathbf{A}\mathbf{p}^{(k-1)}, \mathbf{p}^{(k)}) = \mathbf{p}^{(k)T} \mathbf{A}\mathbf{p}^{(k-1)} = 0$.

Calcolo di β_k e del passo α_k

Imponendo la condizione di coniugazione, si ottiene:

$$(\mathbf{A}\mathbf{p}^{(k-1)}, \mathbf{r}^{(k)} + \beta_k \mathbf{p}^{(k-1)}) = 0$$

$$\beta_k = -\frac{\mathbf{r}^{(k)T} \mathbf{A} \mathbf{p}^{(k-1)}}{\mathbf{p}^{(k-1)T} \mathbf{A} \mathbf{p}^{(k-1)}}.$$

L'insieme di direzioni di ricerca $\mathbf{p}^{(k)}$ così ottenuto formano un insieme di vettori linearmente indipendenti.

Il parametro α_k è scelto per minimizzare $F(\mathbf{x}^{(k+1)})$ lungo $\mathbf{p}^{(k)}$:

$$\begin{aligned} F(\mathbf{x}^{(k+1)}) &= \frac{1}{2} (\mathbf{A}(\mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}), (\mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)})) - (\mathbf{b}, (\mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)})) = \\ &= F(\mathbf{x}^{(k)}) - \alpha_k \mathbf{p}^{(k)T} \mathbf{r}^{(k)} + \frac{1}{2} \alpha_k^2 \mathbf{p}^{(k)T} \mathbf{A} \mathbf{p}^{(k)}, \end{aligned}$$

$$\frac{dF(\mathbf{x}^{(k+1)})}{d\alpha_k} = 0 \quad \rightarrow \quad -\mathbf{p}^{(k)T} \mathbf{r}^{(k)} + \alpha_k \mathbf{p}^{(k)T} \mathbf{A} \mathbf{p}^{(k)} = 0,$$

$$\alpha_k = \frac{\mathbf{p}^{(k)T} \mathbf{r}^{(k)}}{\mathbf{p}^{(k)T} \mathbf{A} \mathbf{p}^{(k)}} = \frac{(\mathbf{r}^{(k)}, \mathbf{p}^{(k)})}{(\mathbf{A} \mathbf{p}^{(k)}, \mathbf{p}^{(k)})}.$$

Listing 11.2: Implementazione del CGM

```
% Inizializzazione
x = x0;                % Stima iniziale
r = b - A * x;         % Residuo iniziale
p = r;                 % Direzione iniziale
k = 0;                 % Contatore iterazioni

% Iterazione fino a convergenza
while |r| > tolleranza
    % Calcolo di alpha_k
    alpha = (p^T * r) / (p^T * A * p);

    % Aggiornamento della soluzione x_k -> x_{k+1}
    x = x + alpha * p;

    % Nuova direzione del residuo r_{k+1}
    r_new = r - alpha * A * p;

    if |r| < tollorenza
        end

    % Calcolo di beta_{k+1}
    beta = (r_new^T * A * p) / (p^T * A * p);

    % Aggiornamento della direzione di ricerca p^k -> p^{k+1}
    p = r_new + beta * p;

    % Preparazione per la prossima iterazione
```

```

    r = r_new;
    k = k + 1;
end

```

- **x0**: Stima iniziale della soluzione.
- **r**: Residuo iniziale, $\mathbf{r}^{(0)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(0)}$.
- **p**: Direzione di ricerca inizializzata come il residuo.
- **alpha**: Parametro di passo calcolato come $\alpha_k = \frac{\mathbf{p}^{(k)T} \mathbf{r}^{(k)}}{\mathbf{p}^{(k)T} \mathbf{A} \mathbf{p}^{(k)}}$.
- **beta**: Parametro di coniugazione calcolato come $\beta_k = \frac{\mathbf{r}^{(k+1)T} \mathbf{A} \mathbf{p}^{(k)}}{\mathbf{p}^{(k)T} \mathbf{A} \mathbf{p}^{(k)}}$.

Proprietà del metodo

Il Conjugate Gradient Method gode di proprietà fondamentali:

- **Ortogonalità dei residui**: $\mathbf{r}^{(i)T} \mathbf{r}^{(j)} = 0$ per $i \neq j$.
- **Coniugazione delle direzioni**: $\mathbf{p}^{(i)T} \mathbf{A} \mathbf{p}^{(j)} = 0$ per $i \neq j$.
- **Convergenza in n iterazioni**: Per un sistema di dimensione n , il metodo converge esattamente alla soluzione in al massimo n passi (a meno di errori di arrotondamento).
- **Efficienza**: Ogni iterazione richiede una sola valutazione del prodotto matrice-vettore $\mathbf{A} \mathbf{p}^{(k)}$.

Si nota anche che per $k \in [0, n]$ e per $\ell \in [0, n - k - 1]$ si verifica:

$$\begin{aligned}
 (\mathbf{r}^{(k+\ell+1)}, \mathbf{r}^{(k)}) &= (\mathbf{r}^{(k+\ell)} - \alpha_{k+\ell} \mathbf{A} \mathbf{p}^{(k+\ell)}, \mathbf{r}^{(k)}) = \\
 &= \mathbf{r}^{(k)T} \mathbf{r}^{(k+\ell)} - \frac{\mathbf{p}^{(k+\ell)T} \mathbf{r}^{(k+\ell)}}{\mathbf{p}^{(k+\ell)T} \mathbf{A} \mathbf{p}^{(k+\ell)}} \mathbf{r}^{(k)T} \mathbf{A} \mathbf{p}^{(k+\ell)} = \\
 &= \mathbf{r}^{(k)T} \mathbf{r}^{(k+\ell)} - \mathbf{r}^{(k+\ell)T} \mathbf{r}^{(k)} \frac{\mathbf{p}^{(k+\ell)T} \mathbf{A} \mathbf{p}^{(k+\ell)}}{\mathbf{p}^{(k+\ell)T} \mathbf{p}^{(k+\ell)}} = \\
 &= 0
 \end{aligned}$$

Il residuo $\mathbf{r}^{(k+1)}$ è ortogonale a tutti i $\mathbf{r}^{(j)}$ precedenti, quindi $\mathbf{r}^{(n)} = 0$ e $\mathbf{x}^{(n)} = \phi$ (soluzione esatta).

Le direzioni $\mathbf{p}^{(k)}$ sono linearmente indipendenti e $\mathbf{A}$ -ortogonali, garantendo la convergenza rapida.

Capitolo 12

Metodi del sottospazio di Krylov

12.0.1 Contesto generale

Nel contesto della risoluzione di sistemi lineari derivanti da equazioni differenziali alle derivate parziali (PDE) discretizzate:

- Le matrici dei coefficienti sono **sparse** (ad esempio, equazioni a 3, 5 o 7 punti).
- I metodi diretti, come la fattorizzazione LU, possono generare un **fill-in** significativo.
- La risoluzione di sistemi triangolari (es. $\mathbf{L}\psi = \mathbf{b}$; $\mathbf{U}\phi = \psi$) può richiedere un numero eccessivo di FLOPs.
- Metodi come il **gradiente coniugato** (applicabile solo a matrici SPD) non aumentano il costo computazionale e la memoria con le iterazioni.
- L'estensione di questi metodi a matrici generali richiede la creazione di una base ortogonale incrementale per il **sottospazio di Krylov**.
- Tuttavia, il numero di FLOPs e la memoria crescono ad ogni iterazione.

12.1 Sistema di equazioni lineari con matrice generale $\mathbf{A}$

Dato il sistema di equazioni lineari:

$$\mathbf{Ax} = \mathbf{b}, \quad \text{con } \mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n.$$

Dato un valore iniziale $\mathbf{x}_0$, si definisce il residuo $\mathbf{r}_0 = \mathbf{b} - \mathbf{Ax}_0$. La soluzione esatta può essere scritta come:

$$\mathbf{x} = \mathbf{x}_0 + \mathbf{A}^{-1}\mathbf{r}_0.$$

Punto chiave: Poiché il calcolo di $\mathbf{A}^{-1}$ è spesso troppo costoso per sistemi sparsi di grandi dimensioni, è preferibile:

- Cercare un'approssimazione di $\mathbf{A}^{-1}\mathbf{r}_0$;
- Utilizzare solo operazioni convenienti (ad esempio, prodotti matrice-vettore).

12.2 Approssimazione di $\mathbf{A}^{-1}$

Invece di calcolare $\mathbf{A}^{-1}$, si approssima la sua azione sul residuo $\mathbf{r}_0$ tramite un polinomio opportuno in $\mathbf{A}$:

$$\mathbf{A}^{-1}\mathbf{r}_0 \approx p_{k-1}(\mathbf{A})\mathbf{r}_0 = \sum_{i=0}^{k-1} c_i \mathbf{A}^i \mathbf{r}_0.$$

Quindi, una soluzione approssimata del sistema $\mathbf{A}\mathbf{x} = \mathbf{b}$ è:

$$\mathbf{x}_k = \mathbf{x}_0 + p_{k-1}(\mathbf{A})\mathbf{r}_0.$$

Punto chiave: Ogni vettore della forma $p_{k-1}(\mathbf{A})\mathbf{r}_0$ appartiene allo spazio:

$$\mathcal{K}_k(\mathbf{A}, \mathbf{r}_0) = \text{span}\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \dots, \mathbf{A}^{k-1}\mathbf{r}_0\}. \quad (5)$$

12.3 Il sottospazio di Krylov

Lo spazio

$$\mathcal{K}_k(\mathbf{A}, \mathbf{r}_0) = \text{span}\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \dots, \mathbf{A}^{k-1}\mathbf{r}_0\}$$

è chiamato **sottospazio di Krylov** di ordine k .

12.3.1 Interpretazione

Cosa rappresentano gli elementi del sottospazio di Krylov? Immaginiamo $\mathbf{A}$ come l'approssimazione di un operatore differenziale:

- $\mathbf{r}_0$: residuo iniziale;
- $\mathbf{A}\mathbf{r}_0$: misura della derivata (locale) del residuo;
- $\mathbf{A}^2\mathbf{r}_0, \dots$: rappresentano derivate di ordine superiore (locali).

Ogni moltiplicazione per $\mathbf{A}$ estrae nuove informazioni sul comportamento del sistema.

12.3.2 Forma del residuo

Secondo l'equazione della soluzione approssimata di cui sopra, il residuo $\mathbf{r}_k$ assume la forma:

$$\begin{aligned} \mathbf{r}_k &= \mathbf{b} - \mathbf{A}\mathbf{x}_k = \mathbf{b} - \mathbf{A}(\mathbf{x}_0 + p_{k-1}(\mathbf{A})\mathbf{r}_0) = \\ &= (\mathbf{I} - \mathbf{A}p_{k-1}(\mathbf{A}))\mathbf{r}_0, \end{aligned}$$

che, introducendo un nuovo polinomio matriciale di ordine k , $q_k(\mathbf{A}) = (\mathbf{I} - \mathbf{A}p_{k-1}(\mathbf{A}))$, si riduce a:

$$\mathbf{r}_k = q_k(\mathbf{A})\mathbf{r}_0.$$

La convergenza è quindi governata dalla capacità del polinomio matriciale $q_k(\mathbf{A})$ di smorzare efficacemente il residuo iniziale $\mathbf{r}_0$.

12.3.3 Metodo di Krylov

Un metodo di Krylov combina:

- Uno spazio di ricerca per il vettore soluzione:

$$\mathbf{x}_0 + \mathcal{K}_k(\mathbf{A}, \mathbf{r}_0); \quad (9)$$

- Un vincolo sul residuo.

Pwe quest'ultimo due scelte classiche sono:

- **Condizione di Galerkin:**

$$\mathbf{r}_k \perp \mathcal{K}_k(\mathbf{A}, \mathbf{r}_0); \quad (10)$$

- **Condizione di residuo minimo:**

$$\|\mathbf{r}_k\|_2 = \min_{\mathbf{x} \in \mathbf{x}_0 + \mathcal{K}_k(\mathbf{A}, \mathbf{r}_0)} \|\mathbf{b} - \mathbf{Ax}\|_2. \quad (11)$$

Il metodo **GMRES** (Generalized Minimal RESidual) appartiene alla seconda classe.

12.4 Da Conjugate Gradients a GMRES

12.4.1 Idea comune

L'idea comune è costruire un'approssimazione della soluzione in un sottospazio di Krylov.

Conjugate Gradients (CG)

- Applicabile solo a matrici **SPD** (Simmetriche e Definite Positive);
- Utilizza direzioni di ricerca **A-ortogonali**;
- Minimizza l'errore nella norma di **A**;
- Il costo computazionale è costante.

GMRES

- Estende i metodi di Krylov a matrici generali;
- Richiede una base ortonormale (metodo di Arnoldi);
- Minimizza il residuo reale;
- Il costo computazionale cresce con le iterazioni.

Regola pratica

Il metodo CG è ottimale quando applicabile; GMRES viene utilizzato quando CG non può essere applicato.

12.5 Metodo Generalized Minimal RESidual (GM-RES)

Dato un valore iniziale $\mathbf{x}_0$, all'iterazione k -esima l'approssimazione $\mathbf{x}_k$ è il vettore, nello spazio affine di Krylov $\mathbf{x}_0 + \mathcal{K}_k(\mathbf{A}, \mathbf{r}_0)$, che minimizza la norma euclidea del residuo:

$$\|\mathbf{r}_k\|_2 = \min_{\mathbf{x} \in \mathbf{x}_0 + \mathcal{K}_k(\mathbf{A}, \mathbf{r}_0)} \|\mathbf{b} - \mathbf{A}\mathbf{x}\|_2, \quad \mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0.$$

Sviluppato originariamente da Y. Saad e M.H. Schultz (SIAM J. SCI. STAT. COMPUT., Vol. 7, No. 3, Luglio 1986).

12.6 Metodo di Arnoldi

Il metodo di Arnoldi costruisce incrementalmente una base ortonormale di $\mathcal{K}_k(\mathbf{A}, \mathbf{r}_0)$:

- Al passo k produce un insieme $\mathbf{V}_k = [\mathbf{v}_1, \dots, \mathbf{v}_k]$, con

$$\mathbf{v}_i \cdot \mathbf{v}_j = \begin{cases} 0, & \forall i \neq j \\ 1, & \text{se } i = j \end{cases} \quad \text{con } i, j \in [1, \dots, k],$$

tale che

$$\text{span}(\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \dots, \mathbf{A}^{k-1}\mathbf{r}_0) = \text{span}(\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k).$$

12.6.1 Procedura

Partendo da $\mathbf{v}_1 = \frac{\mathbf{r}_0}{\|\mathbf{r}_0\|_2}$, in modo che $\|\mathbf{v}_1\|_2 = 1$ per $k = 1, 2, \dots$:

1. Calcolare: $\mathbf{w}_k = \mathbf{A}\mathbf{v}_k$;
2. Ortogonalizzare rispetto alla base corrente:

$$\text{per } i = 1, 2, \dots, k : \quad h_{ik} = (\mathbf{w}_k, \mathbf{v}_i), \quad (15)$$

$$\hat{\mathbf{v}}_{k+1} = \mathbf{w}_k - \sum_{i=1}^k h_{ik} \mathbf{v}_i; \quad (16)$$

3. Porre $h_{k+1,k} = \|\hat{\mathbf{v}}_{k+1}\|_2$;
4. Se $h_{k+1,k} = 0 \rightarrow$ FERMARSI (la base è completa);
5. Altrimenti, normalizzare il nuovo vettore:

$$\mathbf{v}_{k+1} = \frac{\hat{\mathbf{v}}_{k+1}}{h_{k+1,k}}. \quad (17)$$

$$\begin{aligned} h_{11} &= (\mathbf{w}_1, \mathbf{v}_1) = & \hat{\mathbf{v}}_2 &= \mathbf{w}_1 h_{11} \mathbf{v}_1| = \\ &= |\mathbf{w}_1| |\mathbf{v}_1| \cos(\alpha) = & &= |\mathbf{w}_1| + (-h_{11} \mathbf{v}_1) \\ &= |\mathbf{w}_1| \cos(\alpha) \end{aligned}$$

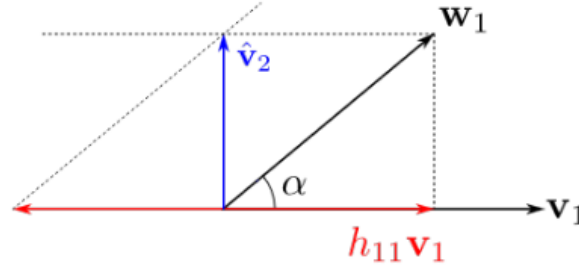


FIG. 12.1: Visualizzazione del primo step del processo di ortogonalizzazione di Arnoldi.

12.6.2 Forma matriciale

Combinando le formule riportate nella procedura, si ottiene:

$$h_{k+1,k} \mathbf{v}_{k+1} = \mathbf{A} \mathbf{v}_k - \sum_{i=1}^k h_{ik} \mathbf{v}_i ,$$

e quindi:

$$\mathbf{A} \mathbf{v}_k = \sum_{i=1}^k h_{ik} \mathbf{v}_i + h_{k+1,k} \mathbf{v}_{k+1} = \sum_{i=1}^{k+1} h_{ik} \mathbf{v}_i .$$

$$\mathbf{A} \mathbf{v}_1 = h_{11} \mathbf{v}_1 + h_{21} \mathbf{v}_2$$

$$\mathbf{A} \mathbf{v}_2 = h_{12} \mathbf{v}_1 + h_{22} \mathbf{v}_2 + h_{32} \mathbf{v}_3$$

$$\vdots$$

$$\mathbf{A} \mathbf{v}_k = h_{1k} \mathbf{v}_1 + h_{2k} \mathbf{v}_2 + \dots + h_{k+1,k} \mathbf{v}_{k+1}$$

In forma matriciale, si ha:

$$\mathbf{A} \mathbf{V}_k = \mathbf{V}_{k+1} \bar{\mathbf{H}}_k ,$$

dove $\bar{\mathbf{H}}_k$ è una matrice di Hessenberg superiore (con zeri sotto la prima sottodiagonale) e $\mathbf{V}_l$ (con $l = k, k+1$) è una matrice le cui colonne sono i vettori $\mathbf{v}_i$ con $i = 1, \dots, l$.

$$\begin{array}{c} \left[\begin{array}{cccccccccccccccc} x & x & x & x & x & x & x & x & x & x & x & x & x & x & x & x \\ x & x & x & x & x & x & x & x & x & x & x & x & x & x & x & x \\ x & x & x & x & x & x & x & x & x & x & x & x & x & x & x & x \\ x & x & x & x & x & x & x & x & x & x & x & x & x & x & x & x \\ x & x & x & x & x & x & x & x & x & x & x & x & x & x & x & x \\ x & x & x & x & x & x & x & x & x & x & x & x & x & x & x & x \\ x & x & x & x & x & x & x & x & x & x & x & x & x & x & x & x \\ x & x & x & x & x & x & x & x & x & x & x & x & x & x & x & x \\ x & x & x & x & x & x & x & x & x & x & x & x & x & x & x & x \\ x & x & x & x & x & x & x & x & x & x & x & x & x & x & x & x \\ x & x & x & x & x & x & x & x & x & x & x & x & x & x & x & x \end{array} \right] \cdot \left[\begin{array}{cccccc} x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \end{array} \right] = \left[\begin{array}{cccccc} x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \\ x & x & x & x & x & x \end{array} \right] \cdot \left[\begin{array}{cccccc} x & x & x & x & x & x \\ \color{blue}{x} & x & x & x & x & x \\ \color{red}{0} & \color{blue}{x} & x & x & x & x \\ \color{red}{0} & \color{red}{0} & \color{blue}{x} & x & x & x \\ \color{red}{0} & \color{red}{0} & \color{red}{0} & \color{blue}{x} & x & x \\ \color{red}{0} & \color{red}{0} & \color{red}{0} & \color{red}{0} & \color{blue}{x} & x \\ \color{red}{0} & \color{red}{0} & \color{red}{0} & \color{red}{0} & \color{red}{0} & \color{blue}{x} \end{array} \right] \\ \mathbf{A}_{m,m} & \mathbf{V}_{m,k} & \mathbf{V}_{m,k+1} & \mathbf{H}_{k+1,k} \end{array}$$

FIG. 12.2: Rappresentazione della forma matriciale.

12.7 Metodo GMRES

Il processo di Arnoldi descritto consente di riscrivere l'equazione iniziale nella forma:

$$\mathbf{x}_k = \mathbf{x}_0 + \mathbf{V}_k \mathbf{y}, \quad [\mathbf{x}_k, \mathbf{x}_0 \in \mathbb{R}^n; \mathbf{y} \in \mathbb{R}^k; \mathbf{V}_k \in \mathbb{R}^{n \times k}] .$$

Quindi, il residuo $\mathbf{r}_k$ diventa:

$$\mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k = \mathbf{r}_0 - \mathbf{A}\mathbf{V}_k\mathbf{y} .$$

Grazie all'equazione di $\mathbf{A}\mathbf{V}_k$, si ottiene:

$$\mathbf{r}_k = \mathbf{r}_0 - \mathbf{V}_{k+1}\bar{\mathbf{H}}_k\mathbf{y}, \quad [\mathbf{V}_{k+1} \in \mathbb{R}^{n \times (k+1)}; \bar{\mathbf{H}}_k \in \mathbb{R}^{(k+1) \times k}] .$$

12.7.1 Minimizzazione del residuo

Ponendo $\beta = \|\mathbf{r}_0\|_2$, si ha $\mathbf{v}_1 = \frac{\mathbf{r}_0}{\beta}$ e $\mathbf{r}_0 = \beta\mathbf{v}_1$. Ovviamente,

$$\mathbf{v}_1 = \mathbf{V}_{k+1}\mathbf{e}_1 ,$$

dove $\mathbf{e}_1$ è la prima colonna della matrice identità $\mathbf{I} \in \mathbb{R}^{(k+1) \times (k+1)}$. $\mathbf{r}_k$ può essere riscritta come:

$$\mathbf{r}_k = \beta\mathbf{V}_{k+1}\mathbf{e}_1 - \mathbf{V}_{k+1}\bar{\mathbf{H}}_k\mathbf{y} = \mathbf{V}_{k+1}(\beta\mathbf{e}_1 - \bar{\mathbf{H}}_k\mathbf{y}) .$$

Poiché $\mathbf{V}_{k+1}$ è una matrice con colonne ortonormali, preserva la lunghezza dei vettori a cui è applicata. Quindi, dall'equazione precedente, si ottiene:

$$\|\mathbf{r}_k\|_2 = \|\beta\mathbf{e}_1 - \bar{\mathbf{H}}_k\mathbf{y}\|_2 .$$

L'obiettivo di GMRES è minimizzare, a ogni passo, la norma del residuo $\mathbf{r}_k$, cioè trovare il vettore $\mathbf{y}_k$ tale che:

$$\mathbf{y}_k = \arg \min_{\mathbf{y} \in \mathbb{R}^k} \|\beta\mathbf{e}_1 - \bar{\mathbf{H}}_k\mathbf{y}\|_2 .$$

In conclusione, il grande sistema lineare sparso $\mathbf{A}\mathbf{x} = \mathbf{b}$ (con n equazioni e n incognite) è ridotto a successivi problemi di minimi quadrati (con $k+1$ equazioni e k incognite) di ordine crescente k . Se la convergenza è raggiunta dopo k^* step ($\|\mathbf{r}_{k^*}\|_2 < \epsilon$), la soluzione sarà $\mathbf{k}^* = \mathbf{x}_0 + \mathbf{V}_{k^*}\mathbf{y}_{k^*}$.

Listing 12.1: Algoritmo GMRES

```

r_0 = b - A * x_0;
beta = || r_0 ||_2;
v_1 = r_0 / beta;
FOR k = 1, ..., k DO:
    w = A * v_k;
    FOR i = 1, ..., k DO:
        h_ik = v_i^T * w;
        w -= h_ik * v_i;
    h_{k+1,k} = || w ||_2;
    IF h_{k+1,k} = 0 THEN
        FERMARSI (lucky breakdown!);
    END IF
    v_{k+1} = w / h_{k+1,k};
    risolvere il problema ai minimi quadrati: min || beta e_1 - H_k y ||_2
    IF || r ||_2 < \epsilon THEN
        uscire dal ciclo for;
    END IF
END FOR
x_k = x_0 + V_k * y_k

```

12.8 GMRES con Riavvio (Restarted GMRES)

Aumentare k richiede:

- Maggiore memoria per immagazzinare i vettori della base ortonormale;
- Tempo di calcolo più lungo per eseguire l'ortogonalizzazione completa;
- Risolvere problemi ai minimi quadrati di ordine crescente k .

In aritmetica esatta, GMRES converge alla soluzione esatta in al più n passi, ma il costo computazionale cresce a livelli inaccettabili.

Strategia di riavvio

Si adotta la seguente strategia, nota come **GMRES**(k_0):

- Scegliere un intero fisso k_0 ;
- Se la convergenza non viene raggiunta in al più k_0 passi, calcolare $\mathbf{x}_{k_0}$;
- Riavviare il processo utilizzando $\mathbf{x}_{k_0}$ come nuova ipotesi iniziale.

Vantaggi

- Memoria limitata;
- Costo di ortogonalizzazione limitato.

Possibili svantaggi

- Convergenza più lenta o stagnazione a causa della perdita di informazioni spettrali accumulate durante i passi precedenti.

Compromesso

Un valore di riavvio più grande, k_0 , generalmente migliora la convergenza, ma aumenta la memoria e il costo per passo.

Takeaways

- Il GMRES completo è spesso più robusto;
- Il GMRES con riavvio è più pratico per problemi su larga scala.

12.9 Precondizionamento

In pratica, i metodi di Krylov vengono raramente utilizzati senza precondizionamento.

L'obiettivo del precondizionamento è sostituire il sistema originale con uno equivalente che abbia migliori proprietà numeriche per l'iterazione.

12.9.1 Scelte comuni

- **Precondizionamento a sinistra:** $\mathbf{M}^{-1}\mathbf{A}\mathbf{x} = \mathbf{M}^{-1}\mathbf{b}$;
- **Precondizionamento a destra:** $\mathbf{A}\mathbf{M}^{-1}\mathbf{y} = \mathbf{b}$, $\mathbf{x} = \mathbf{M}^{-1}\mathbf{y}$.

La matrice $\mathbf{M}^{-1}$ dovrebbe:

- Essere una buona approssimazione di $\mathbf{A}^{-1}$;
- Produrre un nuovo operatore con autovalori più raggruppati e lontani dall'origine;
- Essere economica da applicare.

12.9.2 Differenze tra precondizionamento a sinistra e a destra

Precondizionamento a sinistra

- Dalla definizione di cui sopra: $\tilde{\mathbf{r}}_0 = \mathbf{M}^{-1}(\mathbf{b} - \mathbf{A}\mathbf{x}_0) = \mathbf{M}^{-1}\mathbf{r}_0$;
- Il processo minimizza un residuo diverso, che potrebbe non riflettere accuratamente l'errore della soluzione reale.

Precondizionamento a destra:

- Introduce il cambio di variabile $\mathbf{x} = \mathbf{M}^{-1}\mathbf{y}$;
- Dalla definizione di cui sopra: $\tilde{\mathbf{r}}_0 = \mathbf{b} - \mathbf{A}\mathbf{M}^{-1}\mathbf{y}_0$, ma essendo $\mathbf{y}_0 = \mathbf{M}\mathbf{x}_0$, si ha $\tilde{\mathbf{r}}_0 \equiv \mathbf{r}_0$;
- Il processo minimizza il residuo corretto, permettendo di tenere sotto controllo l'errore della soluzione reale;
- Ottenuta la soluzione $\mathbf{y}$, è necessario risolvere il problema aggiuntivo $\mathbf{M}\mathbf{x} = \mathbf{y}$ per ottenere $\mathbf{x}$.

12.9.3 Esempi di precondizionamento

Precondizionamento di Jacobi

- $\mathbf{M} = \text{diag}(\mathbf{A})$;
- È il più semplice;
- Guadagno limitato;
- Adatto per matrici $\mathbf{A}$ diagonalmente dominanti;
- Avvicina $\mathbf{A}$ alla matrice identità.

Fattorizzazione LU incompleta (ILU)

- Trova $\mathbf{M} = \tilde{\mathbf{L}}\tilde{\mathbf{U}} \approx \mathbf{A}$;
- La versione senza fill-in, ILU(0), forza a rimanere zero gli elementi che sono già zero in $\mathbf{A}$ (sebbene in una fattorizzazione LU completa non lo sarebbero);

- Originariamente ideata per le matrici a 5 e 7 punti derivanti dalla discretizzazione di PDE ellittiche per problemi 2D e 3D.

Capitolo 13

Accelerazione del Metodo delle Potenze: Metodo di Wielandt

13.1 Convergenza del Metodo delle Potenze

La convergenza dell'autovettore fondamentale dipende dalla rapidità con cui gli autovettori di ordine superiore vengono “smorzati”.

L'iterazione del metodo delle potenze può essere espressa come:

$$\phi^{(n)} = \frac{k_1^{n-1}}{k^{(n-1)} \dots k^{(2)} k^{(1)}} \left[c_1 \phi_1 + \sum_{h \geq 2} c_h \left(\frac{k_h}{k_1} \right)^{n-1} \phi_h \right] .$$

In particolare, la velocità di convergenza dipende dal rapporto tra il secondo e il primo autovalore, detto dominance ratio:

$$\sigma = \frac{k_2}{k_1} .$$

Più σ è vicino a 1, più lenta è la convergenza.

13.1.1 Spostamento Spettrale

Un miglioramento della velocità di convergenza si ottiene riducendo σ , cioè separando maggiormente i primi due autovalori (spostamento spettrale). Consideriamo il problema agli autovalori:

$$\phi = \frac{1}{k} \mathbf{K} \phi \quad \Rightarrow \quad \mathbf{K} \phi = k \phi .$$

Supponiamo di conoscere una stima k^* dell'autovalore fondamentale k_1 , con $k^* > k_1$.

Sostituendo $\mathbf{K}$ con la matrice $(k^* \mathbf{I} - \mathbf{K})^{-1}$, il nuovo problema agli autovalori ha gli stessi autovettori e autovalori:

$$\lambda_h = \frac{1}{k^* - k_h} .$$

Si verifica che:

$$(k^*\mathbf{I} - \mathbf{K})^{-1}\phi_h = \lambda_h\phi_h ,$$

e quindi:

$$\mathbf{K}\phi_h = \left(k^* - \frac{1}{\lambda_h}\right)\phi_h .$$

Quindi $(k^*\mathbf{I} - \mathbf{K})^{-1}$ ha gli stessi autovalori di $\mathbf{K}$ e gli autovalori di quest'ultimo sono $k_h = (k^* - 1/\lambda_h)$.

I nuovi autovalori λ_1 e λ_2 sono:

$$\lambda_1 = \frac{1}{k^* - k_1}, \quad \lambda_2 = \frac{1}{k^* - k_2} .$$

Il nuovo dominance ratio σ^* risulta:

$$\sigma^* = \frac{\lambda_2}{\lambda_1} = \frac{k^* - k_1}{k^* - k_2} .$$

Per ottenere $\sigma^* < \sigma$, deve valere $\frac{k^* - k_1}{k^* - k_2} < \frac{k_2}{k_1}$ che si riduce a:

$$k^* < k_1 + k_2 .$$

13.2 Metodo di Wielandt

Consideriamo il problema agli autovalori generalizzato:

$$-(\mathbf{A} + \mathbf{R})\phi = \frac{1}{k}\mathbf{M}\phi .$$

Sottraendo $\frac{1}{k^*}\mathbf{M}\phi$ da entrambi i membri, otteniamo:

$$-\left(\mathbf{A} + \mathbf{R} + \frac{1}{k^*}\mathbf{M}\right)\phi = \left(\frac{1}{k} - \frac{1}{k^*}\right)\mathbf{M}\phi .$$

Riscrivendo, si arriva al problema iterativo equivalente:

$$\phi = -\left(\frac{1}{k} - \frac{1}{k^*}\right)\left(\mathbf{A} + \mathbf{R} + \frac{1}{k^*}\mathbf{M}\right)^{-1}\mathbf{M}\phi .$$

Possiamo definire un nuovo problema iterativo $\phi^{(n+1)} = \frac{1}{\lambda} \hat{\mathbf{K}} \phi^{(n)}$ dove la matrice iterativa $\hat{\mathbf{K}}$ ed il fattore λ sono definiti come:

$$\hat{\mathbf{K}} = - \left(\mathbf{A} + \mathbf{R} + \frac{1}{k^*} \mathbf{M} \right)^{-1} \mathbf{M} , \quad \lambda = \left(\frac{1}{k} + \frac{1}{k^*} \right)^{-1} .$$

I nuovi autovalori λ_1 e λ_2 sono:

$$\lambda_1 = \frac{k_1 k^*}{k^* - k_1}, \quad \lambda_2 = \frac{k_2 k^*}{k^* - k_2} .$$

Il nuovo dominance ratio σ^* diventa:

$$\sigma^* = \frac{\lambda_2}{\lambda_1} = \frac{k^* - k_1}{k^* - k_2} \cdot \frac{k_2}{k_1} = \frac{k^* - k_1}{k^* - k_2} \cdot \sigma .$$

Poiché $k_1 > k_2$, si ha

$$\frac{k^* - k_1}{k^* - k_2} < 1, \text{ quindi } \sigma^* < \sigma \text{ per ogni } k^* > k_1 .$$

Inoltre:

$$\lim_{k^* \rightarrow \infty} \sigma^* = \sigma .$$

Non esiste un limite superiore per k^* per ottenere un vantaggio.

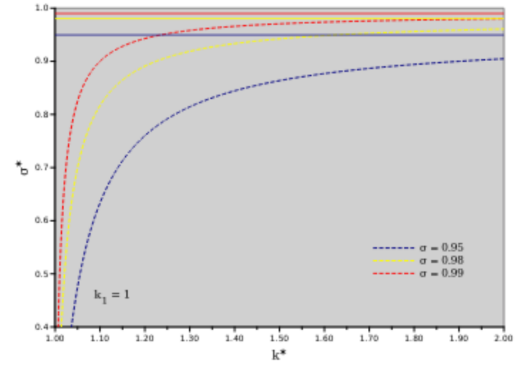


FIG. 13.1: convergenza di σ^* a σ .

13.2.1 Osservazioni Finali

- Il metodo di accelerazione di Wielandt può essere applicato sia a codici deterministici che stocastici.
- Purtroppo, la matrice a sinistra dell'equazione ad inizio dello scorso paragrafo non è più triangolare inferiore e non permette di disaccoppiare le equazioni di diffusione multigruppo.

Capitolo 14

Accelerazione del Metodo delle Potenze: Metodo di Chebyshev

Supponiamo che, dopo n^* iterazioni con il metodo delle potenze, la stima k^* dell'autovalore fondamentale sia considerata adeguata e non venga più aggiornata. Tuttavia, l'autovettore fondamentale potrebbe non essere ancora sufficientemente accurato. Per migliorarlo, si può continuare a iterare. Il metodo delle potenze procederebbe come segue:

$$\begin{aligned}\phi^{(n^*+1)} &= \frac{1}{k^*} \mathbf{K} \phi^{(n^*)} , \\ \phi^{(n^*+2)} &= \frac{1}{k^*} \mathbf{K} \phi^{(n^*+1)} = \left(\frac{1}{k^*} \mathbf{K} \right)^2 \phi^{(n^*)} , \\ &\vdots \\ \phi^{(n^*+n)} &= \left(\frac{1}{k^*} \mathbf{K} \right)^n \phi^{(n^*)} .\end{aligned}$$

In questo processo, il vettore $\phi^{(n^*)}$ è una sorta di nuovo vettore iniziale a cui vengono applicate le successive potenze della matrice $\frac{1}{k^*} \mathbf{K}$.

14.1 Polinomi Matriciali e Accelerazione

L'autovalore fondamentale di $\mathbf{K}/k^*$ è $\frac{k_1}{k^*} \approx 1$. Se, al posto della singola matrice $\left(\frac{1}{k^*} \mathbf{K} \right)^n$, si utilizza un polinomio matriciale di ordine n , $P_n(\mathbf{K}/k^*)$, si può ottenere una convergenza più rapida attraverso una scelta appropriata dei coefficienti del polinomio.

Il processo per $\phi^{(n^*+n)}$ viene sostituito da:

$$\phi^{(n^*+n)} = P_n \left(\frac{\mathbf{K}}{k^*} \right) \phi^{(n^*)} .$$

Espandendo il vettore $\phi^{(n^*)}$ in termini degli autovettori della matrice $\mathbf{K}$:

$$\phi^{(n^*)} = c_1 \varphi_1 + \sum_{h \geq 2} c_h \varphi_h .$$

14.1.1 Applicazione dei Polinomi di Chebyshev

Si può scrivere:

$$\phi^{(n^*+n)} = P_n\left(\frac{\mathbf{K}}{k^*}\right) \left[c_1 \varphi_1 + \sum_{h \geq 2} c_h \varphi_h \right] = c_1 P_n\left(\frac{k_1}{k^*}\right) \varphi_1 + \sum_{h \geq 2} c_h P_n\left(\frac{k_h}{k^*}\right) \varphi_h .$$

Dati tutti gli autovalori k_h ($h \geq 2$) di $\mathbf{K}$, scegliendo un ordine n sufficientemente alto del polinomio matriciale, si possono determinare i suoi coefficienti in modo che:

$$P_n\left(\frac{k_1}{k^*}\right) \approx P_n(1) = 1; \quad P_n\left(\frac{k_h}{k^*}\right) = 0 \quad \forall h \geq 2 .$$

Così l'equazione per $\phi^{(n^*+n)}$ si ridurrebbe a:

$$\phi^{(n^*+n)} = c_1 \varphi_1 .$$

In altre parole, con un numero finito di interazioni (un numero finito di potenze $\mathbf{K}/k^*$) raggiungeremmo la soluzione esatta.

Tuttavia, poiché la conoscenza di tutti gli autovalori di $\mathbf{K}$ è impraticabile, invece di cercare di annullare la somma

$$\sum_{h \geq 2} c_h P_n\left(\frac{k_h}{k^*}\right) \varphi_h ,$$

è meglio minimizzarla.

14.1.2 Minimizzazione del Residuo

Data una stima del dominance ratio σ e assumendo che tutti gli autovalori per $h \geq 2$ siano comunque non negativi, si ha:

$$0 \leq \frac{k_h}{k^*} \leq \sigma < 1 \quad \forall h \geq 2 .$$

Il problema di minimizzazione da risolvere è:

$$P_n(1) = 1; \quad \min_{a_0, a_1, \dots, a_n} \max_{0 \leq x \leq \sigma} |P_n(x)| .$$

La soluzione a questo problema è data dai seguenti polinomi:

$$P_n(x) = \frac{C_n\left(\frac{2x}{\sigma} - 1\right)}{C_n\left(\frac{2}{\sigma} - 1\right)} ,$$

dove C_n sono i polinomi di Chebyshev, definiti dalla relazione di ricorrenza: $C_0(t) = 1$, $C_1(t) = t$, $C_{n+1}(t) = 2tC_n(t) - C_{n-1}(t) \forall n \geq 2$.

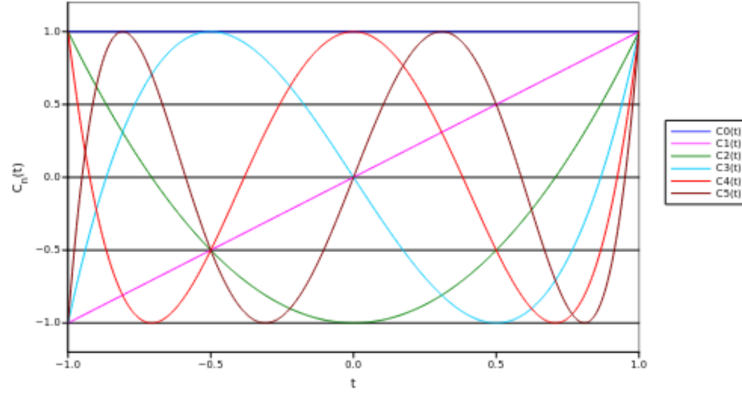


FIG. 14.1

14.1.3 Implementazione Iterativa

Grazie alle relazioni di ricorrenza, non è necessario calcolare il polinomio matriciale $P_n\left(\frac{\mathbf{K}}{k^*}\right)$ per tutti i valori di n come richiesto dall'equazione dei polinomi, ma si può procedere iterativamente:

$$\phi^{(n^*+i+1)} = \phi^{(n^*+i)} + \alpha_n \left(\frac{\mathbf{K}}{k^*} \phi^{(n^*+i)} - \phi^{(n^*+i)} \right) + \beta_n (\phi^{(n^*+i)} - \phi^{(n^*+i-1)}) ,$$

dove:

$$\alpha_0 = \frac{2}{2 - \sigma}; \quad \alpha_1 = \frac{2}{2 - \sigma - \sigma^2 \frac{\alpha_0}{4}}; \quad \alpha_n = \frac{2}{2 - \sigma - \sigma^2 \frac{\alpha_{n-1}}{8}}; \quad \beta_n = \left(1 - \frac{\sigma}{2}\right) \alpha_n - 1 .$$

Come si vede in figura, l'accelerazione del metodo di Chebyshev è molto funzionale per smorzare le armoniche più basse. Per armoniche di ordini superiori può risultare invece meno efficiente del metodo delle potenze stesso.

Nonostante questo, le n^* iterazioni del metodo delle potenze, applicato per ottenere una buona stima, k^* , dell'autovalore fondamentale, sono in genere sufficienti a sopprimere tutte le armoniche di ordine superiore al secondo.

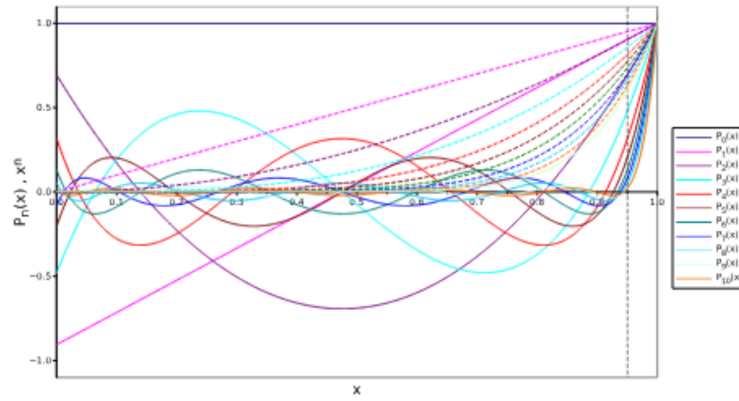


FIG. 14.2

14.1.4 Confronti e Stima del Dominance Ratio

Per applicare il metodo di accelerazione di Chebyshev, è necessario avere una stima del dominance ratio σ . Supponiamo che, dopo n^* iterazioni con il metodo delle potenze, il vettore $\phi^{(n^*)}$ sia approssimato da:

$$\phi^{(n^*)} = c_1 \varphi_1 + c_2 \varphi_2 \quad .$$

Introducendo il vettore:

$$\delta^{(n)} = \phi^{(n)} - \phi^{(n-1)} \quad ,$$

si ha:

$$\begin{aligned} \delta^{(n^*+1)} &= \phi^{(n^*+1)} - \phi^{(n^*)} = \frac{\mathbf{K}}{k^*} \phi^{(n^*)} - \phi^{(n^*)} = \\ &= \frac{\mathbf{K}}{k^*} [c_1 \varphi_1 + c_2 \varphi_2] - c_1 \varphi_1 - c_2 \varphi_2 = \\ &= c_1 \frac{k_1}{k^*} \varphi_1 + c_2 \frac{k_2}{k^*} \varphi_2 - c_1 \varphi_1 - c_2 \varphi_2 \approx c_2 (\sigma - 1) \varphi_2 \quad . \end{aligned}$$

Analogamente:

$$\begin{aligned} \delta^{(n^*+2)} &= \phi^{(n^*+2)} - \phi^{(n^*+1)} = \left(\frac{\mathbf{K}}{k^*} \right)^2 \phi^{(n^*)} - \frac{\mathbf{K}}{k^*} \phi^{(n^*)} = \\ &= \left(\frac{\mathbf{K}}{k^*} \right)^2 [c_1 \varphi_1 + c_2 \varphi_2] - \frac{\mathbf{K}}{k^*} [c_1 \varphi_1 + c_2 \varphi_2] = \\ &\approx c_2 \sigma (\sigma - 1) \varphi_2 \end{aligned}$$

La stima del dominance ratio si ottiene quindi come:

$$\boxed{\frac{\delta^{(n^*+2)}}{\delta^{(n^*+1)}} \approx |\sigma|} \quad .$$

Capitolo 15

Metodi dei Residui Pesati e Applicazioni ai Calcoli di Core

15.1 Introduzione al Metodo dei Residui Pesati

Il metodo dei residui pesati è una tecnica numerica utilizzata per approssimare la soluzione di problemi differenziali. Consideriamo il seguente problema differenziale alle derivate parziali:

$$L\phi(\mathbf{r}) + S(\mathbf{r}) = 0 ,$$

dove L è un operatore differenziale. Nel caso dell'equazione di diffusione, l'operatore L assume la forma:

$$L\cdot = \nabla \cdot D \nabla \cdot - \Sigma_a \cdot \cdot .$$

L'obiettivo è trovare una soluzione approssimata $\tilde{\phi}(\mathbf{r})$ espressa come combinazione lineare di funzioni di prova $u_j(\mathbf{r})$:

$$\tilde{\phi}(\mathbf{r}) = \sum_{j=1}^N c_j u_j(\mathbf{r}).$$

Le funzioni di prova $u_j(\mathbf{r})$ devono essere sufficientemente regolari per consentire l'applicazione dell'operatore L e devono soddisfare le condizioni al contorno del problema.

15.2 Formulazione del Metodo dei Residui Pesati

Sostituendo l'espressione di $\tilde{\phi}(\mathbf{r})$ nell'equazione originale, otteniamo un residuo $R(\mathbf{r})$:

$$L\tilde{\phi}(\mathbf{r}) + S(\mathbf{r}) = R(\mathbf{r}) \neq 0 .$$

Il metodo dei residui pesati richiede che il residuo sia "piccolo" in un senso integrale, imponendo che sia ortogonale $((R(\mathbf{r}), w_i(\mathbf{r})))$ a un insieme di N funzioni peso $w_i(\mathbf{r})$:

$$\int_V R(\mathbf{r}) w_i(\mathbf{r}) dV = 0, \quad i = 1, \dots, N.$$

Questa condizione può essere riscritta come:

$$\int_V \left(L \sum_{j=1}^N c_j u_j(\mathbf{r}) + S(\mathbf{r}) \right) w_i(\mathbf{r}) dV = 0.$$

Sviluppando l'integrale, otteniamo un sistema di N equazioni nelle N incognite c_j :

$$\sum_{j=1}^N c_j \int_V L u_j(\mathbf{r}) w_i(\mathbf{r}) dV + \int_V S(\mathbf{r}) w_i(\mathbf{r}) dV = 0.$$

In principio, le funzioni di prova $u_j(\mathbf{r})$ possono essere funzioni qualsiasi purché rispettino le condizioni al contorno, siano regolari ed è buona cosa che siano anche facili da differenziare ed integrare e che localmente presentino una forma simile alla soluzione esatta. Per questo spesso vengono usati polinomi di basso ordine.

15.3 Scelta delle Funzioni Peso

La scelta delle funzioni peso $w_i(\mathbf{r})$ caratterizza il metodo dei residui pesati. Tre scelte classiche sono:

- **Metodo di Galerkin:** Le funzioni peso coincidono con le funzioni di prova, cioè $w_i(\mathbf{r}) = u_i(\mathbf{r})$.
- **Metodo dei sottodomini:** Le funzioni peso sono definite come 1 all'interno di un sottodominio e 0 altrimenti.
- **Metodo di collocazione:** Le funzioni peso sono funzioni delta di Dirac, che impongono l'annullamento del residuo in punti specifici del dominio.

15.3.1 Metodo di Galerkin

Nel metodo di Galerkin, le funzioni peso sono scelte tra le funzioni di prova. Ad esempio, in un problema monodimensionale, se il flusso è approssimato da un polinomio di ordine N :

$$\tilde{\phi}(x) = \sum_{i=1}^N c_i u_i(x) = c_0 + c_1 x + c_2 x^2 + \cdots + c_N x^N,$$

le funzioni peso possono essere scelte come $w_i(x) = x^k$ per $k = 1, \dots, N$.

15.3.2 Metodo dei Sottodomini

Nel metodo dei sottodomini, il dominio di calcolo V viene suddiviso in sottodomini V_m . Le funzioni peso sono definite come:

$$w_i(\mathbf{r}) = \begin{cases} 1, & \text{se } \mathbf{r} \in V_m, \\ 0, & \text{altrimenti.} \end{cases}$$

In ogni sottodominio, si impone che l'integrale del residuo sia nullo:

$$\int_{V_m} R(\mathbf{r}) dV = 0.$$

Applicazione del Metodo dei Sottodomini all'Equazione di Diffusione

Nel caso dell'equazione di diffusione, il metodo dei sottodomini equivale a imporre il bilancio neutronico sul volume V_m :

$$\int_{V_m} R(\mathbf{r}) dV = 0 \implies \int_{V_m} [\nabla \cdot D \nabla \tilde{\phi} - \Sigma_a \tilde{\phi} + S] dV = 0 .$$

Sviluppando l'integrale, si ottiene:

$$\int_{S_m} D \nabla \tilde{\phi} \cdot \mathbf{n} dA - \int_{V_m} \Sigma_a \tilde{\phi} dV + \int_{V_m} S dV = 0 .$$

Questa equazione può essere interpretata come un bilancio tra le perdite per fuga (leakages), le perdite per assorbimento (absorptions) e la sorgente (source):

$$\underbrace{\int_{S_m} \mathbf{J} \cdot \mathbf{n} dA}_{\text{Perdite per fuga}} + \underbrace{\int_{V_m} \Sigma_a \tilde{\phi} dV}_{\text{Perdite per assorbimento}} = \underbrace{\int_{V_m} S dV}_{\text{Sorgente}} .$$

Questo è un requisito fondamentale che deve essere soddisfatto da qualsiasi schema numerico.

Tuttavia, il bilancio neutronico deve essere garantito anche sull'intero dominio di calcolo V , imponendo opportune condizioni di continuità della corrente attraverso le interfacce dei sottodomini. Questo assicura che il metodo dei sottodomini non solo approssimi localmente la soluzione, ma preservi anche la coerenza fisica globale del sistema.

15.3.3 Metodo di Collocazione

Nel metodo di collocazione, le funzioni peso sono funzioni delta di Dirac:

$$w_i(\mathbf{r}) = \delta(\mathbf{r} - \mathbf{r}_i) .$$

Questo metodo impone che il residuo sia nullo in punti specifici $\mathbf{r}_i$:

$$\int_V R(\mathbf{r}) \delta(\mathbf{r} - \mathbf{r}_i) dV = R(\mathbf{r}_i) = 0 .$$

Allora il metodo di collocazione è equivalente ad imporre che il residuo sia nullo in una certa posizione $\mathbf{r}_i$, *i.e.* la soluzione approssimata risolve esattamente l'equazione differenziale in quel punto.

Fissare il residuo a zero in $\mathbf{r}_i$ non vuol dire che non vi sia errore in quel punto. Per mandare l'errore a zero servirà prendere sempre più funzioni nella sequenza di approssimazione $\tilde{\phi}$.

15.4 Applicazione al Calcolo del Core del Reattore

Il core di un reattore nucleare è caratterizzato da una geometria complessa, che può essere semplificata attraverso un processo di omogeneizzazione. Questo processo trasforma la geometria reale in una serie di celle omogeneizzate, che possono essere trattate come nodi in una griglia grossolana. Ogni nodo rappresenta una sezione assiale di un'assemblaggio, e il flusso neutronico viene approssimato all'interno di ogni nodo utilizzando polinomi.

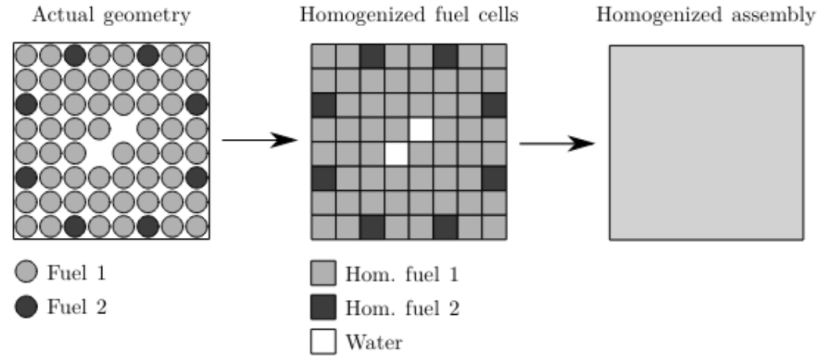


FIG. 15.1

15.5 Applicazione del Metodo dei Sottodomini

15.5.1 Formulazione del Bilancio Neutronico

Applicando il metodo dei sottodomini, si impone che l'integrale del residuo su ogni volume V_{ijl} sia nullo:

$$\int_{V_{ijl}} \left[\nabla \cdot D \nabla \phi - \Sigma_a \phi + \frac{\nu}{k} \Sigma_f \phi \right] dV = 0.$$

Questa condizione equivale a imporre il bilancio neutronico su ogni sottodominio V_{ijl} . Scomponendo l'integrale, si ottiene:

$$\int_{V_{ijl}} \nabla \cdot D \nabla \phi dV = \int_{\partial V_{ijl}} D \nabla \phi \cdot \mathbf{n} dA.$$

Sviluppando ulteriormente, si arriva a espressioni che coinvolgono le derivate del flusso alle interfacce del nodo, ad esempio:

$$\int_{V_{ijl}} \nabla \cdot D \nabla \phi dV = D_{ijl} \left[\left. \frac{\partial \phi}{\partial x} \right|_{x_{i+\frac{1}{2}}} - \left. \frac{\partial \phi}{\partial x} \right|_{x_{i-\frac{1}{2}}} \right] h_{y_j} h_{z_l} + \dots$$

dove $\frac{\partial \phi}{\partial x}$ è espresso in termini dei coefficienti $a_{x,ijl}$ e $b_{x,ijl}$:

$$\frac{\partial \phi}{\partial x} = \phi_{ijl} \left[\frac{a_{x,ijl}}{h_{x_i}} + \frac{2(x - x_i)}{h_{x_i}^2} b_{x,ijl} \right].$$

Sostituendo e semplificando, si ottiene:

$$\int_{V_{ijl}} \nabla \cdot D \nabla \phi dV = \phi_{ijl} D_{ijl} V_{ijl} \frac{2b_{x,ijl}}{h_{x_i}^2} + \dots$$

dove $V_{ijl} = h_{x_i} h_{y_j} h_{z_l}$.

15.5.2 Integrazione del Termine di Assorbimento e Sorgente

Il termine di assorbimento e sorgente viene integrato come segue:

$$\int_{V_{ijl}} \left(\frac{\nu}{k} \Sigma_f - \Sigma_a \right) \phi \, dV = \left(\frac{\nu}{k} \Sigma_f - \Sigma_a \right)_{ijl} \int_{V_{ijl}} \phi \, dV.$$

Utilizzando la trasformazione di coordinate $dV = V_{ijl} \, d\xi \, d\eta \, d\zeta$, l'integrale del flusso diventa:

$$\int_{V_{ijl}} \phi \, dV = \phi_{ijl} V_{ijl} \left(1 + \frac{b_{x,ijl} + b_{y,ijl} + b_{z,ijl}}{12} \right).$$

15.5.3 Equazione Discretizzata

Combinando i risultati precedenti, si ottiene l'equazione discretizzata:

$$2D_{ijl} \left(\frac{b_{x,ijl}}{h_{x_i}^2} + \frac{b_{y,ijl}}{h_{y_i}^2} + \frac{b_{z,ijl}}{h_{z_l}^2} \right) \phi_{ijl} + \left(\frac{\nu}{k} \Sigma_f - \Sigma_a \right)_{ijl} \left(1 + \frac{b_{x,ijl} + b_{y,ijl} + b_{z,ijl}}{12} \right) \phi_{ijl} = 0.$$

I coefficienti b sono espressi in termini dei flussi alle interfacce:

$$b_{x,ijl} = 2 \frac{\phi_{i+\frac{1}{2},jl} - 2\phi_{ijl} + \phi_{i-\frac{1}{2},jl}}{\phi_{ijl}}, \quad b_{y,ijl} = 2 \frac{\phi_{i,j+\frac{1}{2},l} - 2\phi_{ijl} + \phi_{i,j-\frac{1}{2},l}}{\phi_{ijl}}, \quad b_{z,ijl} = 2 \frac{\phi_{ij,l+\frac{1}{2}} - 2\phi_{ijl} + \phi_{ij,l-\frac{1}{2}}}{\phi_{ijl}}.$$

15.5.4 Condizioni di Continuità della Corrente

Per garantire il bilancio neutronico globale, è necessario imporre la continuità della corrente alle interfacce dei nodi. Ad esempio, per l'interfaccia a $x_{i+\frac{1}{2}}$:

$$-D_{ijl} \frac{\partial \phi}{\partial x} \Big|_{x_{i+\frac{1}{2}}} = -D_{i+1,jl} \frac{\partial \phi}{\partial x} \Big|_{x_{i+1-\frac{1}{2}}}.$$

Sostituendo le espressioni per $\frac{\partial \phi}{\partial x}$ e semplificando, si ottiene una relazione per il flusso all'interfaccia:

$$\phi_{i+\frac{1}{2},jl} = \frac{\left(4 - \frac{\phi_{i-\frac{1}{2},jl}}{\phi_{ijl}} \right)}{3 \left(1 + \frac{D_{i+1,jl}}{D_{ijl}} \frac{h_{x_i}}{h_{x_{i+1}}} \right)} \phi_{ijl} + \frac{\left(4 - \frac{\phi_{i+\frac{3}{2},jl}}{\phi_{i+1,jl}} \right)}{3 \left(1 + \frac{D_{ijl}}{D_{i+1,jl}} \frac{h_{x_{i+1}}}{h_{x_i}} \right)} \phi_{i+1,jl}.$$

Questa relazione può essere scritta in forma compatta come:

$$\phi_{i+\frac{1}{2},jl}^{(n)} = \alpha_{(i,i+1),jl}^{(n)} \phi_{ijl}^{(n)} + \beta_{(i,i+1),jl}^{(n)} \phi_{i+1,jl}^{(n)}.$$

15.5.5 Equazione a 7 Punti

Sostituendo le espressioni per i flussi alle interfacce nell'equazione discretizzata, si ottiene un'equazione a 7 punti:

$$O_{ijl} \phi_{ijl} + W_{ijl} \phi_{i-1,jl} + E_{ijl} \phi_{i+1,jl} + S_{ijl} \phi_{i,j-1,l} + N_{ijl} \phi_{i,j+1,l} + D_{ijl} \phi_{ij,l-1} + U_{ijl} \phi_{ij,l+1} = 0.$$

I coefficienti $O_{ijl}, W_{ijl}, \dots$ sono più complessi rispetto al caso delle differenze finite, ma permettono di ottenere una maggiore accuratezza con un numero minore di nodi.

15.6 Metodo CUBBOX

15.6.1 Approssimazione del Flusso con Polinomi di Terzo Ordine

Nel metodo CUBBOX, il flusso in ogni nodo è approssimato da un polinomio di terzo ordine:

$$\phi(x, y, z) = \phi(\xi, \eta, \zeta) = \phi_{ijl} \left(1 + a_{x,ijl}\xi + b_{x,ijl}\xi^2 + c_{x,ijl}\xi \left(\xi^2 - \frac{1}{4} \right) + \dots \right).$$

Il termine aggiuntivo $c_{x,ijl}\xi \left(\xi^2 - \frac{1}{4} \right)$ è nullo alle interfacce del nodo e non contribuisce all'integrale del residuo, quindi l'equazione discretizzata rimane valida.

15.6.2 Determinazione dei Coefficienti c_x, c_y, c_z

I coefficienti c_x, c_y, c_z sono determinati applicando nuovamente il metodo dei residui pesati con pesi di Galerkin:

$$w_1 = \xi \left(\xi^2 - \frac{1}{4} \right), \quad w_2 = \eta \left(\eta^2 - \frac{1}{4} \right), \quad w_3 = \zeta \left(\zeta^2 - \frac{1}{4} \right).$$

Si ottengono così tre equazioni aggiuntive:

$$-6 \frac{D_{ijl}}{h_{x_i}^2} c_x - \left(\frac{\nu \Sigma_{f,ijl}}{k} - \Sigma_{a,ijl} \right) a_x + \frac{1}{7} \left(\frac{\nu \Sigma_{f,ijl}}{k} - \Sigma_{a,ijl} \right) c_x = 0,$$

e analogamente per c_y e c_z . Imponendo le condizioni di continuità della corrente alle interfacce, si ottengono relazioni simili a quelle del metodo QUABOX, ma con coefficienti α e β ancora più complessi.

15.7 Conclusione

I metodi a maglia grossolana (coarse mesh) permettono di ottenere una distribuzione del flusso più accurata rispetto alle tecniche a volumi finiti, pur richiedendo un costo computazionale maggiore per nodo. Tuttavia, è possibile utilizzare un numero minore di nodi per ottenere la stessa accuratezza, riducendo così il tempo di calcolo complessivo. Le equazioni si riducono a schemi a 3, 5 o 7 punti a seconda della dimensionalità del problema.

Capitolo 16

Approssimanti di Padé

16.1 Introduzione

Gli **approssimanti di Padé** rappresentano la migliore approssimazione di una funzione $f(\theta)$ ottenibile tramite una funzione razionale di ordine assegnato. Essi sono definiti dalla relazione:

$$R_{(S,T)}(\theta) = \frac{\sum_{i=0}^T p_i \theta^i}{1 + \sum_{k=1}^S q_k \theta^k} = \frac{p_0 + p_1 \theta + p_2 \theta^2 + \cdots + p_T \theta^T}{1 + q_1 \theta + q_2 \theta^2 + \cdots + q_S \theta^S} .$$

I coefficienti p_i e q_k sono determinati in modo da soddisfare, al massimo ordine possibile, le seguenti identità:

$$\left. \frac{d^n R}{d\theta^n} \right|_0 = \left. \frac{d^n f}{d\theta^n} \right|_0 \quad (n = 0, \dots, S + T)$$

16.2 Applicazione alla funzione esponenziale

16.2.1 Approssimante $R_{(0,1)}(\theta)$

Consideriamo la funzione esponenziale e^θ e definiamo l'approssimante $R_{(0,1)}(\theta)$ come:

$$R_{(0,1)}(\theta) = p_0 + p_1 \theta$$

Imponendo le condizioni di approssimazione di cui sopra:

$$R_{(0,1)}(0) = e^\theta \Big|_0 \implies p_0 = 1$$

$$\left. \frac{dR_{(0,1)}}{d\theta} \right|_0 = \left. \frac{de^\theta}{d\theta} \right|_0 \implies p_1 = 1$$

Si ottiene quindi:

$$R_{(0,1)}(\theta) = 1 + \theta$$

16.2.2 Approssimante $R_{(1,0)}(\theta)$

Definiamo ora l'approssimante $R_{(1,0)}(\theta)$ come:

$$R_{(1,0)}(\theta) = \frac{p_0}{1 + q_1\theta}$$

Imponendo le condizioni di approssimazione:

$$R_{(1,0)}(0) = e^\theta \Big|_0 \implies p_0 = 1$$

$$\frac{dR_{(1,0)}}{d\theta} \Big|_0 = \frac{de^\theta}{d\theta} \Big|_0 \implies -\frac{p_0 q_1}{(1 + q_1\theta)^2} \Big|_0 = 1 \implies q_1 = -1$$

Si ottiene quindi:

$$R_{(1,0)}(\theta) = \frac{1}{1 - \theta}$$

16.2.3 Approssimante $R_{(1,1)}(\theta)$

Definiamo infine l'approssimante $R_{(1,1)}(\theta)$ come:

$$R_{(1,1)}(\theta) = \frac{p_0 + p_1\theta}{1 + q_1\theta}$$

Imponendo le condizioni di approssimazione:

$$R_{(1,1)}(0) = e^\theta \Big|_0 \implies p_0 = 1$$

$$\begin{aligned} \frac{dR_{(1,1)}}{d\theta} \Big|_0 &= \frac{de^\theta}{d\theta} \Big|_0 \implies \frac{p_1 - q_1}{(1 + q_1\theta)^2} \Big|_0 = 1 \\ \frac{d^2 R_{(1,1)}}{d\theta^2} \Big|_0 &= \frac{d^2 e^\theta}{d\theta^2} \Big|_0 \implies -\frac{2(1 + q_1\theta)q_1(p_1 - q_1)}{(1 + q_1\theta)^4} \Big|_0 = 1 \end{aligned}$$

Dalle equazioni precedenti si ricava il sistema:

$$\begin{aligned} p_1 - q_1 &= 1 \\ -2q_1(p_1 - q_1) &= 1 \end{aligned}$$

La cui soluzione è: $p_1 = \frac{1}{2}$, $q_1 = -\frac{1}{2}$.

Si ottiene quindi:

$$R_{(1,1)}(\theta) = \frac{1 + \frac{1}{2}\theta}{1 - \frac{1}{2}\theta}$$